

Democratización tecnológica y agéntica

Diseño, implementación y evaluación de un sistema inteligente multimodal para asistencia personal ejecutado en hardware de consumo.

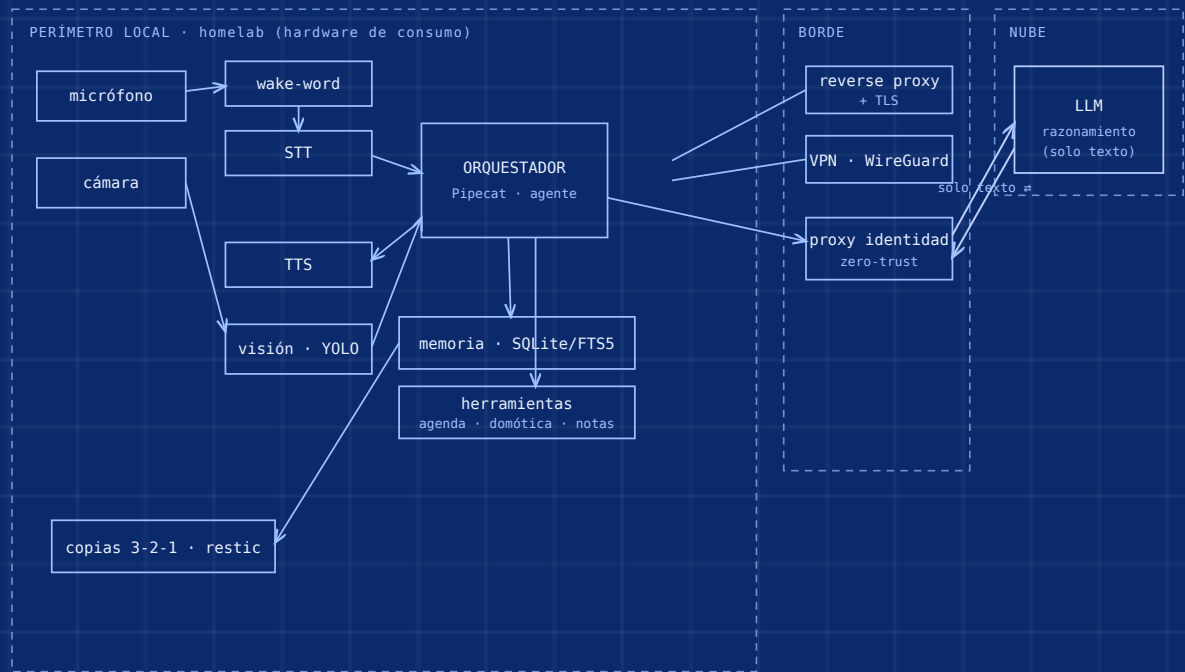


FIG. A – flujo: voz/visión → orquestador → memoria/herramientas · acceso por borde zero-trust · razonamiento delegado (solo texto) a la nube

Índice

Resumen	8
1 Introducción	9
1.1 El problema: la dependencia de la nube	9
1.2 Pregunta y tesis central	9
1.3 Alcance	10
1.4 Metodología	11
1.5 Estructura del documento	11
2 Fundamentos: soberanía, datos y el problema de la nube	12
2.1 La economía de la nube: renta, dependencia y vigilancia	13
2.1.1 De la propiedad a la renta	13
2.1.2 Costes de cambio y dependencia	13
2.1.3 El espejismo de lo gratuito	14
2.1.4 Externalidades y centralización del poder	14
2.2 Soberanía digital y propiedad de los datos	15
2.2.1 Propiedad y custodia frente a alojamiento	16
2.2.2 Jurisdicción y control efectivo	16
2.2.3 Propiedad de los datos como haz de facultades	17
2.3 El paradigma <i>local-first</i>	17
2.3.1 Los siete principios	18
2.3.2 CRDTs y sincronización	18
2.3.3 Relación con offline-first y con el autoalojamiento	19
2.4 Privacidad, confidencialidad y minimización	20
2.4.1 Minimización de datos	20
2.4.2 Cifrado: reposo, tránsito y extremo a extremo	21
2.4.3 El marco del RGPD	21
2.4.4 Superficie de exposición personal	22
2.5 Coste total de propiedad: nube frente a infraestructura propia	22
2.5.1 Componentes del coste propio	22
2.5.2 El coste oculto del tiempo humano	23
2.5.3 Punto de equilibrio y cuándo la nube resulta más barata	23
2.6 Longevidad, lock-in y obsolescencia	24
2.6.1 Las dos caras del lock-in	24
2.6.2 Formatos abiertos como condición de supervivencia	25
2.6.3 Portabilidad y discontinuación de servicios	25

2.7	Honestidad intelectual: cuándo el <i>self-hosting</i> no compensa	26
2.7.1	Fiabilidad y disponibilidad	26
2.7.2	Responsabilidad operativa	27
2.7.3	Seguridad mal hecha es peor que la delegada	27
2.7.4	No para todos, ni para todo	27
2.7.5	Criterios para decidir	28
2.8	Marco legal y regulatorio: RGPD, residencia de datos y NIS2	28
2.8.1	RGPD y autodeterminación informativa	28
2.8.2	Residencia y jurisdicción de los datos	29
2.8.3	Responsable y encargado del tratamiento	29
2.8.4	NIS2 y obligaciones de seguridad	30
2.8.5	Límites legales del autoalojamiento	30
3	Panorama del <i>self-hosting</i>	31
3.1	Una taxonomía de servicios autoalojables	31
3.1.1	Clasificación por dominio y criticidad	32
3.1.2	Servicios de datos, de cómputo y de inteligencia artificial	32
3.1.3	Criterios de madurez y mantenibilidad	33
3.2	Almacenamiento y archivos: NAS y Nextcloud	34
3.2.1	Sistemas de ficheros, RAID y ZFS	34
3.2.2	Copias de seguridad	35
3.2.3	Nextcloud y la sincronización de archivos	35
3.2.4	Alternativas: Syncthing y Seafile	35
3.2.5	Fotografías y documentos	36
3.3	Multimedia: Jellyfin y el ecosistema <i>arr</i>	36
3.3.1	Servidores de medios: del catálogo al reproductor	36
3.3.2	Automatización: el ecosistema <i>arr</i>	37
3.3.3	Transcodificación: el cuello de botella computacional	37
3.3.4	Consideraciones legales y éticas	38
3.4	Domótica: Home Assistant y la automatización local	38
3.4.1	El dilema entre nube y cómputo local	39
3.4.2	Protocolos: el cómputo local como condición de interoperabilidad	39
3.4.3	Privacidad de los sensores y minimización de la exposición	40
3.4.4	Automatizaciones y agencia agéntica	40
3.5	Comunicaciones: correo, Matrix y mensajería	41
3.5.1	El correo electrónico: el caso difícil	41
3.5.2	Mensajería en tiempo real: Matrix y XMPP	42
3.5.3	Federación y compromisos	42
3.6	Conocimiento y productividad	43
3.6.1	Notas y bases de conocimiento	43
3.6.2	Marcadores, RSS y agregación	44
3.6.3	El grafo de conocimiento personal	44

3.7	Identidad, contraseñas y secretos	45
3.7.1	Gestores de contraseñas autoalojados	45
3.7.2	Inicio de sesión único doméstico	45
3.7.3	El problema del secreto raíz	46
3.7.4	Identidad propia	46
3.8	Evaluación de la IA local: calidad, latencia y límites del hardware	47
3.8.1	Calidad: del benchmark genérico a la evaluación por tareas	47
3.8.2	Latencia y caudal: TTFB y tokens por segundo	47
3.8.3	Memoria y los compromisos de la cuantización	48
3.8.4	Voz: WER y la medida de la síntesis	48
3.8.5	Los límites del hardware de consumo y el criterio de delegación	49
3.9	Inteligencia local: LLMs, voz y visión en casa	49
3.9.1	Modelos de lenguaje locales: cuantización y hardware	50
3.9.2	Voz: reconocimiento, síntesis y palabra de activación	50
3.9.3	Visión local	51
3.9.4	El coste y el beneficio de la agencia autoalojada	51
3.10	Interoperabilidad y protocolos abiertos	52
3.10.1	Estándares como condición de la soberanía	52
3.10.2	El fediverso y la federación	52
3.10.3	Datos personales y formatos portables	53
3.10.4	Protocolos para la IA agéntica local	53
3.10.5	El protocolo frente al jardín vallado	54
4	Arquitectura y operación	55
4.1	Hardware y dimensionamiento; energía	55
4.1.1	Del mini-PC al rack	56
4.1.2	CPU, memoria, almacenamiento y GPU	56
4.1.3	Consumo y coste energético	57
4.1.4	Ruido y ubicación	57
4.1.5	Segunda mano y sostenibilidad	58
4.2	Contenedores y orquestación	58
4.2.1	Docker y Compose como unidad de despliegue	58
4.2.2	Aislamiento y sus límites	59
4.2.3	Podman como alternativa sin demonio	59
4.2.4	Orquestadores ligeros	60
4.2.5	Cuándo no usar Kubernetes	60
4.3	Redes domésticas: segmentación, DNS, descubrimiento	61
4.3.1	Segmentación: VLAN y aislamiento del IoT	61
4.3.2	DNS local	61
4.3.3	Descubrimiento de servicios	62
4.3.4	IPv6	62
4.3.5	Cortafuegos doméstico	63

4.4	Exposición segura: reverse proxy, túneles y WireGuard	63
4.4.1	El proxy inverso y el cifrado en tránsito	64
4.4.2	Túneles de salida: exponer sin abrir puertos	65
4.4.3	Red privada virtual: WireGuard y mallas cifradas	66
4.4.4	Criterio de exposición	66
4.5	Datos y estado: bases de datos y persistencia	66
4.5.1	Volúmenes y el estado fuera del contenedor	67
4.5.2	Postgres y SQLite: dos topologías de persistencia	67
4.5.3	Migraciones de esquema	68
4.5.4	Integridad y durabilidad	68
4.5.5	El estado como activo crítico	68
4.6	Copias de seguridad y recuperación (3-2-1)	69
4.6.1	La regla 3-2-1 como mínimo estructural	69
4.6.2	Cifrado e integridad	70
4.6.3	Objetivos de recuperación en el ámbito doméstico	70
4.6.4	El valor de una copia se mide en la restauración	71
4.7	Observabilidad: logs, métricas y salud	71
4.7.1	Logs centralizados	71
4.7.2	Métricas	72
4.7.3	Comprobaciones de salud y disponibilidad	72
4.7.4	Alertas	73
4.8	Reproducibilidad e infraestructura como código	73
4.8.1	Configuración declarativa frente a procedimental	74
4.8.2	Ansible y Compose bajo control de versiones	74
4.8.3	GitOps doméstico	75
4.8.4	Recuperación ante desastres y documentación como código	75
4.9	Migración: entrar y salir de la nube y entre soluciones	76
4.9.1	Salir de la nube: exportación, portabilidad y sus límites	76
4.9.2	Migrar entre soluciones de autoalojamiento	77
4.9.3	Planificación, ventana de corte y validación	77
4.9.4	El lock-in como riesgo operativo y el diseño que lo previene	78
5	Seguridad y amenazas	79
5.1	Modelo de amenazas del <i>homelab</i>	79
5.1.1	Adversarios realistas	80
5.1.2	Activos a proteger	81
5.1.3	STRIDE adaptado al <i>homelab</i>	81
5.1.4	Riesgo proporcional	82
5.2	Superficie de ataque y exposición a Internet	82
5.2.1	Qué se expone y por qué	82
5.2.2	La realidad de la exploración automatizada	83
5.2.3	El principio de no exponer	83
5.2.4	Segmentación y DMZ doméstica	84

5.3	Endurecimiento: mínimo privilegio y aislamiento	84
5.3.1	Usuarios no privilegiados en contenedores	85
5.3.2	Capacidades y restricción de privilegios	85
5.3.3	Seccomp y AppArmor: confinamiento del comportamiento	85
5.3.4	Aislamiento entre servicios	86
5.3.5	Superficie mínima	86
5.4	Gestión de secretos y credenciales	87
5.4.1	El fichero .env monolítico como antipatrón	87
5.4.2	Secretos por servicio y por ámbito	88
5.4.3	Rotación	88
5.4.4	Bóvedas de secretos	88
5.4.5	Secretos en el control de versiones y fugas	89
5.5	Identidad, autenticación y zero-trust	89
5.5.1	De la contraseña a la autenticación fuerte	89
5.5.2	Zero-trust llevado al hogar	90
5.5.3	El proxy de identidad como punto de control	91
5.6	Cadena de suministro y parches	92
5.6.1	Procedencia y confianza en las imágenes	92
5.6.2	Dependencias y vulnerabilidades conocidas	93
5.6.3	Actualización automática frente a actualización controlada	93
5.7	IA local: prompt injection, exfiltración y agencia	94
5.7.1	La inyección de instrucciones como vector primario	94
5.7.2	De la inyección a la exfiltración mediante herramientas	95
5.7.3	El diputado confundido y la amplificación por la agencia	95
5.7.4	Implicaciones para el diseño y enlace con la gobernanza	96
5.8	Continuidad y resiliencia	97
5.8.1	Modos de fallo en infraestructura doméstica	97
5.8.2	Degradación elegante	97
5.8.3	Plan de recuperación	98
6	Gobernanza y agencia segura	99
6.1	De la automatización a la agencia	99
6.1.1	El espectro de autonomía	100
6.1.2	Por qué la IA local doméstica cruza la línea	101
6.2	Riesgos de la agencia: permisos y acciones irreversibles	102
6.2.1	Acciones con efectos en el mundo	102
6.2.2	Irreversibilidad	102
6.2.3	Sobre-permisos	103
6.2.4	El agente como diputado confundido	103
6.2.5	Necesidad de límites	104
6.3	Patrones de contención: confirmación, capabilities y sandboxing	104
6.3.1	Confirmación humana como punto de control	105
6.3.2	Capabilities declarativas y diseño fail-closed	105

6.3.3	Sandboxing: worktrees y contenedores	106
6.3.4	Credenciales efímeras y minimización del secreto	106
6.3.5	Defensa en profundidad	108
6.4	Trazabilidad: auditoría inmutable, aprobaciones y rollback	108
6.4.1	Registro de auditoría inmutable	108
6.4.2	Colas de aprobación	109
6.4.3	Revisiones reversibles y rollback	110
6.5	Gobernanza del dato y del aprendizaje	110
6.5.1	Qué recuerda y qué olvida la memoria del agente	111
6.5.2	Consentimiento y trazabilidad del aprendizaje	111
6.5.3	Decaimiento y archivo	112
6.5.4	Aprendizaje supervisado por el usuario	112
6.6	El humano en el bucle	113
6.6.1	Diseño centrado en el ser humano	113
6.6.2	Cuándo pedir permiso	113
6.6.3	Presencia, contexto y transparencia	114
6.6.4	Explicabilidad y confianza calibrada	114
7	Caso aplicado: un asistente de voz agéntico autoalojado	115
7.1	Contexto, objetivos y restricciones	115
7.1.1	El asistente de voz como caso límite	116
7.1.2	Objetivos	116
7.1.3	Restricciones	117
7.2	Arquitectura del asistente	117
7.2.1	Voz local: palabra de activación, transcripción y síntesis	118
7.2.2	El cerebro: un modelo de lenguaje como motor de decisión	119
7.2.3	Herramientas: del lenguaje a la acción	119
7.2.4	El canal de texto: una interfaz secundaria de mensajería	120
7.2.5	Orquestación en contenedores	120
7.2.6	Síntesis: la encarnación de los principios	120
7.3	Agencia segura en la práctica	121
7.3.1	Delegación con confirmación y mediación de acciones	121
7.3.2	El guardián y la lista de denegación	121
7.3.3	Privilegio acotado en la ejecución	122
7.3.4	Auditoría inmutable y reversión	122
7.3.5	Reposo de recursos e inyección contenida	123
7.4	Resultados y lecciones aprendidas	123
7.4.1	Qué valida el marco	123
7.4.2	Qué lo tensiona	124
7.4.3	Honestidad sobre los límites	125
7.4.4	Generalización a otros despliegues autoalojados	125

8	Discusión y futuro	126
8.1	El factor humano: carga operativa, aprendizaje y factor bus	126
8.1.1	La curva de aprendizaje y la carga operativa	127
8.1.2	El factor bus	127
8.1.3	Usuarios no técnicos: la familia como dependiente	128
8.1.4	Documentación, traspaso y planes de sucesión	128
8.1.5	La soberanía como responsabilidad	128
8.2	Energía y huella ambiental	129
8.2.1	El argumento de la eficiencia hiperescalar	129
8.2.2	Fabricación, segunda mano y el coste oculto	129
8.2.3	Hardware de bajo consumo y dimensionado con criterio	130
8.2.4	Medir antes que afirmar	130
8.2.5	Un equilibrio honesto	130
8.3	Tensiones del <i>self-hosting</i>	131
8.3.1	Esfuerzo y mantenimiento	131
8.3.2	Fiabilidad frente a hiperescala	131
8.3.3	Energía	132
8.3.4	Brecha de habilidad	132
8.3.5	El riesgo de una seguridad mal hecha	132
8.3.6	Sostenibilidad del modelo	133
8.4	Modelos híbridos y estrategias de adopción	133
8.4.1	El criterio de partición: datos en casa, cómputo puntual fuera	133
8.4.2	Adopción gradual y reversibilidad	134
8.4.3	Comunidad, conocimiento compartido y soberanía pragmática	135
8.5	El futuro: IA local, hardware y comunidad	135
8.5.1	Modelos cada vez más capaces en el extremo	135
8.5.2	Hardware accesible: el papel de las NPU	136
8.5.3	Regulación y soberanía de datos	136
8.5.4	El movimiento autoalojado y una visión a cinco-diez años	137
9	Conclusiones	138
9.1	Síntesis de los hallazgos	138
9.2	Contribuciones	139
9.3	Lo que el caso aplicado aporta al marco	139
9.4	Límites del trabajo	140
9.5	Líneas futuras	140
	Referencias	141

Resumen

La externalización masiva de servicios, datos e inteligencia hacia la nube hiperescalar ha convertido capacidades cotidianas en rentas recurrentes sujetas a dependencia, vigilancia y bloqueo de proveedor. Esta tesis sostiene que es posible y deseable recuperar la soberanía sobre el cómputo personal y doméstico alojando en infraestructura propia no solo el almacenamiento y los servicios, sino también la inteligencia agéntica que opera sobre ellos, sin renunciar a una funcionalidad equiparable y bajo garantías de privacidad, control y continuidad que la nube no ofrece.

El método combina tres planos. Primero, una fundamentación crítica de la economía de la nube, la soberanía digital, el paradigma *local-first* y el coste total de propiedad, incluida una valoración honesta de cuándo el autoalojamiento no compensa. Segundo, una sistematización del panorama del *self-hosting* (almacenamiento, multimedia, domótica, comunicaciones, conocimiento, identidad e inteligencia local) y de su arquitectura y operación: hardware y energía, contenedores, redes, exposición segura, persistencia, copias 3-2-1, observabilidad y reproducibilidad. Tercero, un análisis de seguridad y gobernanza que aborda el modelo de amenazas del *homelab* y, de forma central, los riesgos específicos de la IA local y agéntica: inyección de prompts, exfiltración y la transición de la automatización a la agencia con capacidad de acciones irreversibles.

El hilo conductor es la agencia segura: cómo dotar a un asistente de autonomía útil conteniéndola mediante confirmación, capabilities, sandboxing, trazabilidad inmutable, rollback y humano en el bucle. Las contribuciones se validan en un caso aplicado, un asistente de voz agéntico enteramente autoalojado (voz, transcripción y modelos de lenguaje locales), del que se extraen resultados y lecciones. La tesis concluye que la soberanía computacional es una práctica viable, articulada mediante patrones de contención reproducibles, y discute modelos híbridos y el futuro de la IA local, el hardware accesible y la comunidad.

1 Introducción

La informática personal y empresarial de las últimas dos décadas se ha desplazado, de forma silenciosa pero profunda, hacia un modelo en el que el cómputo, el almacenamiento y los datos residen en infraestructuras ajenas operadas por un puñado de grandes proveedores. Correo, documentos, fotografías, copias de seguridad, mensajería, domótica y, más recientemente, los modelos de inteligencia artificial que median en la producción de texto y de decisiones se ejecutan en centros de datos remotos. Lo que se presenta como comodidad encierra una transferencia estructural de control: el usuario deja de poseer el sustrato sobre el que descansa su vida digital y pasa a alquilarlo bajo condiciones que no fija ni, con frecuencia, conoce.

1.1. El problema: la dependencia de la nube

El modelo dominante de la nube no se reduce a una cuestión de localización física de los servidores. Cabe distinguir tres dimensiones del problema. La primera es económica: el cómputo se consume como renta perpetua, de modo que el coste no termina nunca y crece con el uso, mientras el proveedor captura el valor de los datos que custodia. La segunda es de dependencia: la interrupción del servicio, el cambio unilateral de condiciones o la desaparición del proveedor dejan al usuario sin recurso, y la dificultad técnica y contractual de migrar —el *lock-in* o dependencia del proveedor— consolida esa posición de subordinación. La tercera es de privacidad y vigilancia: confiar datos a terceros implica exponerlos a su escrutinio, a las exigencias de las autoridades a las que el proveedor está sujeto y a usos secundarios que el usuario no autoriza de manera informada.

Estas tres dimensiones convergen en una pérdida de *soberanía digital*, entendida como la capacidad efectiva de una persona o una organización para gobernar su propia infraestructura informática, sus datos y los procesos que se ejecutan sobre ellos. La cuestión adquiere una urgencia nueva con la irrupción de la inteligencia artificial generativa. Delegar en servicios remotos no solo el archivo de los datos, sino también su interpretación, su síntesis y la ejecución de acciones a partir de ellos, eleva el grado de dependencia a un plano cualitativamente distinto: ya no se externaliza el almacén, sino el juicio.

1.2. Pregunta y tesis central

De este diagnóstico surge la pregunta que vertebra el presente trabajo: ¿es posible recuperar la soberanía sobre la propia vida digital sin renunciar a las prestaciones que han hecho

atractiva a la nube? Y, de manera más específica, ¿hasta dónde puede llegar esa recuperación cuando se incorpora la frontera más exigente, la de la inteligencia artificial que actúa con cierta autonomía?

La tesis que se defiende es que el *self-hosting* o autoalojamiento —la práctica de operar los propios servicios sobre infraestructura controlada por el usuario— constituye una vía técnicamente viable y conceptualmente coherente para reconstruir la soberanía digital. Esta práctica se inscribe en el paradigma *local-first*, según el cual los datos y la lógica residen prioritariamente en dispositivos bajo control del usuario, y la conectividad externa se convierte en complemento y no en requisito. El autoalojamiento abarca un panorama amplio de servicios —almacenamiento, multimedia, domótica, comunicaciones, conocimiento, identidad— que poseen peso e interés propios. Dentro de ese panorama, la inteligencia local —modelos de lenguaje, voz y visión ejecutados en el *homelab*, el laboratorio doméstico de cómputo— representa la frontera más reciente y más difícil.

Se argumenta, además, que esa frontera no es solo una cuestión de capacidad de cómputo, sino de gobernanza. Cuando un sistema local deja de limitarse a responder y comienza a actuar —a enviar mensajes, modificar archivos o controlar dispositivos—, adquiere lo que aquí se denomina *agencia agéntica*: la facultad de emprender acciones con efectos en el mundo. La agencia agéntica local es la verdadera prueba de la soberanía, porque concentra a la vez su máxima promesa, ejecutar tareas sin ceder datos ni control a terceros, y su máximo riesgo, conferir autonomía a un sistema falible dentro del propio perímetro. El trabajo sostiene que esta frontera es franqueable siempre que la autonomía se acompañe de mecanismos rigurosos de contención, trazabilidad y supervisión humana.

1.3. Alcance

El estudio se ocupa del *self-hosting* en el contexto de un usuario técnicamente competente o de una organización pequeña que opera un *homelab*, no de la infraestructura de grandes corporaciones ni de los centros de datos hiperescalares. Comprende los fundamentos conceptuales del problema, el panorama de servicios autoalojables, las decisiones de arquitectura y operación, la seguridad y el modelado de amenazas, y la gobernanza de los sistemas dotados de agencia. La inteligencia artificial local se trata como un componente integrado en ese conjunto, no como objeto exclusivo; el documento no es una monografía sobre inteligencia artificial, sino un examen del autoalojamiento en el que la inteligencia agéntica local constituye el hilo conductor y el límite que pone a prueba el resto del marco.

Quedan fuera del alcance el análisis comercial de proveedores concretos, las guías de instalación paso a paso y la comparación exhaustiva de productos, que envejecen con rapidez y desvían la atención de los principios. El trabajo tampoco adopta una postura maximalista: se reconoce de manera explícita que existen escenarios en los que el autoalojamiento no compensa, y la honestidad intelectual sobre esos límites se considera parte integrante del argumento.

1.4. Metodología

El método combina dos registros complementarios. El primero es una investigación conceptual y técnica que articula los fundamentos económicos, jurídicos y de ingeniería del problema, y que sistematiza el estado de la práctica del autoalojamiento en sus distintas capas. El segundo es un caso aplicado: el diseño, la construcción y la operación de un asistente de voz agéntico autoalojado, que sirve de ilustración del marco conceptual y de banco de pruebas de sus afirmaciones. El caso se confina a su capítulo correspondiente y se emplea para contrastar, en condiciones reales, las tesis sobre soberanía, arquitectura, seguridad y gobernanza expuestas en la parte general; no constituye el núcleo del trabajo, sino la evidencia de que sus principios resisten la implementación.

1.5. Estructura del documento

El documento se ordena en siete partes. La primera establece los *fundamentos*: la economía de la nube, la noción de soberanía digital, el paradigma *local-first*, la privacidad, el coste total de propiedad, la longevidad frente al *lock-in* y los límites del autoalojamiento. La segunda recorre el *panorama del self-hosting*, con una taxonomía de servicios que abarca desde el almacenamiento hasta la inteligencia local. La tercera aborda la *arquitectura y la operación*: hardware, contenedores, redes, exposición segura, persistencia, copias de seguridad, observabilidad y reproducibilidad. La cuarta se dedica a la *seguridad y las amenazas*, incluido el modelado de riesgos propios de la inteligencia artificial local. La quinta desarrolla la *gobernanza y la agencia segura*, eje en el que culmina el argumento: la transición de la automatización a la agencia y los patrones que la hacen confiable. La sexta presenta el *caso aplicado*. La séptima, de *discusión y futuro*, recapitula las tensiones del modelo, examina las estrategias híbridas de adopción y proyecta la evolución de la inteligencia local, el hardware y la comunidad que sostiene esta práctica.

2 Fundamentos: soberanía, datos y el problema de la nube

La computación contemporánea se ha desplazado, de forma casi imperceptible, hacia un modelo en el que la capacidad de cómputo, el almacenamiento de la información y la lógica de las aplicaciones residen en infraestructuras ajenas, operadas por un número reducido de proveedores. Este desplazamiento ha traído consigo comodidad y elasticidad, pero también una dependencia estructural cuyo coste rara vez se examina en su totalidad. En este capítulo se establecen los fundamentos conceptuales sobre los que se asienta el resto de la obra: se argumenta que la externalización por defecto de los datos y de los procesos hacia la nube no es una necesidad técnica, sino una elección económica y de poder que admite alternativas legítimas. El autoalojamiento (*self-hosting*) se presenta así no como una excentricidad técnica, sino como una respuesta razonada a un problema concreto de dependencia, opacidad y pérdida de control.

La tesis que vertebra el capítulo sostiene que la soberanía digital —entendida como la capacidad efectiva de decidir dónde residen los datos, quién los procesa y bajo qué reglas— constituye un criterio de diseño tan relevante como el rendimiento o el coste. A partir de ahí, se examina la economía de la nube y sus lógicas de renta, dependencia y vigilancia; se delimita el concepto de soberanía digital y su relación con la propiedad de los datos; y se introduce el paradigma *local-first* como marco que reordena las prioridades situando al dispositivo del usuario, y no al servidor remoto, en el centro. Conviene advertir que este eje no se reduce a una cuestión de privacidad individual: afecta a la longevidad de los sistemas, a la confidencialidad de la información y a la estructura misma de costes de cualquier infraestructura.

Las secciones siguientes desarrollan estos hilos de manera ordenada. Se abordan la privacidad, la confidencialidad y el principio de minimización de datos; se compara el coste total de propiedad de la nube frente a la infraestructura propia, más allá de la factura mensual aparente; y se analizan la longevidad, el problema del *lock-in* (dependencia del proveedor) y la obsolescencia inducida. El capítulo se cierra con un ejercicio de honestidad intelectual sobre los escenarios en los que el *self-hosting* no compensa, pues ningún marco conceptual se sostiene si solo enumera sus ventajas. Estos fundamentos preparan el terreno para los capítulos posteriores, donde el panorama del autoalojamiento se conecta con la inteligencia agéntica local: una aplicación particularmente exigente, en la que las cuestiones de soberanía sobre los datos y el cómputo adquieren especial nitidez.

2.1. La economía de la nube: renta, dependencia y vigilancia

La computación en la nube no constituye únicamente un avance técnico de provisión de recursos bajo demanda; configura, ante todo, un modelo económico con incentivos propios que conviene examinar antes de discutir cualquier alternativa de *self-hosting* (autoalojamiento). Comprender ese modelo permite distinguir lo que la nube ofrece como conveniencia genuina de lo que impone como dependencia estructural, y sitúa la cuestión de la soberanía digital en su dimensión correcta: no es una preferencia ideológica, sino una respuesta a una asimetría de poder consolidada.

2.1.1. De la propiedad a la renta

El rasgo definitorio del modelo de nube es la sustitución de la propiedad por el alquiler. Donde antes una organización adquiría servidores, licencias perpetuas y capacidad de cómputo como activos amortizables, hoy contrata un flujo continuo de servicios facturados por uso o por suscripción. Esta transición convierte un gasto de capital puntual en un gasto operativo recurrente y, sobre todo, transfiere al proveedor el control sobre la disponibilidad, el precio y las condiciones del recurso. La renta recurrente es atractiva para el proveedor por razones evidentes: produce ingresos predecibles y crecientes, y desplaza el riesgo de obsolescencia hacia quien presta el servicio. Para el cliente, la contrapartida es una pérdida de soberanía sobre su propia base material de cómputo.

El argumento habitual a favor de este arreglo —elasticidad, ausencia de mantenimiento, economías de escala— es real en muchos escenarios, pero incompleto. Omite que la elasticidad se cobra con un sobreprecio respecto al coste marginal del hardware, y que ese sobreprecio se vuelve estructural cuando la carga de trabajo es estable y previsible. Un servicio que se ejecuta de forma constante durante años no aprovecha la elasticidad por la que paga; sufraga, en cambio, una prima de flexibilidad que no utiliza. La conveniencia, en suma, tiene un precio que rara vez se contabiliza frente al coste de internalizar la infraestructura en un *homelab* o en un servidor propio. Conviene matizar que el cálculo no es universal: para cargas verdaderamente intermitentes o de pico imprevisible, la elasticidad puede justificar su prima; el problema surge cuando se generaliza ese arreglo a cargas que no lo necesitan.

2.1.2. Costes de cambio y dependencia

La renta recurrente se sostiene sobre un segundo mecanismo: el coste de cambio. Las plataformas de nube no compiten solo en precio, sino en la dificultad de abandonarlas. Los datos se almacenan en formatos y servicios propietarios; la lógica de aplicación se acopla a interfaces específicas de cada proveedor; las transferencias de salida de datos se tarifican de manera deliberadamente asimétrica respecto a las de entrada. El resultado es una dependencia tecnológica (*lock-in*) que crece con el tiempo y que encarece progresivamente cualquier migración. Conviene observar que esta fricción no es un defecto accidental de la arquitectura, sino un

activo del modelo de negocio: cuanto mayor es el coste de marcharse, mayor es el poder de fijación de precios del proveedor sobre un cliente cautivo.

La adopción de estándares abiertos y de tecnologías neutrales respecto al proveedor —contenedores portables (Docker, 2024; Merkel, 2014), protocolos públicos, formatos de datos documentados— mitiga esta dependencia, pero no la elimina mientras la infraestructura siga siendo ajena. La portabilidad técnica de una aplicación no implica la portabilidad efectiva de un sistema entero cuando los datos, las identidades y las integraciones están entrelazados con servicios gestionados. El acoplamiento profundo a servicios administrados —bases de datos, colas, funciones, identidad— sustituye una dependencia visible de hardware por una dependencia menos perceptible de interfaces, que resulta más difícil de cuantificar y, por ello, más difícil de revertir.

2.1.3. El espejismo de lo gratuito

Una parte sustancial de la economía digital se ofrece sin coste monetario directo. Buscadores, correo, almacenamiento, redes sociales y asistentes se presentan como gratuitos, pero ningún servicio que requiere centros de datos, energía y personal lo es en sentido económico. Cuando el usuario no paga con dinero, paga con sus datos: la contraprestación es la cesión de información personal y conductual que el proveedor explota mediante publicidad dirigida, perfilado y venta de inferencias. Esta lógica ha sido caracterizada como *capitalismo de vigilancia*, un orden económico que reclama la experiencia humana como materia prima gratuita para extraer datos conductuales con los que predecir y modular el comportamiento (Zuboff, 2019).

El punto relevante no es moral sino estructural. La gratuidad aparente invierte la relación habitual entre proveedor y cliente: el usuario deja de ser el cliente para convertirse en el producto, y el cliente real es el anunciante o el comprador de datos. Esa inversión alinea el diseño del servicio con la captura de atención y de información, no con el interés de quien lo usa. La consecuencia es una asimetría informativa profunda: el proveedor conoce al usuario con un detalle creciente, mientras el usuario ignora qué se infiere de él, cómo se almacena y con quién se comparte. Marcos regulatorios como el Reglamento General de Protección de Datos (European Parliament & Council, 2016) intentan corregir esta asimetría imponiendo límites al tratamiento y reconociendo derechos de acceso, portabilidad y supresión, pero operan sobre un modelo cuyo incentivo de fondo sigue siendo la acumulación de datos. La norma actúa como contención externa, no como rediseño del incentivo que la motiva.

2.1.4. Externalidades y centralización del poder

El modelo de nube genera, además, externalidades que su contabilidad interna no refleja. La concentración del tráfico y del almacenamiento en un puñado de operadores hiperescalares crea puntos únicos de fallo cuya caída afecta simultáneamente a miles de servicios dependientes y traslada a la sociedad el coste de esa fragilidad sistémica. La dependencia de jurisdicciones extranjeras somete datos de ciudadanos y empresas a legislaciones ajenas

y a riesgos geopolíticos sobre los que el usuario carece de control. Y el coste ambiental de los centros de datos —consumo eléctrico e hídrico— se difumina tras la abstracción de un servicio que parece inmaterial.

Por encima de estas externalidades se sitúa el efecto más decisivo: la centralización del poder. Cuando la capacidad de cómputo, el almacenamiento y los servicios esenciales se concentran en unas pocas plataformas, estas adquieren una posición que excede la de un proveedor ordinario. Pueden alterar términos de servicio, retirar acceso, modificar precios o moderar contenidos de forma unilateral, y lo hacen sobre infraestructuras de las que dependen administraciones, empresas y particulares. La asimetría deja de ser meramente económica para volverse política: quien controla la infraestructura controla las condiciones de funcionamiento de cuanto se construye sobre ella.

Es precisamente esta acumulación de renta, dependencia y vigilancia la que dota de sentido al planteamiento *local-first* (Kleppmann et al., 2019) y, más ampliamente, al autoalojamiento como ejercicio de soberanía digital. No se trata de rechazar la nube por principio, ni de negar sus ventajas reales en escalabilidad y operación, sino de reconocer que su modelo económico desplaza sistemáticamente el control, el dato y el valor hacia el proveedor. Frente a ese desplazamiento, recuperar la infraestructura propia no es nostalgia técnica: es restituir al usuario la posición de decisión sobre sus datos y su cómputo que el modelo de renta le ha sustraído. Los capítulos siguientes desarrollan las condiciones técnicas y los compromisos que ese ejercicio de soberanía exige.

2.2. Soberanía digital y propiedad de los datos

La *soberanía digital* designa la capacidad efectiva de un sujeto —individuo, comunidad u organización— para decidir sobre la infraestructura de cómputo, los datos y los flujos de información que le conciernen, sin depender de intermediarios cuya voluntad pueda anular esa decisión. No se trata de un atributo abstracto, sino de un poder de determinación ejercido sobre artefactos concretos: dónde residen los datos, qué procesos los transforman, quién puede leerlos y bajo qué condiciones se interrumpe el acceso. La noción admite dos planos complementarios. En el plano *individual*, la soberanía digital remite a la autodeterminación informativa de la persona, esto es, a su facultad de gobernar la recopilación, el tratamiento y la circulación de los datos que la describen. En el plano *colectivo*, atañe a la autonomía tecnológica de un grupo o de una jurisdicción frente a la dependencia estructural de plataformas extranjeras, que condiciona desde la disponibilidad de servicios hasta la aplicabilidad de leyes propias. Ambos planos se tensan mutuamente: una soberanía individual robusta puede convivir con dependencias colectivas profundas, como ocurre cuando un ciudadano cifra sus comunicaciones pero las hace transitar por infraestructuras que un tercer Estado puede intervenir.

2.2.1. Propiedad y custodia frente a alojamiento

Conviene distinguir tres relaciones que el lenguaje ordinario confunde: la *propiedad* de los datos, su *custodia* y su mero *alojamiento*. La propiedad alude a la titularidad jurídica y moral sobre la información; la custodia, al control técnico cotidiano sobre el medio que la almacena y la procesa; el alojamiento, a la ubicación física o lógica de los bits. La arquitectura de la nube comercial divorcia sistemáticamente estos términos. El usuario conserva, en el mejor de los casos, una propiedad nominal sobre sus datos, mientras cede la custodia y el alojamiento a un proveedor que opera el sustrato material. Esta cesión no es neutra: quien custodia decide de facto sobre la persistencia, el cifrado, la replicación y, llegado el caso, la supresión o el bloqueo del acceso. La propiedad sin custodia equivale a un título sin posesión, jurídicamente válido pero materialmente inerte cuando el custodio interrumpe el servicio o reinterpreta sus condiciones contractuales.

El paradigma *local-first* (Kleppmann et al., 2019) reacciona contra ese divorcio al postular que la copia primaria y autoritativa de los datos debe residir en los dispositivos bajo control del usuario, relegando los servidores remotos a un papel auxiliar de sincronización y respaldo. Bajo esta óptica, el *autoalojamiento* (*self-hosting*) —la operación de los propios servicios sobre infraestructura controlada por el usuario— constituye la expresión infraestructural de la soberanía: reúne propiedad, custodia y alojamiento en una misma esfera de control. La crítica al modelo extractivo que sustenta a las grandes plataformas, articulada en torno a la noción de capitalismo de vigilancia (Zuboff, 2019), ofrece el trasfondo normativo de esta reacción: cuando el dato personal se convierte en materia prima de un modelo de negocio ajeno, la recuperación de la custodia deja de ser una preferencia técnica para erigirse en condición de autonomía.

2.2.2. Jurisdicción y control efectivo

La soberanía digital no se agota en la dimensión técnica; está atravesada por la *jurisdicción*. La ubicación física de un servidor determina qué ordenamiento jurídico rige sobre los datos que aloja y, por tanto, qué autoridades pueden requerir su entrega o su intervención. Esta circunstancia genera disociaciones notables: datos pertenecientes a un ciudadano europeo, alojados por una empresa extranjera en un centro de datos situado en territorio comunitario, quedan simultáneamente sujetos al régimen europeo de protección de datos (European Parliament & Council, 2016) y, según la legislación extraterritorial aplicable al proveedor, al alcance de requerimientos foráneos. El control jurídico nominal y el control efectivo divergen. El Reglamento General de Protección de Datos consagra derechos de acceso, rectificación, supresión y portabilidad que refuerzan la posición del titular, pero su eficacia depende de la cooperación de un custodio que puede residir bajo otra soberanía estatal y obedecer mandatos incompatibles con aquel régimen.

De ahí que la soberanía digital deba medirse por el *control efectivo* y no por la mera titularidad declarada. El control efectivo se manifiesta en la capacidad de ejecutar, sin pedir permiso ni anticipar represalias, operaciones elementales: cifrar antes de transmitir, migrar los datos a

otra plataforma, auditar quién accede a ellos, revocar accesos o destruirlos de forma verificable. Cada una de estas capacidades se debilita en la medida en que la custodia se externaliza. El cifrado del lado del cliente, por ejemplo, restituye parcialmente el control incluso cuando el alojamiento es ajeno, pues sustrae el contenido al conocimiento del custodio; ahora bien, los metadatos —patrones de acceso, tamaños, frecuencias— suelen permanecer expuestos, lo que revela que la soberanía es gradual y rara vez absoluta.

2.2.3. Propiedad de los datos como haz de facultades

La expresión *data ownership* (propiedad de los datos) resulta engañosa si se la concibe a imagen de la propiedad sobre cosas tangibles. Los datos son no rivales —su uso por un agente no impide el uso simultáneo por otro— y se replican sin coste apreciable, de modo que la posesión exclusiva, fundamento de la propiedad clásica, no se sostiene técnicamente. Resulta más fértil entender la propiedad de los datos como un *haz de facultades* diferenciables: el derecho a acceder, a modificar, a copiar, a transmitir, a excluir a terceros y a determinar las finalidades del tratamiento. La soberanía digital se realiza cuando ese haz reposa, de hecho y no solo de derecho, en manos del titular. El autoalojamiento maximiza la concentración de tales facultades, mientras que la delegación en plataformas las fragmenta y redistribuye, normalmente en favor del proveedor. La facultad de excluir resulta aquí decisiva: en un entorno de custodia ajena, el titular rara vez puede garantizar que el proveedor no acceda al contenido ni lo reutilice, de suerte que la exclusión se sostiene sobre una promesa contractual antes que sobre una imposibilidad técnica.

Esta lectura permite precisar el objeto del presente trabajo. Recuperar la soberanía digital no exige rechazar toda infraestructura ajena, sino restituir al titular el control sobre las facultades críticas del haz, situando la copia autoritativa de sus datos y los procesos que los gobiernan en una esfera que él administre. El *autoalojamiento*, sostenido sobre un *homelab* (laboratorio doméstico de cómputo), proporciona el sustrato donde esa restitución se vuelve practicable, al desplazar la confianza desde una promesa contractual hacia una garantía de control técnico verificable. Las secciones siguientes examinan los costes, las renunciaciones y las arquitecturas que dicha opción comporta.

2.3. El paradigma *local-first*

El término *local-first* (de orientación local) designa una filosofía de diseño de software que sitúa el dispositivo del usuario, y no el servidor remoto, como sede primaria de los datos y de la lógica de aplicación. Frente al modelo dominante de la aplicación web alojada en la nube, donde la copia canónica de la información reside en infraestructura ajena y el cliente actúa como mero terminal, el enfoque *local-first* invierte la jerarquía: la copia local es autoritativa y la red queda relegada a un papel auxiliar de sincronización y respaldo. Esta inversión no es meramente técnica; conlleva una reasignación de la propiedad y del control sobre los datos que conecta directamente con la noción de soberanía digital desarrollada en este capítulo.

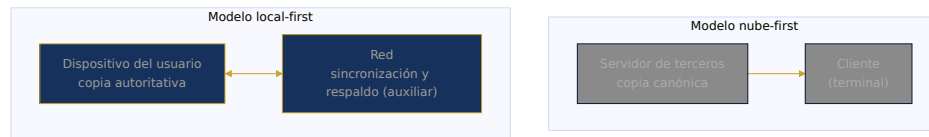


Figura 2.1: Nube-first frente a *local-first*: en el modelo de nube la copia canónica reside en el servidor de un tercero y el cliente actúa como terminal; en *local-first* la copia autoritativa vive en el dispositivo del usuario y la red queda relegada a sincronización y respaldo.

2.3.1. Los siete principios

La formulación canónica del paradigma procede de un trabajo del laboratorio Ink & Switch (Kleppmann et al., 2019), que enuncia siete ideales que un sistema *local-first* debería satisfacer. No constituyen un protocolo ni una especificación, sino criterios evaluativos con los que medir hasta qué punto una aplicación devuelve al usuario el control sobre su información.

1. *Rapidez*: las operaciones deben responder sin la latencia de un viaje de ida y vuelta al servidor, pues la lectura y la escritura se ejecutan contra una copia local.
2. *Multidispositivo*: el trabajo debe sincronizarse de forma transparente entre los distintos equipos de la persona usuaria.
3. *Funcionamiento sin red*: la aplicación ha de ser plenamente operativa sin conexión, condición que aproxima el paradigma al enfoque *offline-first* descrito más adelante.
4. *Colaboración*: varias personas deben poder editar de manera concurrente sin que la disponibilidad dependa de un servidor central permanentemente accesible.
5. *Longevidad*: los datos deben sobrevivir a la desaparición del proveedor o del servicio, condición que el modelo de nube rara vez garantiza.
6. *Privacidad y seguridad*: la información debe poder cifrarse de extremo a extremo, de modo que ningún intermediario tenga acceso al contenido en claro.
7. *Control del usuario*: la persona conserva la titularidad última sobre sus datos, con capacidad de copiarlos, moverlos y conservarlos sin permiso de un tercero.

Conviene observar que estos principios no son independientes entre sí, sino que se refuerzan y, en ocasiones, entran en tensión. La rapidez y el funcionamiento sin red derivan ambos de la existencia de una copia local autoritativa; en cambio, la colaboración concurrente sin servidor central plantea el problema clásico de la consistencia de réplicas, que es donde el paradigma encuentra su mayor dificultad técnica.

2.3.2. CRDTs y sincronización

El obstáculo central del *local-first* es la conciliación de réplicas que han divergido. Si dos dispositivos modifican de forma independiente y sin conexión la misma información, al reconectarse debe existir un mecanismo que fusione ambos estados sin pérdida de datos ni intervención manual. La solución que articula gran parte de la literatura del paradigma son los

Conflict-free Replicated Data Types (tipos de datos replicados sin conflicto, o CRDT), estructuras de datos diseñadas para que las operaciones concurrentes converjan determinísticamente hacia un mismo estado final con independencia del orden en que se apliquen (Shapiro et al., 2011a).

La propiedad que hace posible esta convergencia es algebraica. En la variante basada en estado, la operación de fusión es conmutativa, asociativa e idempotente, de modo que combinar réplicas equivale al cálculo de un supremo en un retículo; en la variante basada en operaciones, son las propias actualizaciones concurrentes las que han de conmutar (Shapiro et al., 2011b). Gracias a ello, dos réplicas que hayan recibido el mismo conjunto de actualizaciones, en cualquier secuencia, alcanzan estados idénticos sin necesidad de coordinación previa ni de un árbitro central. Esta garantía de *convergencia fuerte eventual* permite prescindir del servidor como autoridad de orden, sustituyéndolo por una lógica de fusión distribuida en cada nodo.

El problema subyacente es el del ordenamiento de eventos en sistemas distribuidos, formalizado mucho antes mediante la relación de causalidad y los relojes lógicos (Lamport, 1978), cuya maquinaria conceptual reaparece en los relojes vectoriales y en las marcas de versión que numerosos CRDT emplean para detectar concurrencia. Cabe asimismo encuadrar el paradigma en el marco del teorema CAP (Gilbert & Lynch, 2002): al renunciar a la consistencia fuerte e inmediata en favor de la disponibilidad bajo partición de red, el *local-first* adopta deliberadamente el vértice AP, y los CRDT son precisamente la técnica que vuelve manejable esa consistencia eventual (Brewer, 2012).

2.3.3. Relación con *offline-first* y con el autoalojamiento

El paradigma *local-first* se solapa con otras dos nociones afines de las que conviene distinguirlo. La primera es el enfoque *offline-first* (de prioridad sin conexión), que prescribe que una aplicación funcione sin red y sincronice cuando esta vuelva a estar disponible. Todo sistema *local-first* es necesariamente *offline-first*, pues su tercer principio lo exige; pero la implicación no se invierte. Una aplicación puede operar sin conexión y resolver los conflictos de sincronización mediante una autoridad central —la estrategia *last-write-wins*, que descarta una de las versiones en conflicto— sin satisfacer los principios de longevidad, privacidad o control del usuario. El *local-first* es, por tanto, un superconjunto de exigencias: incorpora la operatividad sin red, pero le añade requisitos de propiedad y de conciliación sin pérdida.

La segunda noción es el *self-hosting* (autoalojamiento), eje conductor de esta tesis. Ambos paradigmas comparten el objetivo de sustraer los datos al control de proveedores externos, pero operan en planos distintos y no equivalentes. El autoalojamiento desplaza el servidor desde la infraestructura de un tercero hacia un *homelab* bajo control directo de la persona usuaria, sin cuestionar necesariamente la arquitectura cliente-servidor: la copia autoritativa sigue residiendo en un servidor, solo que ahora propio. El *local-first*, en cambio, cuestiona la centralidad misma del servidor y la traslada al dispositivo del usuario. Las dos aproximaciones difieren, además, en su tolerancia a fallos: un servicio autoalojado deja de ser accesible

si el servidor doméstico cae o pierde conectividad, mientras que una aplicación *local-first* conserva su plena funcionalidad porque la copia operativa es local.

Lejos de excluirse, ambas estrategias se complementan. Un servidor de sincronización autoalojado puede actuar como punto de encuentro y de respaldo para réplicas *local-first*, aportando descubrimiento entre dispositivos y persistencia duradera sin convertirse en autoridad de consistencia: si ese servidor desaparece, las réplicas siguen siendo válidas y pueden reconciliarse por otra vía. Esta combinación reconcilia las dos vertientes de la soberanía digital: la del dato, que el *local-first* garantiza mediante la copia local cifrable y portable, y la de la infraestructura, que el autoalojamiento asegura al situar los medios de cómputo y almacenamiento bajo gobierno propio. La consecuencia de diseño es nítida: la red se concibe como una optimización para sincronizar y compartir, no como una dependencia para funcionar, lo que invierte el supuesto implícito del modelo de nube examinado en las secciones precedentes.

2.4. Privacidad, confidencialidad y minimización

La privacidad no se reduce al secreto: se entiende mejor como el control efectivo que un sujeto conserva sobre el flujo de la información que le concierne, esto es, sobre quién accede a qué dato, con qué finalidad y durante cuánto tiempo. Esta concepción relacional resulta capital en el contexto del *self-hosting* (autoalojamiento), pues desplaza la pregunta desde «¿está cifrado el dato?» hacia «¿quién decide sobre el dato?». La confidencialidad, por su parte, designa el atributo más estrecho que garantiza que la información solo se revele a entidades autorizadas; es un medio para sostener la privacidad, no su equivalente. La distinción importa porque un servicio en la nube puede ofrecer confidencialidad técnica impecable frente a terceros y, simultáneamente, erosionar la privacidad del usuario al tratar sus datos como insumo para el modelo de negocio. Zuboff (2019) describe precisamente esa lógica: la conversión sistemática de la experiencia humana en excedente conductual explotable, ajena al control del individuo aunque protegida frente a intrusos externos.

2.4.1. Minimización de datos

El principio de minimización sostiene que solo deben recogerse, procesarse y conservarse los datos estrictamente necesarios para una finalidad legítima y explícita. Opera en tres dimensiones: minimización del volumen recopilado, de la duración de la retención y del número de copias y destinatarios. Su valor de diseño es profiláctico: el dato que no se recoge no puede filtrarse, venderse ni reutilizarse fuera de propósito. La minimización guarda afinidad conceptual con el principio de mínimo privilegio formulado por Saltzer y Schroeder (1975), que prescribe conceder a cada componente únicamente los permisos imprescindibles; trasladado a los datos personales, recomienda recopilar el mínimo y exponerlo al mínimo de procesos. La arquitectura *local-first* (Kleppmann et al., 2019) lleva el principio a su consecuencia estructural: si el dato reside y se procesa primordialmente en el dispositivo del usuario, la superficie de recolección remota tiende a cero por construcción, y la minimización deja de ser una política revocable para convertirse en una propiedad del sistema.

Conviene subrayar la diferencia entre minimizar por política y minimizar por arquitectura. Un proveedor centralizado que promete no conservar registros sigue siendo capaz técnicamente de hacerlo y depende de su buena voluntad, su jurisdicción y su solvencia futura; un sistema autoalojado que nunca transmite el dato carece de esa capacidad. La soberanía digital se afirma así no en la confianza depositada en un tercero, sino en la ausencia de necesidad de confiar.

2.4.2. Cifrado: reposo, tránsito y extremo a extremo

El cifrado materializa la confidencialidad en tres planos complementarios. El cifrado *en tránsito* protege los datos mientras circulan por la red; su estándar contemporáneo es TLS 1.3 (Rescorla, 2018), que reduce la latencia del establecimiento de sesión y retira algoritmos heredados frente a versiones anteriores. El cifrado *en reposo* protege los datos almacenados en disco frente al acceso físico o lógico no autorizado; las herramientas de copia de seguridad maduras lo integran de serie, como ilustran BorgBackup (2024) y restic (2024), que combinan cifrado autenticado con deduplicación, de modo que ni siquiera el operador del almacenamiento remoto que aloja el repositorio puede leer su contenido.

El cifrado *extremo a extremo* (end-to-end encryption, E2EE) constituye la garantía más fuerte: los datos se cifran en el origen y solo se descifran en el destino legítimo, de manera que ningún intermediario —incluido el propio proveedor del servicio— posee las claves para leerlos. Aquí reside una asimetría decisiva entre el modelo de nube y el autoalojado. En un servicio centralizado, el cifrado en reposo suele estar bajo control del proveedor, que custodia las claves y puede, por tanto, acceder al contenido o entregarlo ante un requerimiento; el usuario obtiene confidencialidad frente a atacantes externos, pero no frente al operador. En un despliegue propio, el titular concentra el rol de procesador y de poseedor de claves, lo que aproxima en la práctica las garantías del reposo a las del E2EE: la distinción entre «el proveedor puede leer» y «nadie salvo el titular puede leer» se colapsa porque proveedor y titular coinciden. El razonamiento de extremo a extremo (Saltzer et al., 1984) refuerza esta orientación: las funciones de integridad y confidencialidad se realizan de forma correcta y completa solo en los puntos terminales, no en la red intermedia, de modo que situar el control criptográfico en el extremo del usuario resulta coherente con la propia teoría del diseño de sistemas.

2.4.3. El marco del RGPD

El Reglamento General de Protección de Datos (European Parliament & Council, 2016) codifica varios de estos principios como obligaciones jurídicas. Consagra la minimización de datos y la limitación de la finalidad como principios rectores del tratamiento, e introduce la protección de datos desde el diseño y por defecto, que exige incorporar las salvaguardas en la arquitectura y no como añadido posterior. El autoalojamiento ofrece un ajuste natural a estas exigencias: reduce el número de encargados del tratamiento, evita transferencias internacionales problemáticas y simplifica el ejercicio de derechos como el acceso, la rectificación

o la supresión, puesto que el responsable controla directamente el sustrato físico donde residen los datos. No obstante, conviene matizar que autoalojar no exime de cumplimiento; antes bien, traslada al titular responsabilidades que en la nube delegaba, como la gestión de brechas, la documentación del tratamiento y la seguridad técnica efectiva.

2.4.4. Superficie de exposición personal

El conjunto de todos los puntos por los que la información de una persona puede ser observada, inferida o sustraída conforma su *superficie de exposición*. Centralizar datos en proveedores externos la amplía en varias direcciones simultáneas: multiplica los actores con acceso potencial, agrega información de numerosos usuarios en repositorios que se vuelven objetivos de alto valor y habilita la correlación de fuentes dispersas para reconstruir perfiles. El autoalojamiento contrae esa superficie al confinar el dato en una infraestructura acotada bajo control único, pero no la anula: introduce sus propios vectores —exposición de servicios mal configurados, gestión deficiente de claves, ausencia de actualizaciones— que recaen ahora íntegramente sobre el titular. La privacidad efectiva no resulta, pues, de elegir entre nube y *homelab* (laboratorio doméstico) como si una opción fuera inocua, sino de minimizar deliberadamente cuanto se recoge, cifrar en los tres planos descritos y mantener bajo control el reducido perímetro restante.

2.5. Coste total de propiedad: nube frente a infraestructura propia

La decisión entre delegar el cómputo a un proveedor de nube o sostener una infraestructura propia no puede resolverse comparando únicamente la tarifa mensual de un servicio gestionado con el precio de compra de un servidor. Tal comparación, frecuente en la divulgación entusiasta del *self-hosting* (autoalojamiento), induce a error porque ignora componentes de coste que solo se manifiestan a lo largo del ciclo de vida del sistema. El concepto pertinente es el *coste total de propiedad* (en inglés, *total cost of ownership* o TCO), que agrega la inversión inicial, los gastos recurrentes de operación y los costes indirectos asociados al mantenimiento, la indisponibilidad y el tiempo humano. Conviene, por tanto, descomponer el problema antes de pronunciarse sobre su resultado.

2.5.1. Componentes del coste propio

En una infraestructura autoalojada el coste se reparte en varias partidas heterogéneas. La primera es el *hardware*: el desembolso de capital en una o varias máquinas, almacenamiento, memoria y, en su caso, aceleradores. Este coste es amortizable, de modo que su impacto real por mes depende de la vida útil esperada del equipo; un servidor reutilizado o de segunda mano altera de forma sustancial la ecuación frente a la adquisición de plataforma nueva. La segunda partida es la *energía*, que comprende tanto el consumo del equipo en funcionamiento continuo como, en climas o densidades exigentes, su refrigeración. Una máquina que

consume cien vatios de media supone cerca de 876 kilovatios-hora anuales; a los precios europeos de electricidad, esta cifra dista de ser despreciable y, en operación ininterrumpida, puede igualar en pocos años el precio de compra del propio equipo.

A estas se añaden costes con frecuencia omitidos. La *conectividad* adecuada —ancho de banda simétrico, direccionamiento estable— suele exigir contratos superiores a los domésticos. La *redundancia* necesaria para una fiabilidad razonable —discos adicionales, copias de seguridad replicadas, alimentación ininterrumpida— multiplica el hardware base. Y la *sustitución* de componentes que fallan introduce un coste estocástico que solo se aprecia en horizontes largos. Las herramientas de respaldo deduplicado y cifrado, como las descritas por (BorgBackup contributors, 2024) o (restic contributors, 2024), reducen el volumen de almacenamiento que ocupan las copias, pero no eliminan la necesidad de un soporte físico redundante donde alojarlas ni el coste de la capacidad asociada.

2.5.2. El coste oculto del tiempo humano

La partida más subestimada, y a menudo la decisiva, es el tiempo humano de administración. Mantener una infraestructura propia exige instalar y actualizar sistemas, aplicar parches de seguridad, vigilar registros, diagnosticar fallos y restaurar el servicio tras una caída. Este trabajo no desaparece en la nube: se traslada al proveedor y se incorpora, opaco, a la tarifa. Cuando se autoaloja, reaparece como horas de dedicación personal. Si esas horas se valoran al coste de oportunidad de quien las invierte, el TCO de la infraestructura propia puede superar con holgura al de un servicio gestionado, sobre todo en cargas pequeñas, donde el coste fijo de la atención humana no llega a diluirse. La tecnología de contenedores mitiga parcialmente este coste al estandarizar el despliegue y reducir la divergencia entre entornos (Merkel, 2014; Docker, 2024), pero la operación sigue requiriendo competencia técnica y atención sostenida.

Conviene, no obstante, matizar la valoración de ese tiempo. Parte de él no es coste puro, sino inversión: en aprendizaje, en control efectivo sobre los datos y en independencia respecto de decisiones unilaterales del proveedor. Quien autoaloja adquiere un conocimiento transferible y una capacidad de respuesta que la externalización completa atrofia. La cuantificación honesta del TCO debe distinguir, por tanto, entre el tiempo consumido en tareas rutinarias —genuino coste a minimizar mediante automatización— y el invertido en autonomía, cuyo retorno no es monetario pero sí real. Ignorar esta distinción conduce a sobrestimar el coste del autoalojamiento tanto como omitir el tiempo humano lo infravalora.

2.5.3. Punto de equilibrio y cuándo la nube resulta más barata

De la descomposición anterior se sigue que existe un punto de equilibrio en función de la carga, su variabilidad y el horizonte temporal. La nube facturada por uso es eficiente para cargas intermitentes, picos imprevisibles o experimentos de corta vida: se paga solo por lo consumido y se evita inmovilizar capital en hardware ocioso. Su economía deriva de la agregación, que permite repartir entre muchos clientes el coste de personal especializado, redundancia

y eficiencia energética a escala. Para un servicio que se ejecuta unas horas al mes, adquirir y mantener una máquina dedicada es manifiestamente antieconómico.

La infraestructura propia invierte su ventaja cuando la carga es sostenida y predecible. Un servicio que opera de forma continua durante años amortiza el hardware y diluye el coste fijo, mientras que la tarifa por uso de la nube se acumula sin término. El cruce se produce cuando el coste mensual equivalente del equipo propio —amortización más energía más una estimación prudente del tiempo humano— cae por debajo de la factura recurrente del servicio gestionado equivalente. Para cargas modestas pero permanentes, ese cruce suele alcanzarse en plazos del orden de uno a tres años; a partir de ahí, el autoalojamiento tiende a resultar netamente más barato. El cálculo se desplaza de nuevo a favor de la nube cuando intervienen necesidades de elasticidad extrema, distribución geográfica o garantías de disponibilidad que una sola ubicación física no puede satisfacer sin replicar gran parte de la infraestructura del proveedor.

Cabe una advertencia metodológica: el TCO no agota la decisión. La soberanía digital, la confidencialidad de los datos y la resistencia frente a cambios de política del proveedor son valores que no figuran como línea de gasto, pero que pueden inclinar la balanza con independencia del coste estricto. Reducir la elección a su componente económico ignoraría precisamente las razones que motivan buena parte del autoalojamiento. La comparación de costes, en suma, ofrece una condición necesaria pero no suficiente: delimita cuándo lo propio es además asequible, no si es deseable. El análisis riguroso del TCO sirve para desmontar tanto el mito de que autoalojar es siempre más barato como el contrario, y para situar la decisión donde corresponde: en la intersección entre economía, capacidad técnica y autonomía.

2.6. Longevidad, lock-in y obsolescencia

La soberanía digital no se decide únicamente en el instante de la captura del dato, sino a lo largo de toda su vida útil, que en el caso de la información personal puede extenderse durante décadas. Una colección fotográfica, un archivo de correspondencia o un historial médico conservan su valor mucho después de que el servicio que los alojó haya cambiado de modelo de negocio, sido adquirido o cerrado. Esta asimetría temporal —datos que perduran frente a proveedores que no— constituye el núcleo del problema de longevidad y obliga a evaluar cualquier arquitectura por su comportamiento en el largo plazo, y no solo por su comodidad inmediata.

2.6.1. Las dos caras del lock-in

El *vendor lock-in* (dependencia del proveedor) designa el conjunto de costes —económicos, técnicos y de esfuerzo— que disuaden de abandonar un servicio una vez adoptado. Conviene distinguir dos mecanismos que con frecuencia se confunden. El primero es el *lock-in de formato*: los datos se almacenan en estructuras propietarias, no documentadas o sujetas a restricciones contractuales, de modo que extraerlos en una forma reutilizable resulta costoso

o imposible. El segundo es el *lock-in de servicio*: aunque los datos sean exportables, la funcionalidad que les da sentido reside en infraestructura ajena, de manera que migrar implica reconstruir desde cero flujos de trabajo, integraciones y automatismos.

El paradigma *local-first* (datos primero en local) reduce ambos vectores, pero no de forma idéntica. Al situar los datos en dispositivos bajo control de quien los posee y tratar la nube como mero canal de sincronización, atenúa el lock-in de servicio: la aplicación sigue siendo operativa aunque el proveedor desaparezca (Kleppmann et al., 2019). El lock-in de formato, en cambio, persiste si el software local emplea representaciones cerradas; la localidad física del dato no garantiza su inteligibilidad futura. De ahí que la longevidad dependa menos de dónde reside la información que de cómo está codificada.

2.6.2. Formatos abiertos como condición de supervivencia

Un formato abierto se caracteriza por una especificación pública, libre de regalías e implementable por terceros sin permiso del titular original. Su virtud no es ideológica sino práctica: multiplica el número de programas capaces de leerlo y, con ello, la probabilidad de que al menos uno siga existiendo dentro de veinte años. El contraste es ilustrativo. El texto plano, Markdown, CSV, SQLite, PNG o PDF/A son formatos cuya documentación y diversidad de implementaciones los hacen resistentes a la desaparición de cualquier fabricante concreto; un documento en un formato ofimático propietario de hace dos décadas, en cambio, puede requerir hoy software de emulación o conversiones con pérdida para abrirse. La apertura del formato funciona así como una póliza contra la obsolescencia: desacopla la vida del dato de la suerte comercial de quien produjo la herramienta.

Esta lógica enlaza con un principio arquitectónico más general. Cuando la garantía de legibilidad se delega en un servicio intermedio, queda sujeta a la continuidad de ese servicio; cuando reside en una especificación abierta y verificable por la propia parte interesada, se aproxima al argumento extremo a extremo, según el cual las propiedades críticas deben asegurarse en los puntos finales y no en componentes intermedios sobre los que no se tiene control (Saltzer et al., 1984). El formato abierto traslada la garantía de legibilidad al extremo que más interés tiene en preservarla.

2.6.3. Portabilidad y discontinuación de servicios

La portabilidad —capacidad de trasladar datos y funciones de un entorno a otro sin pérdida sustantiva— es el atributo que convierte la apertura teórica en libertad efectiva. Un formato abierto sin mecanismos de exportación sigue siendo una promesa incumplida; por eso la portabilidad real exige tanto especificaciones legibles como vías prácticas para extraer la totalidad del corpus, incluidos los metadatos y las relaciones. El marco normativo europeo reconoce esta dimensión al consagrar el derecho a la portabilidad de los datos en un formato estructurado y de uso común (European Parliament & Council, 2016), si bien el derecho legal no suple la viabilidad técnica: exportar grandes volúmenes de elementos a través de interfaces deliberadamente limitadas puede resultar impracticable.

El riesgo que la portabilidad mitiga es la discontinuación del servicio, fenómeno recurrente y rara vez anunciado con la antelación que la vida de los datos requeriría. El cierre de plataformas de fotografía, redes sociales o servicios de notas ha producido pérdidas de información personal cuando quienes las usaban no dispusieron de copias en formatos reutilizables. Frente a la nube gestionada, el autoalojamiento (*self-hosting*) invierte la relación de poder temporal: la decisión de retirar un servicio deja de pertenecer a un tercero y pasa a quien opera el *homelab* (laboratorio doméstico), que puede mantener una versión funcional de forma indefinida, congelar dependencias o migrar a su propio ritmo. Esta autonomía no es gratuita —impone la carga de la conservación activa, las copias de seguridad y las migraciones periódicas—, pero sustituye un riesgo exógeno e incontrolable por uno endógeno y gestionable.

Cabe, no obstante, matizar el optimismo. El autoalojamiento desplaza el problema de la obsolescencia sin abolirlo: el hardware envejece, las dependencias de software se rompen y la documentación se pierde igual que en cualquier sistema. La diferencia decisiva reside en quién retiene la capacidad de decidir y en si los datos permanecen, en todo momento, en formatos que cualquier herramienta futura pueda interpretar. La longevidad, en suma, no se adquiere con un proveedor ni con una infraestructura, sino con una disciplina sostenida de apertura, exportabilidad y custodia.

2.7. Honestidad intelectual: cuándo el *self-hosting* no compensa

Una defensa rigurosa del *self-hosting* (autoalojamiento) exige reconocer sus límites. Presentarlo como solución universal sería tan deshonesto como ignorar sus virtudes; conviene, por tanto, examinar los contraargumentos serios antes de formular criterios de decisión. La soberanía digital tiene un coste, y ese coste no siempre se justifica. Cabe distinguir, así, entre el caso en que el control propio aporta valor y aquel en que solo añade fragilidad.

2.7.1. Fiabilidad y disponibilidad

El primer contraargumento es de fiabilidad. Un proveedor de nube a gran escala opera con redundancia geográfica, alimentación ininterrumpida, equipos de guardia y acuerdos de nivel de servicio que un particular o una organización pequeña no pueden replicar. El teorema CAP recuerda que, ante una partición de red, todo sistema distribuido debe sacrificar consistencia o disponibilidad (Brewer, 2000; Gilbert & Lynch, 2002); reconsideraciones posteriores matizan que la disyuntiva solo se vuelve estricta durante la partición misma (Brewer, 2012), pero el *homelab* (laboratorio doméstico) afronta además un problema más prosaico: el punto único de fallo. Un corte eléctrico, una avería de disco sin réplica o una conexión residencial caída dejan el servicio inaccesible sin alternativa inmediata. La disponibilidad que una infraestructura gestionada ofrece por contrato debe, en el autoalojamiento, construirse y costearse pieza a pieza, y rara vez alcanza las mismas cifras. Para servicios cuya interrupción acarrea perjuicio grave —dependencia médica, continuidad de negocio crítica— la externalización profesional suele ser la opción responsable.

2.7.2. Responsabilidad operativa

El segundo contraargumento es la carga operativa. Externalizar un servicio externaliza también su mantenimiento: parches de seguridad, actualizaciones, monitorización, copias de seguridad verificadas y respuesta ante incidentes. Quien autoaloja asume todas esas tareas de forma indefinida. No se trata de un esfuerzo puntual de instalación, sino de una obligación recurrente que compite con el tiempo y la atención disponibles. Una copia de seguridad no probada no es, en términos prácticos, una copia de seguridad: herramientas como las descritas por la documentación de restic o BorgBackup facilitan el cifrado y la deduplicación, pero no eximen de la disciplina de verificar restauraciones periódicamente (restic contributors, 2024; BorgBackup contributors, 2024). El autoalojamiento mal mantenido se degrada en silencio hasta que un fallo lo revela en el peor momento. La pregunta pertinente no es si se sabe instalar un servicio, sino si se sostendrá su operación durante años, a través de migraciones, dependencias obsoletas y relevos de quien lo administra.

2.7.3. Seguridad mal hecha es peor que la delegada

El tercer contraargumento, y el más contraintuitivo, es que una seguridad amateur puede resultar peor que la delegada. Exponer servicios a Internet sin endurecimiento adecuado amplía la superficie de ataque en lugar de reducirla. Los principios clásicos de protección —privilegio mínimo, economía del mecanismo, valores por defecto seguros— exigen competencia para aplicarse (Saltzer & Schroeder, 1975), y los marcos de gestión de riesgo presuponen un proceso continuo que muchos despliegues caseros no sostienen (NIST, 2018). Un servicio autoalojado con credenciales por defecto, sin cifrado en tránsito (Rescorla, 2018) o sin segmentación de red constituye un riesgo mayor que confiar los mismos datos a un proveedor que, pese a sus inconvenientes de soberanía, mantiene un equipo de seguridad dedicado. Conviene subrayar que la soberanía sobre los datos no equivale a su protección efectiva: la mera localización de la información en infraestructura propia no es, por sí sola, garantía de privacidad ni de seguridad. Este matiz corrige un entusiasmo frecuente que confunde posesión con control real.

2.7.4. No para todos, ni para todo

El cuarto contraargumento es de adecuación. El autoalojamiento presupone conocimientos, tiempo y cierta tolerancia al riesgo que no son universales. Para una persona sin formación técnica, o para una organización sin función de sistemas, la promesa de control puede traducirse en fragilidad. Incluso para perfiles competentes, no todo servicio merece ser internalizado: la lógica del argumento extremo a extremo sugiere situar cada función donde mejor se cumple, no necesariamente en el extremo propio (Saltzer et al., 1984). Hay cargas —cómputo masivo esporádico, distribución global de contenido, ejecución de modelos de gran tamaño— cuyo coste de oportunidad en infraestructura propia supera con creces el de un servicio gestionado. Internalizarlas por principio, sin sopesar ese coste, traiciona la racionalidad que se invoca para defender la soberanía.

2.7.5. Criterios para decidir

De lo anterior se desprende un conjunto de criterios que orientan la decisión sin prescribir una respuesta única. Conviene autoalojar cuando concurren varias de estas condiciones:

- La sensibilidad de los datos hace inaceptable cederlos a terceros, en especial bajo regímenes de protección como el europeo (European Parliament & Council, 2016), y la lógica de captura de datos por parte de las plataformas resulta indeseable (Zuboff, 2019).
- Existe competencia técnica suficiente, propia o accesible, para sostener la operación y la seguridad a lo largo del tiempo.
- La interrupción ocasional del servicio es tolerable, o puede mitigarse con redundancia asumible.
- El valor de control, aprendizaje o independencia justifica el coste recurrente frente a la alternativa gestionada.

Por el contrario, conviene delegar cuando la disponibilidad es crítica y no puede garantizarse, cuando faltan competencias o tiempo para un mantenimiento riguroso, o cuando la economía del caso favorece de forma clara la externalización. La decisión, en suma, no es ideológica sino contextual: el autoalojamiento es una herramienta valiosa allí donde sus costes están justificados, y una imprudencia donde no lo están. Reconocerlo no debilita la tesis de la soberanía digital; la vuelve defendible, porque la sustenta en un examen sereno de los compromisos antes que en una preferencia previa.

2.8. Marco legal y regulatorio: RGPD, residencia de datos y NIS2

La soberanía digital no es únicamente una propiedad técnica de una arquitectura, sino también una categoría jurídica. Cuando se sostiene que el *self-hosting* devuelve al usuario el control sobre sus datos, conviene precisar qué significa «control» a la luz del derecho vigente, pues la capacidad material de retener un fichero en el propio disco no agota las obligaciones ni los derechos que el ordenamiento atribuye a quien trata información de carácter personal. Cabe distinguir, por tanto, dos planos que con frecuencia se confunden: la soberanía como dominio físico sobre la infraestructura y la soberanía como autodeterminación informativa reconocida y protegida por la ley. El autoalojamiento ofrece argumentos sólidos en el primer plano, pero solo se traduce en garantías efectivas cuando se atiende también al segundo.

2.8.1. RGPD y autodeterminación informativa

El Reglamento General de Protección de Datos (European Parliament & Council, 2016) constituye el marco de referencia en el espacio europeo y articula jurídicamente lo que el *self-hosting* persigue por medios técnicos. Su fundamento no es la mera confidencialidad, sino un derecho más amplio a la autodeterminación informativa: la facultad de la persona de decidir, en lo esencial, quién conoce qué sobre ella y con qué finalidad. De ahí derivan principios que el autoalojamiento puede materializar con particular fidelidad. La minimización

de datos —tratar solo lo necesario para una finalidad determinada— resulta natural en una arquitectura *local-first*, donde no opera el incentivo económico de acumular información para monetizarla que Zuboff (2019) identifica como motor de la economía de plataformas. La limitación de la finalidad, la exactitud y la integridad y confidencialidad encuentran asimismo expresión técnica directa en quien gobierna su propio *homelab*.

Ahora bien, el Reglamento no exime de obligaciones por el hecho de autoalojar; antes bien, traslada la responsabilidad de su cumplimiento a quien antes la delegaba en un proveedor. El principio de responsabilidad proactiva (*accountability*) exige no solo cumplir, sino poder demostrar el cumplimiento, lo que en un entorno *self-hosted* recae íntegramente sobre el titular de la infraestructura. Conviene matizar, además, que el tratamiento puramente doméstico o personal queda en buena medida fuera del ámbito de aplicación material del Reglamento; las obligaciones se activan con plenitud cuando el tratamiento desborda esa esfera estrictamente privada.

2.8.2. Residencia y jurisdicción de los datos

La residencia de los datos —el lugar físico donde la información se almacena y procesa— adquiere relevancia jurídica porque condiciona, en parte, qué legislación les resulta aplicable. El RGPD restringe las transferencias internacionales hacia terceros países que no ofrezcan un nivel de protección adecuado, y la jurisprudencia europea ha invalidado en sucesivas ocasiones los marcos que pretendían habilitar el flujo transatlántico de datos, al apreciar que la legislación de vigilancia de determinados terceros países comprometía las garantías exigidas. Conviene subrayar que residencia y jurisdicción no son equivalentes: un dato almacenado en territorio europeo pero gestionado por una entidad sujeta a la legislación de otro Estado puede quedar expuesto a requerimientos extraterritoriales. El *self-hosting* sobre infraestructura propia y físicamente localizada reduce de raíz esta ambigüedad, pues tiende a alinear la residencia, la jurisdicción y el control efectivo en un mismo punto. Se trata, quizá, de la ventaja jurídica más nítida del autoalojamiento frente a la nube comercial.

2.8.3. Responsable y encargado del tratamiento

El *self-hosting* reconfigura de manera significativa los roles que el RGPD define para repartir responsabilidades. El responsable del tratamiento es quien determina los fines y medios; el encargado, quien trata los datos por cuenta del anterior. En el modelo de nube convencional, el usuario o la organización actúan como responsables y el proveedor como encargado, vínculo que el Reglamento obliga a formalizar mediante un contrato de encargo con garantías precisas. El autoalojamiento colapsa esa dualidad: quien aloja su propio servicio reúne en una sola figura el rol de responsable y el de encargado, eliminando al tercero y, con él, la cadena de subencargados que en la práctica diluye la trazabilidad de la información. Esta concentración simplifica el cumplimiento en lo relativo a la confianza en terceros, pero lo intensifica en lo demás: no existe ya un proveedor sobre quien descargar las medidas de

seguridad, la respuesta ante brechas o la atención de los derechos de acceso, rectificación y supresión. La soberanía, también aquí, es indisociable de la responsabilidad.

2.8.4. NIS2 y obligaciones de seguridad

La Directiva NIS2, que actualiza el marco europeo de ciberseguridad para entidades esenciales e importantes, ilustra que la dimensión legal del cómputo soberano no se agota en la protección de datos personales. Impone obligaciones de gestión de riesgos, medidas técnicas y organizativas proporcionadas, notificación de incidentes significativos en plazos tasados y responsabilidad de los órganos de dirección. Aunque su ámbito subjetivo se circunscribe a determinadas categorías de organizaciones y no alcanza, por lo general, al particular que autoaloja servicios domésticos, su lógica resulta ilustrativa: la seguridad deja de ser una recomendación para convertirse en un deber jurídicamente exigible. Las prácticas que la literatura técnica recomienda —defensa en profundidad, mínimo privilegio, arquitecturas de confianza cero (Rose et al., 2020) y marcos de gestión del riesgo como el de NIST (National Institute of Standards and Technology, 2018)— adquieren así un respaldo normativo. El practicante de *homelab* que aspire a la soberanía sin asumir esta disciplina de seguridad obtiene, en rigor, una soberanía frágil.

2.8.5. Límites legales del autoalojamiento

Cabe finalmente advertir que la soberanía técnica no confiere inmunidad jurídica, y que existen ámbitos donde el autoalojamiento encuentra límites legales o prácticos difíciles de salvar. El correo electrónico autoalojado, técnicamente viable, choca con un ecosistema de reputación de remitentes que penaliza a los servidores domésticos hasta comprometer la fiabilidad de la entrega. El tratamiento de datos de menores está sujeto a salvaguardas reforzadas y a exigencias de consentimiento que el titular de un servicio no puede obviar por el mero hecho de controlarlo. Y, de modo más general, alojar datos de terceros —familiares, allegados, usuarios de un servicio compartido— convierte al autoalojador en responsable del tratamiento frente a esas personas, con todas las obligaciones que ello comporta, incluidas las de seguridad e información. La soberanía sobre los propios datos no se extiende, sin más, a los datos ajenos: ahí donde empieza el dato de otro, empieza también una responsabilidad que el ordenamiento no permite eludir.

3 Panorama del *self-hosting*

El *self-hosting* (autoalojamiento) designa la práctica de operar servicios digitales sobre infraestructura propia, bajo control directo del usuario, en lugar de delegarlos a proveedores de nube ajenos. Lejos de constituir un nicho técnico, esta práctica articula hoy un ecosistema maduro de aplicaciones que cubren el almacenamiento de archivos, la reproducción multimedia, la domótica, las comunicaciones, la gestión del conocimiento y la custodia de identidades y secretos. El propósito de este capítulo es trazar ese panorama de forma ordenada, de manera que las piezas dejen de percibirse como soluciones aisladas y se reconozcan como componentes de una infraestructura coherente al servicio de la *soberanía digital*.

La tesis que recorre el capítulo sostiene que el valor del autoalojamiento no reside en la suma de servicios individuales, sino en su integración dentro de un mismo dominio de confianza, el *homelab*, donde los datos permanecen bajo titularidad de quien los genera. Conviene, por ello, partir de una taxonomía que clasifique los servicios autoalojables según su función, para después examinar las categorías representativas: el almacenamiento con NAS y Nextcloud; la multimedia con Jellyfin y el ecosistema *arr*; la domótica con Home Assistant y la automatización local; las comunicaciones por correo, Matrix y mensajería; las herramientas de conocimiento y productividad; y la gestión de identidad, contraseñas y secretos. Cada categoría se aborda atendiendo a su peso propio, sin subordinarla a ningún otro eje.

Como cierre del recorrido, se sitúa la inteligencia local —modelos de lenguaje, voz y visión ejecutados en casa— en su lugar dentro de este mapa. Cabe advertir que dicha inteligencia no se presenta como finalidad última del autoalojamiento, sino como una capa más que, al apoyarse sobre los datos y servicios ya soberanos, habilita una *agencia agéntica* respetuosa con el principio *local-first*. Se observa así que el panorama del *self-hosting* ofrece el sustrato natural sobre el que la inteligencia agéntica local puede operar sin ceder el gobierno de la información a terceros.

3.1. Una taxonomía de servicios autoalojables

El ecosistema del *self-hosting* (autoalojamiento) abarca una variedad de servicios tan heterogénea que cualquier intento de comprenderlo exige un esquema clasificatorio. Una taxonomía no es un mero inventario: ordena el espacio de decisiones, revela qué propiedades comparten servicios aparentemente dispares y permite razonar sobre las consecuencias de alojar cada pieza en infraestructura propia. Conviene articularla en torno a tres ejes que operan de forma simultánea: el dominio funcional, la criticidad operativa y la madurez del software.

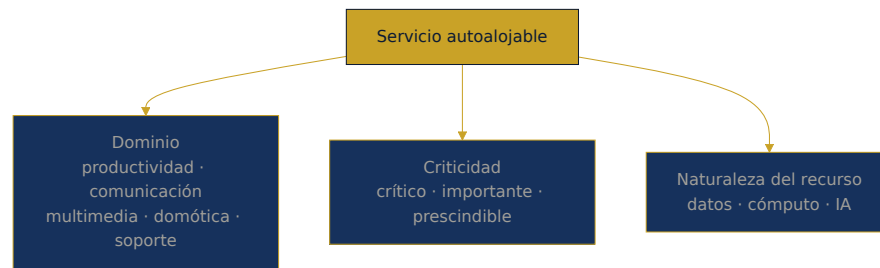


Figura 3.1: Los tres ejes simultáneos para clasificar un servicio autoalojable: el dominio funcional, la criticidad operativa y la naturaleza del recurso predominante (datos, cómputo o IA).

3.1.1. Clasificación por dominio y criticidad

El primer eje, el *dominio*, agrupa los servicios por la función que cumplen para el usuario o la organización. Cabe distinguir, entre otros, el dominio de la productividad personal (sincronización de archivos, notas, calendarios), el de la comunicación (correo, mensajería, videoconferencia), el de la gestión documental y multimedia, el de la automatización del hogar y el de la infraestructura de soporte (proxys inversos, gestión de identidad, monitorización). Plataformas como Nextcloud ilustran la convergencia de varios dominios en una sola pieza, al ofrecer almacenamiento, ofimática colaborativa y agenda bajo un mismo despliegue (Nextcloud GmbH, 2024); Home Assistant, por su parte, concentra el dominio domótico integrando dispositivos de fabricantes diversos en un plano de control local (Home Assistant, 2024).

El segundo eje, la *críticidad*, mide el coste de la indisponibilidad o la pérdida de datos. No todos los servicios merecen el mismo esfuerzo de resiliencia, por lo que resulta útil graduarlos en tres niveles. Los servicios *críticos* son aquellos cuya caída interrumpe funciones esenciales o cuya corrupción de datos es irreversible: la autenticación centralizada, el correo o el sistema de copias de seguridad. Los servicios *importantes* degradan la experiencia, pero admiten interrupciones acotadas, como un servidor multimedia. Los servicios *prescindibles* aportan valor sin que su ausencia comprometa nada esencial. Esta gradación gobierna decisiones de diseño concretas, como la frecuencia de las copias o la conveniencia de la redundancia. El razonamiento es análogo al de los marcos de gestión de riesgo, que priorizan los controles según su impacto sobre la confidencialidad, la integridad y la disponibilidad (NIST, 2018). Combinar ambos ejes en una matriz de dominio y criticidad previene un sesgo recurrente: dedicar un esfuerzo desproporcionado a servicios llamativos pero prescindibles, mientras la autenticación o las copias —invisibles hasta que fallan— quedan desatendidas.

3.1.2. Servicios de datos, de cómputo y de inteligencia artificial

Un tercer corte, ortogonal al dominio, atiende a la naturaleza del recurso predominante y arroja tres categorías con perfiles operativos muy distintos.

Los *servicios de datos* tienen como activo principal el estado persistente: bases de datos, almacenamiento de archivos, copias de seguridad. Su característica definitoria es que el dato sobrevive al proceso y constituye el valor irrecuperable. Aquí la integridad y la durabilidad priman sobre el rendimiento bruto, y las herramientas maduras de respaldo con deduplicación y cifrado autenticado, como restic o BorgBackup, son condición de seriedad antes que un lujo (restic contributors, 2024; BorgBackup contributors, 2024). Cuando estos servicios se replican entre nodos, emergen las tensiones fundamentales de los sistemas distribuidos entre consistencia, disponibilidad y tolerancia a particiones (Brewer, 2012; Gilbert & Lynch, 2002), un asunto que excede el despliegue doméstico habitual, pero condiciona cualquier arquitectura ambiciosa (Kleppmann, 2017).

Los *servicios de cómputo* tienen como activo el procesamiento: servidores de aplicaciones, transcodificación multimedia, motores de automatización. Su estado es a menudo efímero o reconstruible, lo que los convierte en candidatos naturales a la contenedorización, que empaqueta la aplicación y sus dependencias en una unidad reproducible y fácil de recrear (Merkel, 2014; Docker, 2024). Su límite suele residir en la capacidad de CPU o memoria, no en la durabilidad del dato.

Los *servicios de inteligencia artificial* constituyen una categoría emergente con exigencias singulares. Comparten con el cómputo su intensidad de procesamiento, pero la elevan: la inferencia de modelos de lenguaje, derivados de la arquitectura *transformer* (Vaswani et al., 2017), demanda aceleración por hardware (GPU o aceleradores dedicados) y una cuantiosa memoria. A diferencia de un servicio de cómputo convencional, su comportamiento es probabilístico y su superficie de ataque incluye vectores propios, como la inyección de instrucciones a través de los datos que procesa (Greshake et al., 2023; OWASP, 2025). Que la *agencia agéntica* local —la capacidad de un sistema para razonar y actuar de forma autónoma sobre su entorno— se sostenga sobre infraestructura propia depende, en buena medida, de la viabilidad de esta categoría en un *homelab*.

3.1.3. Criterios de madurez y mantenibilidad

Ninguna taxonomía estaría completa sin un eje que valore el software en sí. La *madurez* de un proyecto se infiere de señales observables: la actividad sostenida del desarrollo, la amplitud de la comunidad, la calidad de la documentación, la cadencia de los parches de seguridad y la existencia de una vía de actualización que no rompa los despliegues. La *mantenibilidad*, concepto afín pero distinto, mide el coste recurrente de operación: cuánto esfuerzo exige mantener el servicio actualizado, seguro y funcional a lo largo del tiempo.

Estos criterios introducen una asimetría decisiva frente al modelo de consumo en la nube. Al autoalojar, la organización asume íntegramente la responsabilidad operativa que un proveedor externo absorbería, de modo que un servicio inmaduro o difícil de mantener traslada un coste oculto y continuo al operador. La elección racional pondera, por tanto, la utilidad del servicio frente a ese coste, sin presuponer que toda función deba alojarse en local. Conviene desconfiar del proyecto vistoso pero abandonado y preferir el componente discreto y sólidamente mantenido, porque la sostenibilidad de un *homelab* se juega menos en su instante

de máxima capacidad que en su aptitud para seguir funcionando, sin sobresaltos, durante años.

3.2. Almacenamiento y archivos: NAS y Nextcloud

El almacenamiento constituye el sustrato sobre el que descansa cualquier estrategia de *self-hosting* (autoalojamiento): sin un lugar fiable donde residan los datos, los servicios superiores carecen de fundamento. En un *homelab*, este sustrato suele materializarse en un NAS (*Network Attached Storage*, almacenamiento conectado en red), un equipo dedicado que expone su capacidad a la red local mediante protocolos de ficheros como SMB o NFS. A diferencia del almacenamiento adjunto a una estación de trabajo concreta, el NAS centraliza la persistencia y la desacopla de los dispositivos de consumo, lo que permite que portátiles, teléfonos o contenedores accedan al mismo repositorio. Esta centralización es precisamente lo que habilita la soberanía digital sobre los datos: el control físico y lógico del medio donde se guardan recae en el operador, no en un proveedor externo.

3.2.1. Sistemas de ficheros, RAID y ZFS

La fiabilidad del almacenamiento no se deriva del hardware aislado, sino de la disciplina con que se gestiona la redundancia. Los discos mecánicos y los de estado sólido fallan, y un único dispositivo sin réplica representa un punto único de fallo inaceptable para datos no reproducibles. La técnica RAID (*Redundant Array of Independent Disks*, matriz redundante de discos independientes) agrupa varios discos para tolerar la pérdida de uno o más de ellos: RAID 1 duplica el contenido en espejo, mientras que RAID 5 y RAID 6 distribuyen información de paridad para sobrevivir, respectivamente, al fallo de uno o dos discos. Conviene subrayar que RAID protege frente a la avería de medios, no frente al borrado accidental ni frente a la corrupción de datos; no sustituye, por tanto, a una política de copias de seguridad.

La elección del sistema de ficheros añade una capa de garantías sobre esa redundancia. Los sistemas tradicionales como ext4 o XFS gestionan correctamente la organización de bloques, pero no detectan la corrupción silenciosa: alteraciones de bits que el medio no señala como error y que se propagan inadvertidas. ZFS aborda este problema combinando la gestión de volúmenes y el sistema de ficheros en una sola capa, con sumas de verificación de extremo a extremo para cada bloque, *snapshots* de bajo coste por copia en escritura y la capacidad de reparar datos dañados a partir de la redundancia disponible. Esta integración refleja un principio de diseño bien establecido: situar las garantías de integridad en el punto donde existe el conocimiento completo del dato, antes que confiarlas a capas intermedias que solo lo manipulan parcialmente (Saltzer et al., 1984). Frente a ZFS, alternativas como Btrfs ofrecen funcionalidades análogas de *snapshots* y sumas de verificación, con un grado de madurez históricamente desigual según la configuración empleada. La contrapartida de ZFS es su mayor consumo de memoria y la rigidez relativa para reconfigurar conjuntos de discos una vez creados.

3.2.2. Copias de seguridad

Puesto que la redundancia de discos no preserva frente al borrado, la corrupción lógica o el cifrado malicioso, la integridad de un repositorio autoalojado exige una política de copias de seguridad independiente del medio primario. La práctica consolidada se resume en la regla de mantener varias copias en soportes distintos, con al menos una fuera del emplazamiento físico, de modo que ningún fallo único comprometa todas las réplicas a la vez. Herramientas como BorgBackup o restic materializan este enfoque mediante copias deduplicadas, comprimidas y cifradas de forma autenticada, lo que reduce el espacio ocupado por versiones sucesivas y protege la confidencialidad del respaldo incluso cuando reside en almacenamiento remoto no confiable (BorgBackup contributors, 2024; restic contributors, 2024). El cifrado en el origen es aquí coherente con el principio de situar la garantía donde se posee el conocimiento completo del dato (Saltzer et al., 1984): el respaldo viaja y reposa cifrado, sin que el destino deba ser depositario de confianza alguna.

3.2.3. Nextcloud y la sincronización de archivos

Sobre el sustrato de almacenamiento se asienta la capa de servicios que lo hace utilizable a diario. Nextcloud es la plataforma de referencia en el autoalojamiento para la sincronización de archivos, el equivalente funcional y soberano de los servicios comerciales de disco en la nube (Nextcloud GmbH, 2024). Su modelo replica una carpeta entre el servidor y los clientes de escritorio o móviles, de modo que cada modificación se propaga al resto de dispositivos. Más allá del simple almacén, Nextcloud agrega edición colaborativa de documentos, calendario, contactos y un ecosistema de aplicaciones, lo que la convierte en un concentrador de la actividad personal o de un grupo reducido.

El valor de Nextcloud no reside únicamente en sustituir un servicio externo, sino en alinear la gestión de los datos con el paradigma *local-first*, donde la copia primaria reside en dispositivos bajo control del usuario y la disponibilidad no depende de la conectividad permanente con un tercero (Kleppmann et al., 2019). Esta orientación adquiere relevancia jurídica cuando se manejan datos personales, pues el control directo sobre la ubicación y el procesamiento facilita el cumplimiento de las obligaciones de protección de datos (European Parliament & Council, 2016) y limita la exposición a la lógica extractiva del capitalismo de vigilancia (Zuboff, 2019). La sincronización, sin embargo, no es un problema trivial: cuando un mismo fichero se edita en dos dispositivos sin haberse propagado el cambio previo, surge un conflicto. Nextcloud lo resuelve de forma conservadora, conservando ambas versiones, lo que evita la pérdida de datos a costa de exigir una reconciliación manual.

3.2.4. Alternativas: Syncthing y Seafile

El ecosistema ofrece enfoques distintos según la prioridad del operador. Syncthing prescinde por completo del servidor central: sincroniza carpetas directamente entre dispositivos mediante una topología de pares, cifrando el tráfico de extremo a extremo. Resulta idóneo

cuando interesa la replicación pura entre máquinas propias sin un punto de coordinación, aunque renuncia a la interfaz web y a las funciones colaborativas de Nextcloud. Seafile, en cambio, conserva el modelo cliente-servidor pero organiza los archivos en bibliotecas con un almacenamiento por bloques y deduplicación, lo que le confiere un rendimiento notable en la sincronización de grandes volúmenes y en el versionado. La elección entre estas opciones es un ejercicio de compromiso: Nextcloud maximiza la amplitud funcional, Seafile la eficiencia en el manejo de archivos, y Syncthing la simplicidad descentralizada.

3.2.5. Fotografías y documentos

Dos clases de datos justifican un tratamiento específico por su volumen y su valor personal. Las fotografías, generadas de forma continua por los dispositivos móviles, encuentran en aplicaciones autoalojadas como Nextcloud Memories o Immich una alternativa a los servicios comerciales de galería, con respaldo automático desde el teléfono, organización por fecha y lugar, y reconocimiento de objetos o rostros ejecutado en el propio servidor, sin que las imágenes abandonen la infraestructura controlada. Los documentos, por su parte, se benefician de la indexación y la búsqueda de texto completo; sistemas de gestión documental como Paperless-ngx aplican reconocimiento óptico de caracteres a los archivos digitalizados y los clasifican mediante etiquetas y metadatos, transformando un archivo inerte en un corpus consultable.

En conjunto, la pila de almacenamiento de un *homelab* ilustra el principio que vertebra el autoalojamiento: la persistencia, la integridad y la disponibilidad de los datos dejan de delegarse en un proveedor y pasan a ser responsabilidades asumidas de forma explícita. Esa asunción tiene un coste en esfuerzo de operación, pero a cambio restituye al usuario el control efectivo sobre la materia prima de su vida digital, condición necesaria para cualquier servicio soberano que se construya por encima.

3.3. Multimedia: Jellyfin y el ecosistema *arr*

La gestión multimedia constituye uno de los dominios donde el autoalojamiento (*self-hosting*) ha alcanzado mayor madurez funcional, hasta el punto de rivalizar en comodidad con los servicios comerciales de transmisión. Frente a las plataformas que arriendan un acceso revocable a catálogos ajenos, el modelo autoalojado materializa el principio *local-first*: la colección reside en infraestructura propia, se cataloga con metadatos enriquecidos y se sirve a cualquier dispositivo de la red doméstica o, mediante túnel cifrado, desde el exterior. Esta sección examina los servidores de medios, la automatización del ciclo de adquisición, las exigencias de transcodificación y las implicaciones jurídicas y éticas que el conjunto suscita.

3.3.1. Servidores de medios: del catálogo al reproductor

Un servidor de medios indexa archivos de vídeo, audio e imagen, los enriquece con carátulas, sinopsis y datos técnicos, y expone una interfaz de reproducción adaptada a televisores,

navegadores y aplicaciones móviles. En el ámbito del *homelab* coexisten varias opciones de referencia. *Plex* fue pionero en ofrecer una experiencia pulida, pero su arquitectura se apoya en una cuenta central y en servidores de autenticación remotos, lo que introduce una dependencia externa difícil de conciliar con la soberanía digital: el inicio de sesión y ciertas funciones avanzadas requieren la nube del proveedor. *Emby* adoptó un modelo intermedio, con núcleo abierto y extensiones de pago. *Jellyfin*, surgido como bifurcación libre de *Emby*, prescinde por completo de servicios externos: la autenticación, la biblioteca y la transmisión se resuelven en el propio servidor, sin telemetría ni licencias por usuario.

La elección de *Jellyfin* no es meramente ideológica. Al eliminar el intermediario de autenticación, el sistema sigue operativo aunque se interrumpa la conectividad con Internet, propiedad coherente con el ideal de que el usuario conserve el control de sus datos pese a la nube (Kleppmann et al., 2019). El precio es una menor pulcritud en algunos clientes y la ausencia de funciones de descubrimiento social. La comparación ilustra una tensión recurrente en el autoalojamiento: la conveniencia gestionada frente a la autonomía completa.

3.3.2. Automatización: el ecosistema *arr*

La adquisición y organización manual de una biblioteca extensa resulta inviable a escala. El denominado ecosistema *arr* —familia de aplicaciones cuyos nombres comparten el sufijo *-arr*— automatiza el ciclo completo mediante componentes especializados que cooperan a través de interfaces de programación. *Sonarr* gestiona series; *Radarr*, películas; *Lidarr*, música; *Readarr*, libros; y *Prowlarr* centraliza los indexadores que alimentan a los anteriores. El patrón es ilustrativo de una arquitectura de microservicios doméstica: cada herramienta cumple una función acotada y delega en *clientes de descarga* (como un cliente BitTorrent o de Usenet) la obtención efectiva de los archivos, que después renombra y archiva según convenciones uniformes.

Este encadenamiento revela una filosofía de diseño afín al principio de extremo a extremo (Saltzer et al., 1984): la inteligencia reside en los nodos especializados, mientras el transporte permanece neutro. La integración suele apoyarse en interfaces de estilo REST (Fielding, 2000), que permiten orquestar los componentes y exponer paneles unificados. El despliegue mediante contenedores (Merkel, 2014; Docker, 2024) simplifica la convivencia de tantos servicios, al aislar dependencias y normalizar la configuración, aunque a costa de una curva de aprendizaje notable en redes, volúmenes y permisos. La modularidad tiene contrapartida: la superficie de fallos se multiplica con el número de servicios, y la coordinación entre ellos exige una observabilidad que el operador debe construir de forma deliberada.

3.3.3. Transcodificación: el cuello de botella computacional

Servir un archivo no siempre equivale a reproducirlo. Cuando el dispositivo cliente no admite el códec, el contenedor o la resolución del original, el servidor debe *transcodificar*: decodificar el flujo y recodificarlo en tiempo real a un formato compatible. Esta operación es la más exigente del sistema y condiciona el dimensionado del *hardware*. La transcodificación

por software, que recae sobre la unidad central de proceso, ofrece máxima compatibilidad pero consume recursos desproporcionados; una sola conversión de alta definición puede ocupar varios núcleos de manera sostenida. La transcodificación acelerada por *hardware*, que delega el trabajo en bloques dedicados de la tarjeta gráfica o del procesador, incrementa la capacidad de flujos simultáneos con un consumo energético sensiblemente inferior.

De ahí que el diseño óptimo procure evitar la transcodificación siempre que sea posible, mediante la transmisión directa del archivo cuando el cliente lo soporta, y reserve la conversión para casos ineludibles como el acceso remoto con ancho de banda limitado. La decisión entre ambas estrategias expresa un compromiso entre compatibilidad universal y eficiencia, recurrente en todo el *homelab*.

3.3.4. Consideraciones legales y éticas

La capacidad técnica no exime de responsabilidad jurídica. Digitalizar y servir una colección de medios adquiridos legítimamente para uso personal se sitúa, en términos generales, dentro de los límites de la copia privada reconocida en numerosos ordenamientos, si bien su alcance exacto varía por jurisdicción y suele excluir la elusión de medidas tecnológicas de protección. La automatización del ecosistema *arr*, sin embargo, facilita con idéntica fluidez la obtención de obras protegidas sin licencia, conducta que infringe los derechos de autor y traslada al titular de la infraestructura una responsabilidad que ningún diseño técnico neutraliza.

Conviene, por tanto, distinguir el instrumento de su uso. Las mismas herramientas que ordenan una videoteca lícita pueden articular una distribución ilícita; la legitimidad reside en la procedencia del contenido, no en el *software*. Desde una perspectiva ética, el autoalojamiento multimedia ofrece un argumento sólido frente a la lógica extractiva de las plataformas comerciales, que tratan los hábitos de consumo como materia prima de explotación (Zuboff, 2019): recuperar el control sobre qué se ve, cuándo y sin vigilancia restituye una autonomía que el arrendamiento de catálogos erosiona. Esa legitimidad ética, no obstante, presupone el respeto a los derechos de quienes producen las obras; de lo contrario, la soberanía digital del usuario se construiría sobre la apropiación del trabajo ajeno. El autoalojamiento multimedia se revela así como un banco de pruebas donde la autonomía técnica y la responsabilidad jurídica han de equilibrarse de manera deliberada.

3.4. Domótica: Home Assistant y la automatización local

La domótica representa el punto donde el autoalojamiento (*self-hosting*) abandona el terreno exclusivo de los datos y los servicios para incidir directamente sobre el entorno físico: iluminación, climatización, cierres, riego, consumo eléctrico o presencia. Esta proyección sobre el mundo material agudiza las tensiones ya descritas en otros dominios del *homelab*, pues los datos que se manejan no son ficheros inertes, sino registros continuos de la actividad de las personas en su propio domicilio. La cuestión de quién observa, almacena y decide sobre

esa información deja de ser un asunto técnico para convertirse en una condición de la soberanía digital sobre el espacio íntimo. Home Assistant ocupa en este dominio una posición de referencia: la plataforma que permite sustraer la automatización del hogar al control de proveedores externos, integrando dispositivos heterogéneos bajo una administración propia (Home Assistant, 2024).

3.4.1. El dilema entre nube y cómputo local

El internet de las cosas (*Internet of Things*, IoT) comercial se ha consolidado en torno a un modelo donde el dispositivo es poco más que un terminal de un servicio en la nube. Un sensor de movimiento, un termostato o un altavoz inteligente capturan datos en el domicilio, los transmiten a la infraestructura del fabricante y reciben de vuelta las órdenes que ejecutan. Este diseño traslada la lógica de decisión fuera del hogar y subordina su funcionamiento a la conectividad permanente con un tercero. De ello se derivan tres consecuencias indeseables. La primera es la fragilidad: una interrupción de la conexión, o el simple cierre del servicio por parte del fabricante, puede inutilizar dispositivos plenamente funcionales en el plano físico. La segunda es la exposición: la actividad doméstica más cotidiana queda registrada en servidores ajenos, alimentando la lógica extractiva que caracteriza al capitalismo de vigilancia (Zuboff, 2019). La tercera es la pérdida de control sobre la propia automatización, cuyas reglas y límites los fija el proveedor.

El paradigma *local-first* invierte estos términos: la copia primaria de los datos y la lógica de decisión residen en infraestructura bajo control del usuario, y la dependencia de la nube se reduce a lo estrictamente opcional (Kleppmann et al., 2019). Home Assistant materializa este principio al ejecutarse íntegramente en un equipo del propio *homelab* —con frecuencia un computador de placa reducida— y concentrar en él la integración de los dispositivos, el almacenamiento del historial de estados y la evaluación de las automatizaciones. El sistema sigue gobernando la vivienda aunque la conexión exterior se interrumpa, propiedad que en el contexto domótico no es una comodidad sino un requisito de seguridad y habitabilidad.

3.4.2. Protocolos: el cómputo local como condición de interoperabilidad

La viabilidad de este modelo depende de que los dispositivos puedan comunicarse sin pasar por la nube del fabricante, lo que remite a la elección de protocolos. Los dispositivos basados en Wi-Fi tienden a orientarse al patrón de nube descrito, mientras que las tecnologías de malla de baja potencia favorecen la operación local. *Zigbee* establece una red mallada propia, coordinada por una pasarela conectada al servidor doméstico, en la que los dispositivos enchufados a la red eléctrica actúan como repetidores y extienden la cobertura sin saturar la red Wi-Fi. Su adopción local elude por completo los servidores del fabricante, a costa de requerir un coordinador físico y de afrontar cierta heterogeneidad histórica entre implementaciones.

Matter, estándar más reciente respaldado por buena parte del sector, persigue precisamente reducir esa heterogeneidad mediante un modelo de interoperabilidad que admite el control

local sobre redes IP, apoyándose en protocolos de malla de baja potencia para los dispositivos más limitados. Su propuesta es relevante para el autoalojamiento, pues normaliza el control sin intermediarios; conviene, no obstante, valorar con cautela hasta qué punto cada implementación concreta honra esa posibilidad o reintroduce dependencias de la nube por la vía de la puesta en marcha y la gestión de credenciales. La lección de fondo es que la soberanía sobre el hogar inteligente se decide, en buena medida, en la capa de protocolo: solo lo que puede comunicarse localmente puede gobernarse localmente.

3.4.3. Privacidad de los sensores y minimización de la exposición

La densidad de sensores que acompaña a una instalación domótica madura convierte el hogar en una fuente continua de datos de altísima sensibilidad. Patrones de presencia, horarios de sueño, hábitos de consumo o la sola apertura de una puerta componen, agregados, un retrato detallado de la vida privada. El procesamiento local de estos flujos constituye la salvaguarda más eficaz, pues sitúa el tratamiento en el punto donde reside el conocimiento completo del dato y evita que abandone el perímetro doméstico, en coherencia con el argumento de extremo a extremo (Saltzer et al., 1984). Cuando se manejan datos personales de este orden, el control directo sobre su ubicación y procesamiento facilita además el cumplimiento de las obligaciones de protección de datos (European Parliament & Council, 2016). Un caso ilustrativo es el reconocimiento de presencia o de imagen mediante cámaras: ejecutado en el servidor del *homelab*, el análisis nunca expone las grabaciones a un tercero, mientras que su delegación en una nube comercial implica transmitir vídeo del interior del domicilio a una infraestructura ajena. La minimización de la exposición no es, así, una mera buena práctica, sino la consecuencia directa de mantener el cómputo donde se origina el dato.

3.4.4. Automatizaciones y agencia agéntica

El valor de Home Assistant no se agota en agregar dispositivos, sino en habilitar automatizaciones: reglas que vinculan condiciones del entorno con acciones sobre él. Una automatización típica adopta la estructura de desencadenante, condición y acción —por ejemplo, atenuar la iluminación al detectar ausencia y caer la luz natural por debajo de un umbral—. Este esquema reactivo, gobernado por reglas declarativas, constituye una forma elemental y determinista de automatización: su comportamiento es enteramente predecible y auditable, virtud que en el control del entorno físico conviene primar.

Esta base reactiva enlaza con el eje conductor del trabajo. La agencia agéntica local —la capacidad de un sistema autoalojado para percibir, razonar y actuar de forma autónoma— encuentra en la domótica su sustrato de actuación natural: los sensores aportan la percepción y los actuadores, los efectos. Un agente capaz de interpretar instrucciones en lenguaje natural y de orquestar estos servicios extiende la automatización reactiva hacia una mediación más flexible, sin renunciar al cómputo local como condición. Conviene, sin embargo, subrayar la asimetría de riesgos: mientras una regla determinista falla de manera acotada, un agente con capacidad de actuación sobre el entorno físico amplía la superficie de consecuencias de un

error o de una manipulación, lo que aconseja conservar las automatizaciones deterministas como capa de garantía y acotar el alcance de cualquier decisión autónoma. La domótica autoalojada provee, en suma, el cuerpo sobre el que la agencia local puede ejercerse de forma soberana y, a la vez, recuerda que esa potencia exige contención proporcional a su impacto.

3.5. Comunicaciones: correo, Matrix y mensajería

Las comunicaciones constituyen un dominio especialmente revelador del *self-hosting* (autoalojamiento), porque en él la promesa de soberanía digital choca con realidades técnicas y económicas que el resto de servicios autoalojados no afrontan con igual crudeza. Conviene examinar tres planos —correo electrónico, mensajería en tiempo real y federación— y los compromisos que cada uno impone, pues ilustran con nitidez que la autonomía no es un atributo que se posea o no, sino una magnitud que se gradúa según cuánto trabajo de operación se está dispuesto a asumir.

3.5.1. El correo electrónico: el caso difícil

El correo es, paradójicamente, uno de los protocolos más antiguos y abiertos de internet y, a la vez, uno de los más arduos de autoalojar con garantías de servicio. La dificultad no reside en levantar los demonios de transporte y entrega: una pila moderna que combine un agente de transporte (SMTP), un servidor IMAP, un filtro antispam y autenticación de buzones puede desplegarse en contenedores con relativa comodidad (Docker, 2024). El problema es ajeno al servidor propio y se concentra en la *entregabilidad*: la probabilidad de que un mensaje legítimo alcance la bandeja de entrada del destinatario y no su carpeta de correo no deseado.

Esa probabilidad depende de la reputación de la dirección IP emisora, un activo que el operador individual no controla por completo. Los grandes proveedores de buzones evalúan cada conexión entrante contra listas de bloqueo, historiales de envío y señales de autenticación; una IP doméstica o de un rango asignado a hospedaje barato suele arrastrar mala reputación heredada de abusos previos, y muchos rangos residenciales están vetados *a priori* para el tráfico SMTP saliente. A ello se añade la obligación de configurar correctamente los mecanismos de autenticación de remitente —registros que vinculan el dominio con los servidores autorizados a enviar en su nombre y firmas criptográficas sobre el mensaje—, sin los cuales la entrega se degrada o se rechaza. La fiabilidad de la entrega resulta así, en buena medida, una propiedad emergente del sistema global y no del nodo individual: aunque la función de transporte se ejecute en los extremos, su éxito queda condicionado por el comportamiento de la red intermedia, lo que matiza la lectura más ingenua del argumento extremo a extremo (Saltzer et al., 1984).

El resultado es una asimetría incómoda: recibir correo de forma autoalojada es viable y robusto; enviarlo con plena autonomía es frágil. La práctica habitual consiste en delegar el envío en un servicio de retransmisión de reputación consolidada, conservando en propiedad el almacenamiento y la lógica de filtrado. Se mantiene entonces la soberanía sobre los datos —los

mensajes residen en infraestructura controlada, lo que importa frente al tratamiento masivo de comunicaciones con fines comerciales (Zuboff, 2019)— pero se cede parcialmente la soberanía sobre el canal de salida. Cabe leer este compromiso a la luz de los principios clásicos de protección: la economía de mecanismo y el privilegio mínimo aconsejan no asumir la complejidad de gestionar reputación de envío cuando el beneficio marginal es escaso frente al riesgo de degradar un servicio crítico (Saltzer & Schroeder, 1975).

3.5.2. Mensajería en tiempo real: Matrix y XMPP

Frente a la rigidez del correo, la mensajería instantánea autoalojada ofrece un terreno más favorable, articulado en torno a dos protocolos abiertos y federados. *XMPP* es el más veterano: un protocolo extensible basado en flujos de XML, modular y de implementación ligera, idóneo cuando se prioriza la eficiencia y un servidor de bajo consumo. Su extensibilidad mediante módulos opcionales es a la vez virtud y debilidad, pues la interoperabilidad efectiva entre clientes depende de qué extensiones soporte cada uno, lo que fragmenta funcionalidades como el cifrado de extremo a extremo o la sincronización entre dispositivos.

Matrix adopta una filosofía distinta. Modela cada conversación como un historial replicado entre los servidores participantes: cada *homeserver* mantiene una copia del grafo de eventos de las salas en las que tiene usuarios, y el estado actual de cada sala se obtiene resolviendo ese grafo de forma determinista. Este diseño emparenta a Matrix con los sistemas que priorizan la disponibilidad y la tolerancia a particiones replicando el estado y reconciliándolo más tarde, un planteamiento afín en su motivación al de las estructuras de datos que convergen sin coordinación central (Shapiro et al., 2011a). Tal opción ejemplifica además el compromiso que el teorema CAP impone entre consistencia y disponibilidad cuando la red se particiona, formalizado como la imposibilidad de garantizar ambas simultáneamente en un servicio distribuido (Gilbert & Lynch, 2002): Matrix sacrifica la consistencia inmediata para que una sala sobreviva a la caída temporal de servidores participantes y se reconcilie al restablecerse la conectividad. El coste es igualmente apreciable: la replicación del historial conlleva un crecimiento sostenido del almacenamiento y una carga de cómputo no despreciable, de modo que un *homeserver* en una *homelab* (laboratorio doméstico) modesta puede volverse pesado al federar salas muy concurridas.

3.5.3. Federación y compromisos

La federación es el rasgo que distingue estas arquitecturas de las plataformas centralizadas: cada operador gestiona su nodo y, aun así, sus usuarios se comunican con los de cualquier otro nodo sin una autoridad única. Tal modelo materializa el ideal *local-first*, en el que el dato y el control residen junto al usuario sin renunciar a la colaboración en red (Kleppmann et al., 2019). La contrapartida es operativa. Quien federa asume la responsabilidad de mantener el servicio disponible, actualizado y defendido, pues un nodo federado expone superficie a la red abierta; conviene tratar cada conexión entrante como no confiable por defecto y verificar identidad y autorización en el perímetro del propio servidor, en la línea de las arquitecturas

de confianza cero (Rose et al., 2020). La federación traslada, asimismo, una carga de moderación y de gestión de abusos que en los servicios centralizados queda invisibilizada bajo la operación del proveedor.

En suma, los tres planos ilustran un mismo patrón con intensidades distintas. El correo concentra el coste en una variable externa —la reputación— difícil de gobernar desde un único nodo; la mensajería en tiempo real desplaza ese coste hacia el consumo de recursos y la complejidad de federar estado replicado. En ambos casos, la soberanía digital no se obtiene gratuitamente, sino que se adquiere con trabajo de operación y con decisiones de diseño que dosifican, de forma deliberada, cuánta autonomía se retiene y cuánta se delega.

3.6. Conocimiento y productividad

La gestión personal del conocimiento constituye uno de los dominios donde el *self-hosting* (autoalojamiento) muestra con mayor nitidez su pertinencia. A diferencia de un servicio de almacenamiento genérico, las herramientas de conocimiento concentran el rastro intelectual de su propietario —lecturas, anotaciones, conexiones de ideas, proyectos en curso— y materializan de forma especialmente densa lo que suele denominarse *segundo cerebro*: un sistema externo de captura y organización destinado a complementar la memoria humana. Que ese repositorio resida en infraestructura propia, y no en una plataforma comercial sujeta a cambios de modelo de negocio, marca la diferencia entre poseer el conocimiento acumulado y arrendarlo.

3.6.1. Notas y bases de conocimiento

El núcleo de este dominio lo ocupan las aplicaciones de notas y las wikis personales. Herramientas como Obsidian o Joplin operan sobre archivos locales en formato abierto —típicamente *Markdown*—, lo que las inscribe de lleno en el paradigma *local-first*: los datos residen en el dispositivo del usuario y la sincronización, cuando existe, es un añadido y no un requisito de funcionamiento (Kleppmann et al., 2019). Esta propiedad tiene consecuencias prácticas. Un corpus de notas en *Markdown* sigue siendo legible y editable con cualquier editor de texto al cabo de los años, con independencia de que el programa original sobreviva; en cambio, una base de notas atrapada en un formato propietario y opaco queda condicionada a la continuidad del proveedor. La portabilidad del formato es, así, una garantía de soberanía digital tan importante como el control de la infraestructura, pues preserva el acceso al contenido aunque cambie la herramienta que lo gestiona.

Las wikis autoalojadas —de DokuWiki a soluciones más recientes como BookStack o Wiki.js— atienden un caso de uso contiguo pero distinto: la documentación estructurada y compartida, frente a la nota efímera y exploratoria. Su valor se aprecia sobre todo en el *homelab* mismo, donde conviene registrar configuraciones, decisiones de arquitectura y procedimientos de recuperación. La documentación operativa de la propia infraestructura constituye, en este sentido, un activo de conocimiento que no debería depender de un servicio externo cuya

caída podría coincidir, precisamente, con el momento en que esa documentación resulta más necesaria.

3.6.2. Marcadores, RSS y agregación

La soberanía sobre el conocimiento no se limita a lo que el usuario escribe, sino que abarca lo que consume y selecciona. Los gestores de marcadores autoalojados (Linkding, Shaarli) y los servicios de lectura diferida (Wallabag) preservan no solo los enlaces, sino con frecuencia una copia íntegra del contenido, lo que protege frente a la degradación de los enlaces y a la desaparición silenciosa de las páginas originales. Los agregadores de RSS (FreshRSS, Miniflux) recuperan, por su parte, un modelo de consumo informativo basado en la suscripción explícita y el orden cronológico, en contraposición a las cronologías algorítmicas de las plataformas comerciales. Este contraste no es meramente técnico. La lógica de la economía de la atención orienta esas cronologías hacia la maximización del tiempo de permanencia y la extracción de datos de comportamiento (Zuboff, 2019); un lector de RSS autoalojado devuelve al usuario el control sobre qué fuentes sigue y en qué términos, sin intermediación publicitaria ni perfilado.

3.6.3. El grafo de conocimiento personal

La evolución más significativa de estas herramientas es la adopción del enlazado bidireccional y la representación en grafo. En lugar de una jerarquía rígida de carpetas, las notas se vinculan entre sí mediante referencias explícitas, de modo que el conjunto adopta la forma de un grafo en el que los nodos son ideas y las aristas, relaciones. Este modelo se aproxima a la asociatividad del pensamiento humano y favorece la emergencia de conexiones no previstas en el momento de crear cada nota. Su relevancia para el eje de esta tesis es doble. Por un lado, el grafo constituye una estructura de datos rica y semánticamente densa, idónea como sustrato para el procesamiento posterior. Por otro, ese mismo grafo es materia prima apropiada para sistemas de recuperación aumentada —esto es, para dotar a un modelo de lenguaje de contexto personal recuperado en el momento de la consulta— sin necesidad de exponer los datos a terceros: la soberanía del segundo cerebro se convierte así en condición de posibilidad de una asistencia inteligente que no sacrifique la privacidad.

Conviene, no obstante, señalar una tensión recurrente. La sincronización de un grafo de notas entre varios dispositivos reintroduce el problema clásico de la consistencia ante ediciones concurrentes y desconexiones. Las soluciones más maduras en el espacio *local-first* recurren a tipos de datos replicados sin conflicto, que garantizan la convergencia de las réplicas sin coordinación central (Shapiro et al., 2011a). Mientras esa convergencia automática no esté plenamente disponible en todas las herramientas, el usuario asume una compensación deliberada: a cambio de la soberanía sobre sus datos, gestiona él mismo una sincronización que las plataformas comerciales ofrecen ya resuelta, aunque al precio de la cesión de esos mismos datos.

3.7. Identidad, contraseñas y secretos

La gestión de identidades y secretos constituye el sustrato sobre el que se asienta cualquier infraestructura autoalojada. En un despliegue de *self-hosting* (autoalojamiento), el administrador deja de delegar la autenticación en proveedores externos y asume la custodia íntegra de credenciales, claves y políticas de acceso. Esta asunción, coherente con el principio de *soberanía digital*, traslada al operador una responsabilidad que en los servicios comerciales permanece oculta: garantizar la confidencialidad de los secretos sin reintroducir, por la puerta trasera, las dependencias que el *homelab* pretendía eliminar.

3.7.1. Gestores de contraseñas autoalojados

El primer eslabón es el gestor de contraseñas. Vaultwarden, reimplementación ligera del servidor de Bitwarden compatible con sus clientes oficiales, permite alojar una bóveda cifrada en el propio dominio del usuario. Su diseño se alinea con el argumento del extremo a extremo (Saltzer et al., 1984): el cifrado y el descifrado de la bóveda ocurren en el cliente, de modo que el servidor almacena material opaco y nunca accede al texto en claro. Esta arquitectura resulta deseable porque acota la superficie de confianza al dispositivo del usuario y reduce el valor de una eventual exfiltración del servidor.

Conviene matizar, no obstante, los límites de esa garantía. El servidor sigue conociendo metadatos —número de entradas, marcas temporales, direcciones de correo— y, sobre todo, sirve los propios clientes web, lo que reabre el canal de ataque si la interfaz se ve comprometida. La clave maestra del usuario, de la que se deriva la clave de cifrado mediante funciones de coste configurable, es el único secreto que protege toda la bóveda; su robustez condiciona la del sistema entero. Frente al modelo comercial, donde la disponibilidad y el respaldo recaen en el proveedor, el autoalojamiento exige que el operador asuma de forma explícita la replicación y la copia de seguridad cifrada de la bóveda, pues su pérdida es irreversible.

3.7.2. Inicio de sesión único doméstico

A medida que el *homelab* acumula servicios —almacenamiento, automatización, paneles de administración—, multiplicar credenciales independientes degrada tanto la seguridad como la usabilidad. El inicio de sesión único (*single sign-on*, SSO) responde a esta dispersión centralizando la autenticación en un proveedor de identidad. En el ámbito doméstico predominan dos soluciones de perfil distinto. Authelia se sitúa como portal de autenticación y autorización ligero, habitualmente acoplado a un proxy inverso: intercepta cada petición y exige una sesión válida antes de reenviarla al servicio protegido. Keycloak, en cambio, es un servidor de identidad completo que implementa protocolos estándar como OAuth 2.0 (Hardt, 2012) y emite tokens en formato JWT (Jones et al., 2015), a costa de un mayor consumo de recursos y de una configuración más prolija.

La elección entre ambos ilustra una tensión recurrente en el autoalojamiento entre simplicidad operativa y completitud funcional. Authelia basta para proteger un conjunto reducido

de aplicaciones mediante autorización por proxy y un segundo factor; Keycloak se justifica cuando se requiere federación, gestión granular de roles o integración con aplicaciones que delegan formalmente en un emisor de identidad. En ambos casos, la centralización aporta un beneficio capital: un único punto donde aplicar políticas de contraseña, autenticación multifactor y registro de accesos, en línea con los controles que recomiendan los marcos de seguridad reconocidos (NIST, 2013). La autenticación multifactor resulta especialmente pertinente, pues un esquema *local-first* expuesto a internet hereda toda la hostilidad del entorno público sin el aparato defensivo de un proveedor a gran escala.

3.7.3. El problema del secreto raíz

La centralización desplaza el riesgo, pero no lo elimina. Tanto la bóveda de contraseñas como el proveedor de SSO descansan, en última instancia, sobre un *secreto raíz*: una clave maestra, una clave de firma de tokens o una credencial de base de datos cuyo compromiso desbarata todo el sistema de protección. Se reproduce así una recursión incómoda: para custodiar secretos se necesita un secreto que, por definición, no puede protegerse con el mismo mecanismo que ampara a los demás. Este problema del arranque de confianza carece de solución perfecta y solo admite mitigaciones.

Entre ellas figuran almacenar el secreto raíz fuera de línea o en un módulo de hardware, cifrar las copias de seguridad con una clave distinta de la operativa y aplicar el principio de mínimo privilegio para que ningún componente concentre más capacidad de la estrictamente necesaria (Saltzer & Schroeder, 1975). La defensa en profundidad y la segmentación reconocen que ninguna barrera es infalible y que conviene multiplicar las capas, criterio que comparten tanto los marcos de gestión de riesgo (NIST, 2018) como las arquitecturas de confianza cero, las cuales sustituyen la confianza implícita en la red por la verificación explícita y continua de cada acceso (Rose et al., 2020).

3.7.4. Identidad propia

El corolario de todo lo anterior es la posesión de una identidad propia. Frente al modelo dominante, en el que la identidad digital del individuo queda fragmentada y mediada por plataformas que la convierten en activo económico (Zuboff, 2019), el autoalojamiento de la identidad devuelve al usuario el control sobre quién autentica sus accesos y bajo qué condiciones. Disponer de un emisor de identidad propio permite, además, ejercer el cumplimiento normativo —por ejemplo, las exigencias de minimización y control de datos del RGPD (European Parliament & Council, 2016)— sobre infraestructura gobernada por el propio interesado. Esta autonomía, sin embargo, no es gratuita: comporta la carga permanente de operar, actualizar y respaldar los componentes críticos descritos. La soberanía sobre la identidad se sostiene, en definitiva, sobre la disciplina con que se gestionen sus secretos.

3.8. Evaluación de la IA local: calidad, latencia y límites del hardware

Una vez que el *homelab* incorpora modelos de lenguaje, voz y visión, surge la pregunta que decide si esa inteligencia local es algo más que una curiosidad técnica: ¿resulta lo bastante buena, lo bastante rápida y lo bastante barata de operar para el uso al que se la destina? Responderla exige medir, y medir un sistema de inferencia autoalojado es una disciplina con métricas propias, compromisos explícitos y umbrales que dependen tanto de la tarea como del hardware disponible. Conviene abordar el problema en tres planos —calidad, latencia y memoria— y cerrar con un criterio para decidir cuándo el cómputo soberano basta y cuándo conviene delegar en la nube. El objetivo no es maximizar una cifra abstracta, sino sostener la promesa *local-first* y la soberanía digital sin sacrificar la utilidad práctica.

3.8.1. Calidad: del benchmark genérico a la evaluación por tareas

La calidad de un modelo de lenguaje no es una magnitud única. Los *benchmarks* públicos —conjuntos estandarizados de preguntas de conocimiento, razonamiento o programación— ofrecen una primera ordenación gruesa y permiten comparar candidatos antes de descargarlos, pero arrastran dos limitaciones serias en el contexto del autoalojamiento. La primera es la contaminación: un modelo puede haber visto las respuestas durante su entrenamiento, lo que infla su puntuación sin reflejar competencia real. La segunda, más relevante para el *self-hosting*, es la distancia entre el banco de pruebas y el uso concreto: un modelo que destaca en preguntas de cultura general puede fallar al extraer campos de un correo en español o al invocar una herramienta con el formato exacto que el sistema agéntico espera.

Por ello, cabe argumentar que la evaluación útil en el *homelab* es la *evaluación por tareas*: construir un conjunto reducido de casos representativos del trabajo que se delegará al modelo —clasificar mensajes, resumir notas, decidir qué herramienta llamar— y verificar las respuestas. Cuando la corrección es objetiva (un campo extraído, una clasificación), la verificación puede automatizarse; cuando es subjetiva (la calidad de un resumen), se recurre a una valoración humana o, con cautela, a un modelo de mayor capacidad empleado como juez. Este enfoque conecta de forma natural con la recomendación de componer flujos simples y verificables antes que arquitecturas autónomas opacas (Anthropic, 2024): si cada paso del agente se puede evaluar de manera aislada, la calidad del conjunto deja de ser un acto de fe. Lo mismo se aplica a los patrones de razonamiento y acción intercalados sobre los que descansa la agencia agéntica (Yao et al., 2023), donde la fiabilidad del sistema depende de que cada decisión de invocar una herramienta sea, en sí misma, observable y evaluable.

3.8.2. Latencia y caudal: TTFB y tokens por segundo

La experiencia de uso de un modelo local depende menos de su puntuación en un *benchmark* que de dos magnitudes temporales. La primera es el *time to first byte* (TTFB), el tiempo que media entre el envío de la petición y la aparición del primer token; está dominado por la fase

de *prefill*, en la que el modelo procesa todo el contexto de entrada, y crece con la longitud de ese contexto. La segunda es el caudal de generación, medido en *tokens por segundo*, que gobierna la velocidad a la que se completa la respuesta una vez iniciada. En la fase de generación, el factor limitante en hardware doméstico no suele ser la potencia de cálculo, sino el ancho de banda de memoria: cada token nuevo exige recorrer la totalidad de los pesos del modelo, de modo que la velocidad queda acotada por la rapidez con que la memoria los entrega. Esta dependencia explica por qué una configuración con memoria unificada de alto ancho de banda puede superar a una unidad central de proceso convencional con la misma cantidad nominal de memoria.

El umbral de aceptabilidad varía con el modo de interacción. Para una respuesta que el usuario lee a medida que se genera, un caudal en torno a las decenas de tokens por segundo resulta cómodo, pues iguala o supera la velocidad de lectura. Para una interfaz de voz, en cambio, lo decisivo es la latencia acumulada de toda la cadena —reconocimiento, razonamiento, síntesis—, donde un TTFB elevado rompe la sensación de conversación aunque el caudal posterior sea alto. Medir, por tanto, no consiste en anotar una cifra de pico, sino en perfilar el comportamiento bajo la longitud de contexto y el número de peticiones simultáneas que el uso real impondrá.

3.8.3. Memoria y los compromisos de la cuantización

La memoria es la restricción que define qué modelos son siquiera ejecutables, y la *cuantización* —representar los pesos con menos bits por valor— es la palanca principal para gobernarla. Reducir de dieciséis a cuatro bits comprime el modelo aproximadamente a una cuarta parte de su tamaño, pero el compromiso no es lineal ni uniforme. Los esquemas de cinco y cuatro bits suelen preservar la calidad con una degradación apenas perceptible; por debajo de cuatro bits, en cambio, la pérdida se acelera y afecta antes a las tareas de razonamiento encadenado que a las de recuperación simple. Existe además un compromiso secundario que conviene no olvidar: la memoria no la consumen solo los pesos, sino también la caché de claves y valores del mecanismo de atención (Vaswani et al., 2017), que crece con la longitud del contexto. De ello se sigue que un modelo holgado con un contexto corto puede desbordar la memoria al alimentarlo con documentos extensos. La decisión de cuantización es, así, un equilibrio entre el mayor modelo que cabe, la calidad que conserva y el contexto que se quiere manejar.

3.8.4. Voz: WER y la medida de la síntesis

La inteligencia local rara vez se limita al texto, y sus componentes de voz exigen métricas distintas. La calidad del reconocimiento del habla (*Speech-to-Text*) se cuantifica mediante la *tasa de error de palabra* (*Word Error Rate*, WER), que computa la proporción de inserciones, supresiones y sustituciones respecto a una transcripción de referencia; una WER baja en condiciones de laboratorio puede deteriorarse notablemente con ruido ambiental, acentos o vocabulario específico, por lo que su medición debe realizarse sobre audio representativo del entorno doméstico real. La síntesis de voz (*Text-to-Speech*) carece de una métrica objetiva tan

limpia: su calidad —naturalidad y claridad— se evalúa de forma perceptual, mientras que su viabilidad para la interacción en tiempo real se mide por el factor de tiempo real, esto es, si el sistema genera audio más rápido de lo que dura su reproducción.

3.8.5. Los límites del hardware de consumo y el criterio de delegación

Cabe ser explícito sobre el techo del hardware doméstico sin acelerador dedicado. Una unidad central de proceso con memoria suficiente puede servir modelos pequeños cuantizados a velocidades de lectura aceptables, pero ofrece un caudal modesto, un TTFB elevado con contextos largos y una capacidad escasa para atender peticiones simultáneas; los modelos de mayor escala, donde reside la competencia en razonamiento complejo, quedan fuera de su alcance o se ejecutan a velocidades impracticables. Reconocer este límite no es una concesión, sino la base de un criterio racional de delegación.

Se argumenta que la decisión no debe plantearse como una disyuntiva absoluta entre lo local y la nube, sino tarea por tarea. La IA local es suficiente —y por tanto preferible— cuando la tarea está acotada y es evaluable, cuando la latencia alcanzable satisface el modo de uso, cuando el modelo que cabe en el hardware supera el umbral de calidad medido para esa tarea concreta, y cuando la sensibilidad de los datos o el deseo de independencia frente al proveedor inclinan la balanza, en coherencia con el principio *local-first* de que el usuario conserve sus datos pese a la nube (Kleppmann et al., 2019). La delegación al exterior se justifica, en cambio, ante tareas que exigen la capacidad de los modelos de frontera, ante picos de demanda que el hardware local no absorbe, o ante necesidades esporádicas cuyo coste no compensa la inversión en infraestructura. Este criterio, medido y no dogmático, prepara el terreno para las arquitecturas híbridas que el documento examinará más adelante, en las que el *homelab* resuelve localmente lo que puede y reserva para el exterior, de forma deliberada y trazable, solo aquello que excede sus límites.

3.9. Inteligencia local: LLMs, voz y visión en casa

La incorporación de capacidades de inteligencia artificial al *homelab* —el laboratorio doméstico de cómputo autoalojado— constituye uno de los desarrollos más significativos del *self-hosting* (autoalojamiento) reciente. Durante años, el procesamiento de lenguaje natural, el reconocimiento de voz o el análisis de imágenes se consideraron cargas de trabajo reservadas a centros de datos remotos, accesibles únicamente mediante interfaces de programación propietarias. La aparición de modelos abiertos suficientemente competentes, junto con técnicas de compresión que reducen sus exigencias de cómputo, ha desplazado esa frontera y ha hecho viable ejecutar inferencia de calidad razonable en hardware de consumo. Esta sección examina los componentes técnicos de esa inteligencia local y abre, a la vez, el eje que vertebra el conjunto del documento: la *agencia agéntica* local, entendida como la capacidad de un sistema autoalojado para razonar, decidir y actuar sobre su entorno sin delegar la cognición en servicios de terceros.

3.9.1. Modelos de lenguaje locales: cuantización y hardware

El núcleo de la inteligencia local lo ocupan los grandes modelos de lenguaje (*Large Language Models*, LLM), arquitecturas basadas en el mecanismo de atención que prescinde de la recurrencia para modelar dependencias a larga distancia en una secuencia (Vaswani et al., 2017). Un modelo con varios miles de millones de parámetros impone dos costes: el almacenamiento de los pesos y el ancho de banda de memoria necesario para recorrerlos en cada paso de generación. En precisión de coma flotante de 16 bits, un modelo de siete mil millones de parámetros requiere del orden de catorce gigabytes de memoria solo para los pesos, cifra que excluye a la mayoría de las tarjetas gráficas domésticas y a prácticamente todos los equipos sin acelerador dedicado.

La *cuantización* —la representación de los pesos con menor número de bits por valor— es la técnica que atenúa esta tensión. Reducir la precisión de 16 a 4 bits comprime el modelo aproximadamente a una cuarta parte de su tamaño, lo que permite alojar ese mismo modelo de siete mil millones de parámetros en torno a cuatro gigabytes y ejecutarlo en una tarjeta gráfica modesta o incluso en la unidad central de proceso con memoria suficiente. La contrapartida es una degradación de la calidad de las respuestas, por lo general leve en los esquemas de cuantización de 4 y 5 bits, pero perceptible en los más agresivos. Conviene, por tanto, distinguir tres perfiles de hardware doméstico: el equipo sin aceleración, capaz de servir modelos pequeños cuantizados a velocidades de lectura cómodas, aunque no plenamente conversacionales; la tarjeta gráfica de consumo con ocho a doce gigabytes de memoria, que cubre con holgura los modelos de siete a catorce mil millones de parámetros; y las configuraciones con memoria unificada de alta capacidad, que acercan al *homelab* modelos de treinta mil millones de parámetros o más. La elección no es trivial: gobierna directamente la latencia percibida, el número de peticiones simultáneas y el coste energético, variables que el resto del documento examinará al confrontar el cómputo soberano con la alternativa de la nube.

3.9.2. Voz: reconocimiento, síntesis y palabra de activación

Una interfaz de inteligencia local plenamente útil rara vez se limita al texto. La interacción por voz descansa en tres componentes encadenados. El primero es el reconocimiento del habla (*Speech-to-Text*, STT), que transcribe la señal acústica a texto; existen modelos abiertos de transcripción multilingüe que operan con buena precisión sobre una tarjeta gráfica de consumo e incluso, en sus variantes reducidas, sobre la unidad central de proceso. El segundo es la síntesis de voz (*Text-to-Speech*, TTS), que recorre el camino inverso; los sintetizadores neuronales autoalojables han alcanzado una naturalidad notable y permiten generar voz en tiempo real sin recurrir a servicios externos. El tercero, de naturaleza distinta, es la detección de la palabra de activación (*wake-word*): un modelo ligero y siempre activo que vigila el flujo de audio en busca de una expresión concreta y solo entonces despierta la cadena completa de procesamiento.

El diseño de esta cadena ilustra un principio recurrente en el autoalojamiento. Mantener el reconocimiento de la palabra de activación en el dispositivo, sin transmitir audio continuo a

ningún servidor, materializa el argumento extremo a extremo: las funciones que dependen del conocimiento del punto final de la comunicación se implementan mejor en ese punto final, no en los nodos intermedios (Saltzer et al., 1984). En el caso de la voz, ese conocimiento incluye una dimensión de privacidad, pues el audio del hogar nunca abandona el perímetro local, lo que neutraliza de raíz la captación de datos personales que caracteriza a los asistentes comerciales y que la literatura crítica ha situado en el centro de las economías de extracción de datos (Zuboff, 2019).

3.9.3. Visión local

El tercer pilar es el análisis de imagen y vídeo. Las tareas de visión por ordenador —detección de objetos, clasificación, lectura de matrículas, reconocimiento de presencia— resultan especialmente idóneas para el cómputo local, porque su valor depende de procesar flujos de cámaras del propio hogar cuya transmisión a la nube es indeseable tanto por coste de ancho de banda como por sensibilidad. Modelos de detección eficientes ejecutan inferencia sobre vídeo en tiempo real en aceleradores asequibles, y su integración con plataformas de automatización del hogar permite que un evento visual —la llegada de una persona, la aparición de un vehículo— desencadene acciones sin intermediación externa (Home Assistant, 2024). La visión local desplaza así la frontera entre el dato bruto y la inferencia hacia el interior del perímetro doméstico.

3.9.4. El coste y el beneficio de la agencia autoalojada

La inteligencia local adquiere su forma más ambiciosa cuando deja de limitarse a responder y empieza a actuar. Un sistema *agéntico* combina un modelo de lenguaje con la capacidad de invocar herramientas, observar sus resultados y encadenar decisiones hacia un objetivo. Los patrones que estructuran este comportamiento —la alternancia de razonamiento y acción que articula el ciclo de un agente (Yao et al., 2023), o la recomendación de componer flujos simples y verificables antes que arquitecturas autónomas opacas (Anthropic, 2024)— son independientes de dónde se ejecute el modelo, lo que convierte la soberanía sobre esa ejecución en la cuestión decisiva.

El balance entre coste y beneficio admite una lectura matizada. En el lado del coste figuran la inversión inicial en hardware, el consumo energético sostenido, el esfuerzo de operación y mantenimiento, y una brecha de capacidad real frente a los modelos de frontera servidos en la nube: un modelo cuantizado de catorce mil millones de parámetros no iguala a los sistemas comerciales de mayor escala en tareas de razonamiento complejo. En el lado del beneficio se alinean la privacidad por diseño, la ausencia de costes variables por petición, la independencia frente a cambios de precio o de política del proveedor, y la posibilidad de integrar el modelo con datos y dispositivos privados que difícilmente se confiarían a un tercero. A ello se añade una superficie de ataque específica y a menudo subestimada: los sistemas agénticos que consumen contenido externo son vulnerables a la inyección indirecta de instrucciones,

por la que datos aparentemente inertes alteran el comportamiento del agente (Greshake et al., 2023), riesgo que la confinación local mitiga pero no elimina.

De este modo, la inteligencia local no es un fin en sí mismo, sino la expresión más exigente del cómputo soberano: el punto donde el autoalojamiento deja de custodiar datos en reposo para asumir la cognición y la acción. Los capítulos siguientes desarrollan, sobre la infraestructura, las redes, el almacenamiento y la seguridad del *homelab*, las condiciones que hacen sostenible esa ambición.

3.10. Interoperabilidad y protocolos abiertos

La soberanía digital no se decide únicamente en el plano de la propiedad del hardware ni en el de la titularidad de los datos. Se decide, de forma menos visible pero igualmente determinante, en el plano de los protocolos: en las reglas que gobiernan cómo un sistema conversa con otro. Un servicio autoalojado que solo sabe hablar consigo mismo apenas mejora la posición del usuario respecto al servicio propietario que pretende sustituir; ambos lo confinan, el primero por aislamiento y el segundo por dependencia. Conviene por ello sostener una tesis fuerte: en el *self-hosting*, un protocolo abierto vale más que cualquier característica. Las características se copian, envejecen y se descartan; un protocolo abierto, una vez adoptado, redistribuye el poder de manera duradera, porque convierte la interoperabilidad en un derecho de facto en lugar de en una concesión revocable.

3.10.1. Estándares como condición de la soberanía

La arquitectura de internet ofrece el modelo de referencia. El argumento extremo a extremo de Saltzer, Reed y Clark (1984) sostiene que la inteligencia y el control deben residir en los puntos finales de la red, no en su núcleo, de modo que la red transporte sin imponer y los extremos decidan. Trasladado al ecosistema del *self-hosting*, este principio implica que el valor y la autoridad deben recaer en los servidores que el usuario controla, mientras que el protocolo se limita a garantizar un lenguaje común. Un estándar abierto cumple esa función cuando es público, admite múltiples implementaciones independientes y no requiere permiso de una autoridad central para participar. La pila clásica de correo —IMAP y SMTP— ilustra el caso maduro: cualquier servidor puede entregar mensajes a cualquier otro porque el formato y el transporte están especificados con independencia de quién los implemente. Que el correo siga siendo, pese a su concentración comercial, federable y autoalojable se debe precisamente a que su protocolo nació abierto y nadie ha podido cerrarlo del todo.

3.10.2. El fediverso y la federación

El fediverso extiende esta lógica a las aplicaciones sociales. Un protocolo de publicación y suscripción entre servidores permite que instancias distintas intercambien mensajes, seguimientos y respuestas sin pertenecer a una misma organización. La consecuencia es estructural: la identidad de una persona deja de estar atada a una plataforma y pasa a residir en un

dominio que puede ser propio. Quien aloja su propia instancia conserva la conversación aunque otra instancia desaparezca o cambie de políticas. La mensajería federada en tiempo real aplica el mismo principio, con la particularidad de que cada conversación puede replicarse como un historial compartido entre los servidores participantes, de modo que ninguno es la fuente única de verdad. Esta replicación distribuida no es gratuita: introduce los problemas clásicos de los sistemas distribuidos, donde la disponibilidad y la consistencia entran en tensión bajo particiones de red (Gilbert y Lynch, 2002), y donde el orden de los eventos solo puede establecerse mediante relaciones causales explícitas en ausencia de un reloj común (Lamport, 1978). La investigación sobre tipos de datos replicados sin conflicto muestra que es posible diseñar estructuras que convergen al mismo estado sin coordinación central, lo que ofrece una vía técnica para reconciliar réplicas divergentes en sistemas federados y *local-first* (Shapiro et al., 2011). La federación, en suma, traslada al protocolo una complejidad que los jardines vallados resuelven por simple centralización; el precio de la soberanía es asumir esa complejidad en lugar de delegarla.

3.10.3. Datos personales y formatos portables

En el terreno de los datos personales, CalDAV y CardDAV cumplen para calendarios y contactos el papel que IMAP cumple para el correo: estandarizan el acceso y la sincronización de modo que el cliente y el servidor puedan provenir de proveedores distintos. Su valor no está en ninguna prestación brillante, sino en que desacoplan la libreta de direcciones de la aplicación que la muestra. Aquí se aprecia con nitidez la diferencia entre interoperabilidad y mera exportación. Un jardín vallado puede ofrecer un botón de exportación y proclamarse abierto; sin embargo, exportar es un acto puntual y unidireccional, mientras que interoperar es una relación continua y simétrica. La portabilidad genuina exige formatos abiertos y documentados —vCard, iCalendar, texto plano y estándares de marcado—, porque solo un formato cuya especificación es pública garantiza que los datos seguirán siendo legibles cuando el programa que los creó ya no exista. El paradigma *local-first* articula esta exigencia como principio de diseño: el software debe garantizar la propiedad efectiva de los datos al margen de la nube, lo que pasa por formatos que el usuario pueda leer y transformar con herramientas independientes (Kleppmann et al., 2019).

3.10.4. Protocolos para la IA agéntica local

Esta misma lógica empieza a desplegarse en el dominio emergente de la IA agéntica. Un agente local resulta útil en la medida en que puede actuar sobre herramientas, datos y servicios; pero si cada integración exige un conector propietario, el agente queda atrapado en un nuevo jardín vallado, esta vez disfrazado de asistente. La estandarización de la forma en que un modelo descubre e invoca herramientas y accede a recursos externos responde a esta necesidad: permite que un mismo servidor de herramientas sirva a clientes distintos y que la agencia agéntica se vuelva portable entre modelos y proveedores. Su importancia para el cómputo soberano es directa, pues habilita que el usuario aloje sus propias herramientas y

exponga sus propios recursos sin depender del catálogo cerrado de un fabricante. La advertencia es la habitual en sistemas distribuidos abiertos: ampliar la superficie de interacción amplía también la superficie de ataque, como muestran los trabajos sobre inyección indirecta de instrucciones en aplicaciones que integran modelos de lenguaje (Greshake et al., 2023). Un protocolo abierto no exime del diseño cuidadoso de la confianza; al contrario, lo vuelve más exigente, porque la apertura que habilita la composición también habilita el abuso. La soberanía sobre la agencia agéntica, así, no consiste en renunciar a la interoperabilidad, sino en gobernarla con límites de privilegio explícitos.

3.10.5. El protocolo frente al jardín vallado

Cabe distinguir, para concluir, dos formas de pensar el valor en el software. La primera lo sitúa en las características, atributos que un proveedor controla y que constituyen su ventaja competitiva; la segunda lo sitúa en los protocolos, lenguajes compartidos que ningún proveedor controla en exclusiva. El jardín vallado prospera maximizando las primeras y minimizando los segundos, porque la interoperabilidad erosiona el coste de abandonarlo. El *self-hosting* invierte esa prioridad: prefiere un servicio modesto que hable un protocolo abierto a un servicio brillante que solo hable consigo mismo, porque el primero preserva la libertad de salida y el segundo la consume. Se argumenta, en definitiva, que la interoperabilidad no es un rasgo deseable entre otros, sino la condición técnica de la soberanía digital. Donde hay protocolo abierto, el usuario puede marcharse sin perder lo suyo; donde solo hay características, permanece porque marcharse cuesta demasiado. La soberanía, así entendida, no es la ausencia de dependencia —ningún sistema es autosuficiente—, sino la libertad de elegir y de revocar las dependencias, libertad que únicamente los protocolos abiertos hacen efectiva.

4 Arquitectura y operación

El presente capítulo desciende del plano conceptual al plano material: examina cómo se construye y se sostiene una infraestructura de cómputo soberano en el entorno de un *homelab* (laboratorio doméstico). Si los capítulos previos justifican por qué conviene autoalojar servicios y datos, aquí se argumenta cómo hacerlo de forma fiable, segura y reproducible. La tesis que vertebra estas páginas es que la soberanía digital no se decreta, sino que se opera: depende menos de la adopción de una herramienta concreta que de un conjunto coherente de decisiones técnicas sobre dimensionamiento, aislamiento, exposición, persistencia y recuperación. Sin esa base operativa, cualquier promesa de autonomía resulta frágil.

Se recorre, en orden, la pila completa que da soporte al autoalojamiento. Conviene partir del *hardware* y su dimensionamiento, atendiendo al consumo energético como restricción real de un entorno doméstico; sobre él se asienta la ejecución de servicios mediante contenedores y su orquestación. A continuación se aborda la red local —segmentación, resolución de nombres y descubrimiento de servicios— y la cuestión crítica de la exposición segura hacia el exterior mediante *reverse proxy*, túneles y WireGuard. La discusión prosigue con la gestión de datos y estado, las copias de seguridad bajo la regla 3-2-1, la observabilidad de logs, métricas y salud, y, por último, la reproducibilidad entendida como infraestructura como código. Cada sección aporta el criterio de diseño y no solo el procedimiento.

Aunque el eje de la tesis es la inteligencia agéntica local, esta no introduce una arquitectura distinta, sino que intensifica exigencias ya presentes en el *self-hosting* (autoalojamiento) general. Un agente que ejecuta acciones sobre el sistema, accede a datos personales y mantiene estado a lo largo del tiempo eleva el listón en materia de aislamiento, persistencia, trazabilidad y capacidad de restauración. Por ello, los principios que aquí se exponen sirven por igual a un servicio convencional y a una carga de trabajo agéntica: la segunda se trata como un caso particularmente exigente de la primera, no como una excepción que requiera reglas propias.

4.1. Hardware y dimensionamiento; energía

El dimensionamiento del hardware constituye la primera decisión material de cualquier despliegue de *self-hosting* (autoalojamiento) y condiciona el resto de la arquitectura: la capacidad de cómputo disponible delimita qué servicios pueden coexistir, qué cargas de inferencia local resultan viables y a qué coste energético sostenido. Conviene abordar esta decisión desde un principio de proporcionalidad: el equipo debe ajustarse a la carga real prevista y a su crecimiento razonable, no a la máxima carga imaginable. El sobredimensionamiento, además de

un desembolso ocioso, penaliza de forma permanente el consumo eléctrico; el infradimensionamiento obliga a migraciones tempranas que erosionan la fiabilidad del *homelab*. Entre ambos extremos, el criterio rector es dimensionar para la carga sostenida y prever cabeceras de ampliación allí donde resulten baratas.

4.1.1. Del mini-PC al rack

El espectro de plataformas viables es amplio y admite una gradación nítida. En el extremo inferior se sitúan los ordenadores de placa única, tipo Raspberry Pi, adecuados para servicios ligeros y de baja concurrencia: un servidor DNS, un agregador de notas o un panel domótico. Su atractivo reside en un consumo de pocos vatios y un coste de adquisición mínimo, a cambio de una capacidad de ampliación y un rendimiento de entrada/salida limitados.

El escalón intermedio, y probablemente el punto óptimo para la mayoría de instalaciones domésticas, lo ocupan los mini-PC de bajo consumo, habitualmente equipos de oficina reaprovechados o modelos compactos de arquitectura x86 con procesadores de gama móvil. Ofrecen virtualización completa, soporte de contenedores sin penalizaciones de emulación —ventaja relevante frente a las plataformas ARM cuando se ejecutan imágenes pensadas para x86 (Docker, 2024)— y un consumo en reposo contenido. Sobre un único equipo de este tipo cabe consolidar mediante contenedores una docena de servicios sin dificultad, pues la contenedorización reduce la sobrecarga frente a la virtualización completa al compartir el núcleo del anfitrión (Merkel, 2014).

El extremo superior corresponde a las torres con capacidad de expansión y a los montajes en *rack* de diecinueve pulgadas, propios de quien gestiona almacenamiento masivo, virtualización densa o varias unidades de procesamiento gráfico. El *rack* aporta densidad, modularidad y gestión térmica ordenada, pero introduce un coste de entrada, un consumo y, sobre todo, un nivel de ruido que pocas viviendas toleran sin una sala dedicada. La transición de la torre al *rack* rara vez se justifica por capacidad bruta; suele responder a la necesidad de organizar redundancia, conmutación de red gestionada y alimentación ininterrumpida en un mismo bastidor.

4.1.2. CPU, memoria, almacenamiento y GPU

Cabe distinguir cuatro recursos con perfiles de demanda muy distintos. La **CPU** rara vez es el cuello de botella en cargas de servidor convencionales —servir páginas, sincronizar archivos, ejecutar bases de datos modestas—, donde el número de núcleos importa más que la frecuencia de pico, porque la concurrencia entre contenedores se beneficia del paralelismo. La excepción la marca la inferencia de modelos de lenguaje sin aceleración gráfica, donde la CPU sí domina y la latencia solo resulta aceptable para modelos pequeños y cuantizados.

La **memoria RAM** suele ser el primer límite real al consolidar servicios. Cada base de datos, cada caché y cada tiempo de ejecución reserva su porción, y la holgura de memoria determina cuántos servicios conviven sin recurrir a intercambio en disco, que degrada el rendimiento de forma desproporcionada. Una regla prudente consiste en presupuestar memoria con margen

amplio desde el inicio, pues su ampliación posterior es barata cuando el zócalo lo permite y costosa cuando obliga a sustituir la plataforma entera.

El **almacenamiento** exige separar dos dimensiones: rendimiento y capacidad. Las unidades de estado sólido NVMe aportan la latencia que necesitan las bases de datos y el sistema operativo; los discos mecánicos siguen siendo la opción razonable por terabyte para datos fríos y copias de seguridad. La redundancia mediante agrupaciones de discos protege frente a fallos de hardware, pero no sustituye a una política de respaldo: preserva la disponibilidad ante un disco averiado, no ante un borrado accidental, una corrupción lógica o un cifrado malicioso, escenarios que solo cubren copias versionadas y verificadas (restic contributors, 2024; BorgBackup contributors, 2024).

La **unidad de procesamiento gráfico** merece consideración aparte por su papel en la *agencia agéntica* local. La inferencia de modelos basados en transformadores se beneficia del paralelismo masivo del hardware gráfico, dado que su arquitectura descansa en operaciones matriciales densas y mecanismos de atención plenamente paralelizables (Vaswani et al., 2017). El factor decisivo no es la potencia bruta, sino la memoria de vídeo disponible, que determina qué tamaño de modelo cabe cargar. Una GPU de gama media con doce o dieciséis gigabytes permite ejecutar modelos cuantizados de utilidad práctica; saltar a modelos mayores exige una memoria que multiplica el coste de adquisición y, de forma sostenida, el consumo. Conviene, por tanto, decidir primero qué clase de modelo se desea servir y dimensionar la GPU en consecuencia, y no a la inversa.

4.1.3. Consumo y coste energético

El coste energético no es un detalle accesorio, sino una partida recurrente que, a lo largo de la vida útil del equipo, puede aproximarse al precio de compra o superarlo. Un servidor que consume cuarenta vatios de media en funcionamiento continuo acumula del orden de 350 kilovatios-hora al año, magnitud que, a las tarifas eléctricas europeas, constituye un argumento de peso a favor de los equipos de bajo consumo en reposo. La métrica relevante no es el consumo de pico, que se alcanza esporádicamente, sino el consumo en reposo y a carga media, donde un servidor doméstico pasa la práctica totalidad de su tiempo.

De aquí se derivan dos consecuencias de diseño. Primera, conviene favorecer plataformas con buena gestión de estados de bajo consumo y evitar componentes ociosos de alto gasto, señaladamente GPU potentes que permanecen encendidas para una inferencia esporádica. Segunda, la consolidación mediante contenedores sobre un único equipo eficiente resulta casi siempre preferible, en términos energéticos, a repartir la carga entre varias máquinas, pues elimina el consumo basal multiplicado de cada una.

4.1.4. Ruido y ubicación

La ubicación física condiciona el tipo de hardware tolerable. Un equipo destinado a un espacio habitado debe operar en silencio, lo que favorece la refrigeración pasiva o los ventiladores de gran diámetro girando despacio, y descarta la mayoría de servidores de *rack*, diseñados

para salas técnicas donde el ruido es irrelevante. La temperatura ambiente, la circulación de aire y la acumulación de polvo afectan directamente a la longevidad del hardware y a la frecuencia de giro de los ventiladores. Un emplazamiento ventilado, fresco y de acceso cómodo para el mantenimiento vale más, en la práctica, que varias especificaciones nominales superiores.

4.1.5. Segunda mano y sostenibilidad

El mercado de segunda mano y los equipos corporativos retirados ofrecen una vía especialmente racional para construir un *homelab*. Estaciones de trabajo y servidores de hace pocas generaciones se adquieren a una fracción de su precio original y conservan capacidad sobrada para las cargas descritas, con la ventaja añadida de una construcción robusta y abundante documentación. Esta opción tiene además una dimensión de sostenibilidad: prolongar la vida útil de hardware existente evita la huella ambiental asociada a la fabricación de equipo nuevo. La contrapartida es un consumo en reposo a menudo mayor en el material empresarial antiguo, por lo que conviene contrastar el ahorro en la compra con el sobrecoste energético a lo largo de los años de servicio previstos. La decisión sensata pondera ambos factores en lugar de absolutizar uno solo, y reconoce que la *soberanía digital* se sostiene también sobre una gestión material y económicamente responsable de la infraestructura.

4.2. Contenedores y orquestación

La operación de servicios en un *homelab* (laboratorio doméstico de cómputo) exige una unidad de despliegue que sea reproducible, portable y aislable. El contenedor cumple esa función: empaqueta una aplicación junto con sus dependencias en una imagen inmutable que se ejecuta sobre el núcleo del anfitrión sin la sobrecarga de un sistema operativo huésped completo. A diferencia de la máquina virtual, que virtualiza el hardware, el contenedor comparte el *kernel* y aísla los procesos mediante primitivas del propio Linux —espacios de nombres (*namespaces*) y grupos de control (*cgroups*)—, lo que reduce el consumo de memoria y acelera el arranque (Merkel, 2014). Esta ligereza es decisiva en infraestructura autoalojada, donde el cómputo y la energía son recursos finitos y un único nodo suele concentrar decenas de servicios heterogéneos. La inmutabilidad de la imagen, además, aporta una propiedad operativa central: el entorno de ejecución deja de depender del estado acumulado en el anfitrión y pasa a estar descrito de forma explícita, lo que reduce la divergencia entre la configuración documentada y la que efectivamente corre.

4.2.1. Docker y Compose como unidad de despliegue

Docker popularizó este modelo al ofrecer un formato de imagen, un registro distribuido y un flujo de construcción declarativo mediante el *Dockerfile* (Docker, 2024). Sin embargo, la unidad operativa relevante en el autoalojamiento rara vez es un contenedor aislado, sino un conjunto de servicios interdependientes: una base de datos, un servidor de aplicación, un

proxy inverso y, con frecuencia, un servicio de caché. Docker Compose responde a esta necesidad describiendo la topología completa en un fichero declarativo: define los servicios, las redes que los comunican, los volúmenes que persisten su estado y las variables de entorno que los configuran. El valor de esta descripción no es meramente operativo, sino documental: el fichero *Compose* constituye una especificación versionable de la arquitectura, de modo que el despliegue se vuelve reproducible y la recuperación ante un fallo de hardware se reduce a clonar un repositorio y restaurar los volúmenes de datos. Conviene insistir en esta separación: la imagen y su configuración son efímeras y desechables; el estado vive en volúmenes explícitos cuya copia de seguridad debe planificarse de forma independiente. Esta distinción entre lo desechable y lo persistente es la que permite tratar la infraestructura como código y razonar sobre la continuidad del servicio en términos de datos, no de máquinas concretas.

4.2.2. Aislamiento y sus límites

El aislamiento que ofrece el contenedor es real pero gradual, y entenderlo con precisión es condición de una operación segura. Los espacios de nombres separan la vista que cada contenedor tiene de procesos, red, puntos de montaje e identificadores de usuario; los grupos de control limitan el consumo de CPU, memoria y entrada/salida. No obstante, todos los contenedores comparten el mismo *kernel*, de manera que una vulnerabilidad de escalada en él compromete potencialmente a todos. Esta frontera, más débil que la de un hipervisor, obliga a aplicar el principio de mínimo privilegio (Saltzer & Schroeder, 1975): ejecutar los procesos sin usuario *root*, descartar capacidades innecesarias, montar los sistemas de ficheros en modo de solo lectura cuando sea posible y evitar el modo privilegiado salvo justificación expresa. La superficie de ataque se reduce además eligiendo imágenes base mínimas y manteniéndolas actualizadas, pues cada paquete superfluo es una dependencia que vigilar. Cabe subrayar que estas mitigaciones operan en capas: ninguna basta por sí sola, y su valor proviene de la acumulación de barreras independientes. El aislamiento, en suma, no se obtiene por defecto: se configura.

4.2.3. Podman como alternativa sin demonio

Docker se apoya históricamente en un demonio central que se ejecuta con privilegios de administrador y media en todas las operaciones. Esta arquitectura concentra el riesgo: quien controla el demonio controla el anfitrión. Podman propone un modelo alternativo, compatible con el mismo formato de imagen y con una interfaz de línea de órdenes casi idéntica, pero sin demonio persistente y con soporte de primera clase para contenedores *rootless*, ejecutados íntegramente con privilegios de usuario no administrador. Para el autoalojamiento, donde un solo operador asume la responsabilidad de la seguridad, esta reducción de superficie es valiosa, pues alinea la herramienta con el mismo principio de mínimo privilegio que gobierna el resto de la operación. Podman se integra además con *systemd*, lo que permite gestionar los contenedores como servicios del sistema con sus mecanismos nativos de arranque, supervisión y registro, en lugar de delegar esa función en un demonio adicional. La compatibilidad con los ficheros *Compose* facilita la migración sin reescribir la descripción de la

infraestructura, de modo que el coste de cambiar de motor no recaer sobre la especificación de la arquitectura, sino únicamente sobre el sustrato que la ejecuta.

4.2.4. Orquestadores ligeros

Cuando los servicios superan la capacidad de un único nodo, o cuando se busca recuperación automática ante fallos y reprogramación de cargas, aparece la orquestación. El referente del sector es Kubernetes, pero su complejidad operativa resulta desproporcionada para la mayoría de los laboratorios domésticos. Existen alternativas concebidas para entornos de recursos modestos. *k3s* es una distribución de Kubernetes reducida a un binario único, que sustituye componentes pesados por equivalentes ligeros y conserva la compatibilidad con la interfaz de programación estándar; resulta idóneo cuando se desea aprender o reproducir el ecosistema de Kubernetes sobre hardware limitado, como un pequeño grupo de ordenadores de placa reducida. Nomad, por su parte, ofrece un orquestador de un solo binario, más simple conceptualmente, capaz de planificar no solo contenedores sino también procesos nativos y máquinas virtuales, lo que conviene cuando la carga de trabajo es heterogénea. Ambos introducen, no obstante, una capa de abstracción —programación, descubrimiento de servicios, almacenamiento distribuido— cuyo coste de mantenimiento debe justificarse. La elección entre ellos no es solo técnica: depende de cuánta complejidad operativa puede absorber de forma sostenible quien administra la infraestructura.

4.2.5. Cuándo no usar Kubernetes

La decisión de orquestar admite un análisis sobrio. Kubernetes resuelve problemas de escala, alta disponibilidad y operación de múltiples equipos sobre flotas de nodos; aporta poco, y a un coste elevado, cuando la infraestructura cabe en uno o dos anfitriones gobernados por un único operador. La alta disponibilidad que promete exige al menos tres nodos de control para el consenso del almacén de configuración, hardware que muchos laboratorios no poseen ni necesitan. Su modelo de red, su gestión de volúmenes y su superficie de configuración imponen una carga cognitiva permanente que compite con el objetivo mismo del autoalojamiento: que los servicios funcionen de manera fiable con esfuerzo sostenible. Por ello, la recomendación razonable es escalar la herramienta a la necesidad. Para la mayoría de los escenarios domésticos, Docker o Podman con Compose, complementados con copias de seguridad disciplinadas y supervisión básica, ofrecen reproducibilidad y aislamiento suficientes. La orquestación distribuida se introduce solo cuando concurren requisitos verificables de disponibilidad, escala o reprogramación automática que las herramientas más simples no satisfacen. Adoptar Kubernetes por anticipación, antes de que tales requisitos existan, suele traducirse en complejidad sin contrapartida, contraria a la economía de medios que define un *homelab* bien operado.

4.3. Redes domésticas: segmentación, DNS, descubrimiento

La red doméstica deja de ser un mero conducto de acceso a internet cuando alberga servicios autoalojados (*self-hosting*). Pasa a constituir el sustrato sobre el que reposan la disponibilidad, la confidencialidad y la operabilidad de cuanto se ejecuta en el *homelab*. Conviene, por tanto, abordar su diseño con criterios análogos a los de una infraestructura productiva, aunque ajustados a la escala y a las restricciones del hogar. Cabe distinguir cinco ejes de trabajo: la segmentación lógica, la resolución de nombres local, el descubrimiento de servicios, la adopción de IPv6 y el filtrado perimetral e interno. Los cinco se entrelazan: una decisión en cualquiera de ellos condiciona a los demás, y la tensión recurrente —que se examina en varios puntos— enfrenta el aislamiento que exige la seguridad con la conectividad que reclama la comodidad de uso.

4.3.1. Segmentación: VLAN y aislamiento del IoT

La red plana —un único dominio de difusión donde todo dispositivo alcanza a cualquier otro— resulta cómoda pero frágil. Un termostato, una cámara o un altavoz inteligente comprometidos quedan, en ese modelo, a un salto de los servicios que custodian datos personales. La segmentación mediante *VLAN* (redes de área local virtuales, normalizadas en IEEE 802.1Q) divide la infraestructura física en varios dominios lógicos sobre el mismo cableado y los mismos conmutadores, de modo que el tráfico entre segmentos atraviesa obligatoriamente un encaminador donde se aplican políticas explícitas.

Un esquema doméstico razonable separa, como mínimo, una VLAN de confianza para equipos personales, una VLAN de servidores donde residen los servicios autoalojados, una VLAN de dispositivos del *Internet de las cosas* (IoT) y una VLAN de invitados con acceso únicamente a internet. El aislamiento del IoT es especialmente pertinente: tales dispositivos rara vez reciben actualizaciones de seguridad, telemetrizan hacia el exterior y exhiben una superficie de ataque opaca. Recluirlos en un segmento sin acceso lateral a la VLAN de servidores limita el daño de un compromiso y materializa el principio de privilegio mínimo (Saltzer & Schroeder, 1975). Esta compartimentación interna anticipa, además, la lógica de la arquitectura de confianza cero (Rose et al., 2020): la pertenencia a la red interna no confiere por sí sola legitimidad, y cada flujo entre segmentos se autoriza de forma individualizada en lugar de presuponerse a partir de la mera ubicación topológica.

La segmentación introduce, eso sí, una tensión operativa con el descubrimiento de servicios, que se examina más abajo: los protocolos de difusión y multidifusión no cruzan los límites de VLAN sin mediación expresa.

4.3.2. DNS local

La resolución de nombres es la pieza que vuelve habitable un *homelab*. Memorizar direcciones IP resulta impracticable en cuanto el número de servicios crece, y depender de resolutores

públicos para nombres internos es indeseable por latencia, por privacidad y por la imposibilidad de nombrar recursos que no existen fuera de la red. Conviene operar un resolutor DNS propio —*dnsmasq*, *Pi-hole* o *Unbound* son opciones habituales— que sirva un dominio interno reservado y reenvíe el resto de consultas hacia resolutores recursivos.

La elección del sufijo importa. El uso de pseudo-dominios como `.local` colisiona con *mDNS* (descrito a continuación), y `.home` o los nombres inventados carecen de garantía de no colisionar con el espacio público; resulta más prudente delegar un subdominio realmente poseído (por ejemplo, `casa.ejemplo.com`) o emplear el dominio especial `home.arpa`, reservado para este fin. Un resolutor local centraliza también el filtrado de dominios publicitarios y de seguimiento, y permite encaminar las consultas hacia el exterior por un canal cifrado —apoyado en TLS (Rescorla, 2018)—, de manera que el proveedor de acceso no observe el patrón de navegación. La contrapartida es que el resolutor se convierte en un punto único de fallo: si cae, la red percibe una avería generalizada aunque la conectividad subsista, lo que aconseja prever un resolutor secundario.

4.3.3. Descubrimiento de servicios

Frente al DNS configurado de forma explícita, el descubrimiento automático permite que los dispositivos anuncien y localicen servicios sin intervención manual. Las dos familias predominantes son *mDNS* (DNS de multidifusión) junto a *DNS-SD* —base de las implementaciones conocidas como Bonjour o Avahi— y *SSDP*, empleado por *UPnP*. Ambas se apoyan en difusión o multidifusión y, por diseño, quedan confinadas al dominio de capa de enlace: no atraviesan las fronteras de VLAN.

Aquí reaparece la tensión señalada. La segmentación que aísla el IoT rompe, simultáneamente, el descubrimiento que muchos de esos dispositivos requieren para ser controlados desde la VLAN de confianza. La solución no consiste en colapsar los segmentos, sino en reenviar de forma selectiva el tráfico de multidifusión mediante un *reflector* de *mDNS* (Avahi en modo reflexión, o las funciones equivalentes de los encaminadores) que replique únicamente los anuncios pertinentes entre VLAN concretas, conservando el aislamiento del tráfico unidireccional ordinario. Conviene, por su parte, deshabilitar *UPnP* en el encaminador de frontera: su capacidad para abrir puertos hacia internet de forma automática contradice cualquier postura defensiva seria.

4.3.4. IPv6

La adopción de IPv6 introduce un cambio de paradigma respecto a la traducción de direcciones (NAT) que caracteriza a IPv4. En IPv6, los anfitriones reciben direcciones globalmente encaminables, de manera que el NAT deja de actuar como cortafuegos implícito. Esta circunstancia se interpreta a menudo, erróneamente, como una pérdida de seguridad; en rigor, restaura la conectividad extremo a extremo que el modelo original de internet suponía (Saltzer et al., 1984), trasladando la responsabilidad del filtrado a donde corresponde: una política explícita de cortafuegos.

El operador del *homelab* debe atender a varios matices. Los prefijos delegados por el proveedor pueden ser dinámicos, lo que complica las reglas de filtrado y los registros DNS estables; las extensiones de privacidad —direcciones temporales que rotan— dificultan asociar un flujo a un anfitrión concreto en los registros; y la coexistencia con IPv4, el escenario de doble pila, obliga a mantener coherentes dos juegos de políticas. Ignorar IPv6, dejándolo activo pero sin filtrar, expone los servicios internos directamente a internet sin que el operador lo perciba, un fallo silencioso más peligroso que su uso deliberado.

4.3.5. Cortafuegos doméstico

El cortafuegos articula las decisiones anteriores. En el perímetro, una postura de denegación por defecto en sentido entrante —solo se admite lo explícitamente autorizado— constituye la base mínima. Internamente, las reglas inter-VLAN concretan la segmentación: la VLAN de servidores no inicia conexiones hacia la de IoT salvo en los flujos previstos, los invitados no alcanzan recurso interno alguno y el IoT no contacta con los servicios personales. Soluciones de encaminamiento abiertas como OPNsense o pfSense, o el filtrado nativo del núcleo Linux (nftables), permiten expresar estas políticas con la granularidad necesaria.

La exposición de servicios al exterior merece una mención aparte. En lugar de abrir puertos entrantes —lo que amplía la superficie de ataque y exige direcciones fijas o resolución dinámica—, resulta preferible un túnel cifrado de acceso remoto, como los que provee WireGuard (Donenfeld, 2017), que mantiene el cortafuegos perimetral cerrado a toda conexión no iniciada desde dentro o no autenticada criptográficamente. Así, el filtrado deja de ser una barrera estática para convertirse en la expresión operativa de una política de mínimo privilegio coherente con el resto de la arquitectura.

4.4. Exposición segura: reverse proxy, túneles y WireGuard

Un *homelab* (laboratorio doméstico de cómputo) cobra buena parte de su utilidad cuando sus servicios resultan accesibles desde fuera de la red local: el calendario consultado desde el teléfono en movilidad, el repositorio de ficheros disponible durante un viaje o el agente local invocado desde una conexión ajena. Esa accesibilidad, sin embargo, abre una superficie de ataque que conviene tratar con disciplina. La cuestión central no es *si* exponer, sino *cómo*, y la respuesta articula tres mecanismos complementarios —el proxy inverso, el túnel de salida y la red privada virtual— que pueden combinarse según el grado de exposición tolerable. El principio rector es de raíz clásica: conceder el mínimo privilegio y reducir al estricto necesario la superficie expuesta, en la línea del diseño económico de los mecanismos de protección (Saltzer & Schroeder, 1975).

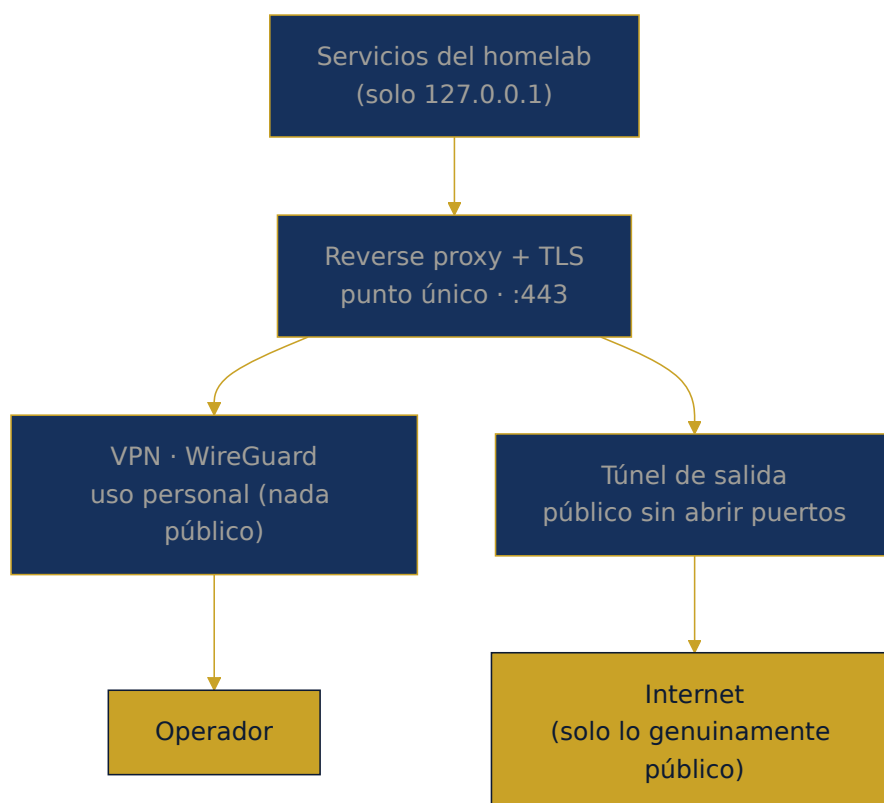


Figura 4.1: Los tres mecanismos de exposición ordenados por grado: la red privada (VPN) para el uso personal y el túnel de salida para lo genuinamente público —sin abrir puertos— se apoyan ambos en un proxy inverso con TLS como sustrato común que media, cifra y autentica.

4.4.1. El proxy inverso y el cifrado en tránsito

El primer error operativo, y el más extendido, consiste en abrir un puerto por cada servicio y redirigirlo directamente desde el encaminador hacia el contenedor correspondiente. Esta práctica multiplica los puntos de entrada, expone implementaciones internas sin una capa de mediación y dificulta la gestión coherente del cifrado. El *reverse proxy* (proxy inverso) resuelve ese desorden interponiendo un único punto de entrada que recibe todas las peticiones HTTP y las encamina internamente al servicio adecuado en función del nombre de dominio o de la ruta solicitada (Fielding & Reschke, 2014). De este modo, una sola dirección y un único puerto —el 443— atienden a numerosos servicios, mientras la topología interna permanece oculta tras el mediador.

Esta concentración habilita además la terminación centralizada de TLS (*Transport Layer Security*, seguridad de la capa de transporte), el protocolo que cifra y autentica el tráfico web en su versión vigente (Rescorla, 2018). El proxy presenta los certificados, descifra la conexión entrante y reenvía la petición al servicio interno, que así queda relevado de gestionar criptografía. Los exponentes habituales de esta categoría reflejan filosofías distintas. Algunos automatizan la obtención y renovación de certificados mediante el protocolo ACME contra

una autoridad de certificación gratuita, con una configuración declarativa de pocas líneas que los hace idóneos para el operador individual. Otros priorizan la integración dinámica con orquestadores de contenedores, descubriendo servicios y generando rutas a partir de etiquetas, lo que encaja en despliegues que cambian con frecuencia. Los más veteranos, de configuración explícita, ofrecen máximo control y rendimiento a costa de gestionar la renovación de certificados por medios externos. La elección depende del compromiso buscado entre automatismo, dinamismo y control granular; para un *homelab* gobernado por una persona, el automatismo de la renovación tiende a inclinar la balanza.

El proxy inverso aporta también un emplazamiento natural para funciones transversales: cabeceras de seguridad, limitación de tasa, registro centralizado de accesos y, sobre todo, autenticación previa. Anteponer un portal de identidad que valide la sesión antes de que la petición alcance la aplicación protege incluso a servicios que carecen de autenticación robusta propia, materializando una defensa en profundidad coherente con un modelo de confianza cero, donde ninguna petición se considera fiable por su mero origen en la red (Rose et al., 2020). Ese portal puede apoyarse en estándares de delegación y aserción de identidad ya consolidados, como la autorización por *tokens* y su representación verificable (Hardt, 2012; Jones et al., 2015), de modo que la credencial se valide una sola vez en el borde y se propague de forma controlada.

4.4.2. Túneles de salida: exponer sin abrir puertos

Exponer el proxy inverso exige todavía una dirección pública alcanzable, lo que no siempre es viable: muchas conexiones residenciales operan tras NAT de operador (*Carrier-Grade NAT*) o asignan direcciones dinámicas, y abrir puertos entrantes sigue revelando la ubicación del domicilio. El túnel de salida invierte el planteamiento. En lugar de aceptar conexiones entrantes, el servidor establece una conexión *saliente* y persistente hacia un punto de presencia externo, que actúa como intermediario y reenvía hacia el túnel el tráfico dirigido al dominio. Bajo este modelo, el cortafuegos del domicilio no abre ningún puerto entrante, la dirección de red doméstica permanece oculta tras la red del intermediario y este puede absorber además mitigación de denegación de servicio y filtrado de borde.

La contrapartida es de naturaleza tanto técnica como conceptual. Cuando el intermediario termina el TLS, ve el tráfico en claro, lo que reintroduce una dependencia de un tercero precisamente en una arquitectura cuyo propósito es la soberanía digital. Se trata de un compromiso legítimo cuando se prioriza la robustez frente a la exposición directa, pero contradice el ideal *local-first* (prioridad de lo local) si se generaliza sin matices (Kleppmann et al., 2019). Conviene, por ello, reservar el túnel para los servicios que deban ser genuinamente públicos y excluir de él los datos más sensibles, cuyo recorrido debería mantenerse íntegro de extremo a extremo en coherencia con el argumento de situar las funciones críticas en los extremos de la comunicación (Saltzer et al., 1984).

4.4.3. Red privada virtual: WireGuard y mallas cifradas

Buena parte de los servicios de un *homelab* no necesitan ser públicos: basta con que sean accesibles para su operador. Para ese caso, la red privada virtual ofrece la solución más alineada con la soberanía, pues no expone nada al conjunto de la red abierta. *WireGuard* constituye hoy una referencia: un túnel cifrado de nivel de núcleo, deliberadamente minimalista, con una base de código reducida que facilita su auditoría y un rendimiento competitivo frente a las soluciones precedentes (Donenfeld, 2017). Su modelo criptográfico, basado en pares de claves y en una lista explícita de pares autorizados, sustituye la negociación compleja por una configuración estática y verificable.

WireGuard, no obstante, presupone que al menos un extremo posea una dirección alcanzable y exige gestionar manualmente claves y rutas, una fricción que crece con el número de dispositivos. Sobre su mismo protocolo se han construido capas de coordinación que añaden un plano de control encargado de la identidad, la distribución de claves y el atravesamiento de NAT, de modo que los dispositivos forman una malla cifrada sin configuración manual de rutas. El precio es la dependencia de ese plano de control externo para la coordinación, si bien el tráfico de datos circula directamente entre pares y no a través del proveedor. La disyuntiva reproduce un patrón recurrente: la configuración manual maximiza la autonomía a costa del esfuerzo operativo; la malla coordinada sacrifica una parte de esa autonomía a cambio de comodidad y escalabilidad. La decisión, de nuevo, se resuelve ponderando soberanía frente a conveniencia, sin que exista una respuesta única para todo despliegue.

4.4.4. Criterio de exposición

Los tres mecanismos no compiten, sino que se ordenan por grado de exposición. La red privada es la opción por defecto para todo servicio de uso personal; el túnel de salida atiende lo que deba ser público sin sacrificar la ocultación de la red doméstica; y el proxy inverso con TLS automático constituye el sustrato común que, en cualquiera de los dos casos, media, cifra y autentica. Estos mecanismos no operan en el vacío: deben inscribirse en un programa de seguridad que contemple identificación, protección, detección y respuesta como funciones articuladas (NIST, 2018), de manera que la exposición de servicios se gobierne con criterios reproducibles y no caso a caso. La máxima operativa que recorre esta sección es sencilla y exigente: no abrir un puerto sin haber justificado por qué ningún mecanismo menos expuesto bastaba.

4.5. Datos y estado: bases de datos y persistencia

La operación de un *homelab* (laboratorio doméstico de cómputo) descansa sobre una distinción que el modelo de contenedores hace explícita pero no resuelve por sí mismo: la separación entre lo efímero y lo persistente. Un contenedor encarna un proceso reproducible y desechable; su sistema de ficheros, salvo intervención expresa, se destruye con él. El estado

—las filas de una base de datos, los ficheros de configuración, los índices, los secretos— constituye, en cambio, el activo cuya pérdida no se repara recreando una imagen. Conviene por tanto partir de un principio operativo: en una infraestructura autoalojada (*self-hosting*) se asume que cualquier contenedor puede morir en cualquier momento, y que el valor reside íntegramente en lo que sobrevive a esa muerte.

4.5.1. Volúmenes y el estado fuera del contenedor

El mecanismo canónico para externalizar el estado es el volumen. La documentación del entorno de contenedores distingue entre volúmenes gestionados por el motor y montajes directos de directorios del anfitrión (Docker, 2024); ambos persiguen el mismo fin, desacoplar el ciclo de vida del dato del ciclo de vida del proceso. La virtud de los contenedores como unidad de despliegue ligera y reproducible (Merkel, 2014) se vuelve trampa cuando el estado se aloja dentro de la capa de escritura efímera: una actualización de imagen, una recreación rutinaria del contenedor o un fallo del orquestador bastan para aniquilarlo. La regla de diseño es nítida: el contenedor debe poder reconstruirse desde su definición declarativa sin pérdida alguna, lo que obliga a que todo dato con valor resida en volúmenes explícitos. Esta exigencia reproduce, en el plano operativo, el argumento extremo a extremo (Saltzer et al., 1984): la garantía de durabilidad no puede delegarse en una capa intermedia, sino que debe situarse donde el dato realmente se custodia.

4.5.2. Postgres y SQLite: dos topologías de persistencia

La elección del motor de base de datos no es accesorio, pues determina la topología misma de la persistencia. SQLite materializa la base de datos como un único fichero en el volumen: no hay proceso servidor, no hay puerto, no hay gestión de conexiones. Esta simplicidad la hace idónea para servicios de baja concurrencia y para despliegues donde minimizar componentes es prioritario; su límite es la escritura concurrente, serializada mediante bloqueos a nivel de fichero. PostgreSQL, en cambio, introduce un proceso servidor con control de concurrencia multiversión, tipos ricos y un ecosistema de extensiones; su precio es un componente más que administrar, respaldar y actualizar. La decisión se enmarca en compromisos bien estudiados entre consistencia, disponibilidad y tolerancia a particiones (Brewer, 2012): en un único nodo doméstico, donde la partición de red interna es improbable, el dilema se desplaza hacia la durabilidad local y la integridad transaccional más que hacia la replicación distribuida. La distinción entre modelos de almacenamiento orientados a transacciones y los orientados a análisis o a grandes volúmenes (Kleppmann, 2017) ayuda a situar cada motor frente a la carga prevista. Conviene resistir la tentación de adoptar el motor más potente por defecto; la sobreingeniería de la capa de datos multiplica las superficies de fallo sin beneficio proporcional.

4.5.3. Migraciones de esquema

El esquema evoluciona con el software, y esa evolución es una de las fuentes más comunes de incidentes en la operación autoalojada. Una actualización de imagen que presupone una versión de esquema distinta de la persistida puede dejar el servicio inservible o, peor, corromper datos silenciosamente. La práctica robusta exige tratar las migraciones como artefactos versionados, idempotentes y aplicados de forma controlada, preferentemente automáticos al arranque pero auditables. El operador de un *homelab* debe verificar la compatibilidad entre la versión de la aplicación y la del esquema antes de promover una actualización, y disponer de un punto de retorno: un respaldo consistente tomado inmediatamente antes de migrar. La irreversibilidad de muchas migraciones convierte esta cautela en condición necesaria, no en refinamiento opcional.

4.5.4. Integridad y durabilidad

La integridad del estado se sostiene sobre dos pilares complementarios. El primero es la garantía transaccional que ofrece el propio motor: atomicidad y durabilidad de las escrituras confirmadas, de modo que un corte eléctrico no deje la base en un estado intermedio incoherente. El segundo es la verificación de los respaldos. Un respaldo de un fichero de base de datos copiado mientras el motor escribe puede resultar inconsistente; de ahí que se prefieran volcados lógicos coherentes o instantáneas tomadas con el servicio quiescido. Las herramientas de copia con deduplicación, compresión y cifrado autenticado (BorgBackup contributors, 2024; restic contributors, 2024) aportan, además, verificación de integridad del propio archivo de respaldo mediante comprobación criptográfica, lo que detecta la corrupción del medio antes de que se necesite restaurar. Cabe insistir en que un respaldo no verificado mediante una restauración de prueba es una hipótesis, no una garantía.

4.5.5. El estado como activo crítico

La conclusión que articula esta sección es una inversión de prioridades respecto a la intuición ingenua. En una arquitectura de contenedores efímeros, el cómputo es fungible y el estado es irremplazable. El esfuerzo de diseño debe concentrarse, por tanto, donde se concentra el valor: en la custodia, la verificación y la recuperabilidad del dato. Tratar el estado como ciudadano de primera clase —externalizado en volúmenes, gobernado por un motor adecuado a la carga, migrado con disciplina y respaldado con verificación— es lo que distingue una infraestructura autoalojada operable de una colección frágil de servicios. La soberanía digital sobre los propios datos carece de sentido si esos datos no sobreviven con integridad al primer incidente; la persistencia bien gobernada es la condición material que sostiene esa soberanía.

4.6. Copias de seguridad y recuperación (3-2-1)

La continuidad de un *homelab* (laboratorio doméstico de servidores y servicios autoalojados) no descansa en la ausencia de fallos, sino en la capacidad de recuperarse de ellos. Un nodo de cómputo soberano concentra datos cuya pérdida sería irreversible: documentos personales, históricos de domótica, modelos ajustados localmente o el estado de los agentes que operan sin intermediación de terceros. A diferencia del modelo de nube comercial, donde la durabilidad se delega en el proveedor, en el paradigma *local-first* (datos y lógica residentes primero en el dispositivo del usuario) la responsabilidad de preservar la información recae enteramente en quien la administra (Kleppmann et al., 2019). De ahí que la estrategia de copias de seguridad no sea un complemento operativo, sino una condición de posibilidad de la soberanía digital: no se posee aquello que no se puede recuperar.

4.6.1. La regla 3-2-1 como mínimo estructural

La heurística 3-2-1 ofrece un punto de partida sobrio: conservar al menos *tres* copias de los datos, almacenarlas en *dos* tipos de soporte distintos y mantener *una* de ellas fuera del emplazamiento físico principal (*off-site*). Su valor no reside en los números exactos, sino en el principio subyacente, a saber, diversificar los modos de fallo. Tres copias protegen frente al deterioro silencioso de un soporte; dos soportes distintos —por ejemplo, un disco de estado sólido interno y un disco mecánico externo, o un servicio de almacenamiento de objetos remoto— evitan que un defecto común a una tecnología comprometa todas las réplicas a la vez; la copia *off-site* cubre los siniestros que afectan a una ubicación completa, como un incendio, una inundación, un robo o un cifrado malicioso que recorra la red local.

Conviene precisar qué *no* es una copia de seguridad. La redundancia de discos en RAID protege frente al fallo de una unidad, pero replica de inmediato un borrado accidental, una corrupción lógica o la acción de un programa de secuestro de datos; no constituye, por tanto, una copia. Análogamente, una instantánea del sistema de archivos residente en el mismo equipo comparte su destino ante un siniestro físico. La regla 3-2-1 obliga a separar las copias en el espacio y en el dominio de fallo, no solo a multiplicarlas.



Figura 4.2: La regla 3-2-1: tres copias de los datos, en dos tipos de soporte distintos, con una de ellas fuera del emplazamiento (*off-site*); la copia remota se cifra en el cliente y la estrategia se valida con pruebas de restauración periódicas.

4.6.2. Cifrado e integridad

En un contexto doméstico, la copia *off-site* suele residir en almacenamiento que el administrador no controla físicamente —un servicio de objetos compatible con S3 o el equipo de un familiar—, lo que exige cifrado del lado del cliente antes de la transmisión. Dos herramientas consolidan este enfoque. *restic* y *BorgBackup* implementan copias deduplicadas, comprimidas y cifradas con autenticación, de modo que el repositorio remoto nunca observa los datos en claro ni puede alterarlos sin que la verificación lo detecte (*restic contributors*, 2024; *BorgBackup contributors*, 2024). La deduplicación por bloques de contenido reduce el volumen transmitido y almacenado: tras la primera copia completa, cada ejecución posterior solo añade los fragmentos nuevos, lo que vuelve asequible conservar un historial profundo de versiones sin un coste de espacio que crezca de forma lineal con el número de copias.

El cifrado autenticado aporta, además de confidencialidad, integridad verificable: la copia detecta tanto la corrupción de bloques como la manipulación deliberada. Esta propiedad encaja con los principios de mínimo privilegio y de defensa en profundidad, pues el soporte remoto se trata como un medio no confiable y la confianza se sitúa en la clave que custodia el administrador (Saltzer & Schroeder, 1975). La contrapartida es operativa y severa: la pérdida de esa clave equivale a la pérdida de todas las copias, ya que sin ella el repositorio cifrado resulta indescifrable. La gestión de la clave de cifrado —su custodia redundante y su separación física del propio repositorio— se convierte así en el punto crítico de toda la estrategia, pues concentra en un único secreto la viabilidad de la recuperación.

4.6.3. Objetivos de recuperación en el ámbito doméstico

Toda copia de seguridad materializa, implícita o explícitamente, dos magnitudes. El *RPO* (*Recovery Point Objective*, punto de recuperación objetivo) expresa cuántos datos recientes se acepta perder, y queda determinado por la frecuencia de las copias: una ejecución diaria fija un RPO máximo de veinticuatro horas. El *RTO* (*Recovery Time Objective*, tiempo de recuperación objetivo) mide cuánto puede tardar el sistema en volver a estar operativo tras un incidente.

En el ámbito doméstico estos objetivos se calibran con realismo, no con la exigencia de un centro de datos. No todos los datos comparten valor: las fotografías familiares o los documentos no reproducibles justifican un RPO corto y varias réplicas, mientras que la configuración de servicios, reconstruible mediante infraestructura declarativa o contenedores versionados, tolera un RPO laxo y un RTO mayor (Docker, 2024). Esta gradación evita el error frecuente de tratar todo el almacenamiento con el mismo celo, lo que encarece y complica la operación sin mejorar la protección de lo verdaderamente irremplazable. Un RTO honesto en un *homelab* contempla, asimismo, el tiempo de adquirir o reparar el soporte físico, restaurar desde la copia *off-site* —limitada por el ancho de banda de subida del proveedor remoto— y reconstruir el entorno de ejecución antes de que el servicio vuelva a estar disponible.

4.6.4. El valor de una copia se mide en la restauración

La sentencia según la cual una copia no probada equivale a la ausencia de copia no es una exageración retórica, sino la constatación de un modo de fallo recurrente: respaldos que se ejecutan durante meses, informan de éxito y resultan ilegibles, incompletos o cifrados con una clave extraviada justo cuando se los necesita. La copia de seguridad solo adquiere valor en el acto de la restauración; hasta entonces es una hipótesis no verificada.

De ello se sigue que la prueba de restauración debe formar parte del procedimiento ordinario, no de la respuesta a la emergencia. *restic* y *BorgBackup* ofrecen comandos de verificación de integridad del repositorio, pero la integridad del archivo no garantiza la utilidad del dato restaurado: conviene recuperar periódicamente un subconjunto a un entorno limpio y comprobar que los ficheros se abren, que una base de datos arranca y que el servicio reconstruido funciona. Esta práctica válida, de paso, que el procedimiento esté documentado y sea ejecutable por quien deba aplicarlo bajo presión, posiblemente meses después de haberlo diseñado. La automatización de la verificación —una restauración programada y contrastada contra un valor de comprobación conocido— traslada el descubrimiento del fallo del momento del desastre al de una jornada ordinaria, que es precisamente el objetivo de toda la disciplina de recuperación.

4.7. Observabilidad: logs, métricas y salud

La operación de un *homelab* —entendido como infraestructura informática autoalojada (*self-hosting*) de escala doméstica o profesional reducida— exige conocer en todo momento qué hacen los servicios desplegados y por qué. La observabilidad designa la capacidad de inferir el estado interno de un sistema a partir de las señales que emite hacia el exterior. Conviene distinguirla de la mera monitorización: la monitorización vigila condiciones previstas de antemano, mientras que la observabilidad aspira a responder preguntas no anticipadas sobre comportamientos emergentes. En un entorno autoalojado, donde el operador es a la vez administrador, desarrollador y único responsable de la disponibilidad, esa distinción resulta determinante, pues ninguna instancia externa acudirá a diagnosticar una degradación silenciosa.

La práctica consolida tres pilares de señal: registros (*logs*), métricas y trazas. Los dos primeros bastan para cubrir la mayor parte de las necesidades de un *homelab*; las trazas distribuidas, propias de arquitecturas de microservicios extensas, rara vez justifican su coste en este contexto y pueden omitirse sin merma apreciable.

4.7.1. Logs centralizados

Cada contenedor, demonio y proxy inverso produce registros, pero dispersos resultan poco útiles: diagnosticar un fallo obligaría a iniciar sesión en cada máquina y correlacionar manualmente marcas temporales. La centralización consiste en agregar todos los flujos en un

único almacén consultable. El motor de contenedización facilita el primer paso, al normalizar la salida estándar de cada proceso como flujo de registro recolectable mediante controladores de *logging* (Docker, 2024). Sobre esa base, las soluciones de agregación que indexan por etiquetas en lugar de por contenido completo reducen el almacenamiento exigido frente a las alternativas que indexan cada término, a costa de un menor poder de búsqueda libre sobre el cuerpo del mensaje.

La correlación temporal entre nodos plantea un problema sutil. Los relojes de máquinas distintas divergen, de modo que ordenar eventos por su marca local puede inducir a error sobre la secuencia causal real. La noción de orden parcial basado en la relación de precedencia (Lamport, 1978) recuerda que, en ausencia de un reloj global perfecto, lo relevante no es el instante absoluto sino la relación «sucedió antes que». En la práctica, la sincronización de relojes por protocolos de red y la inclusión de identificadores de petición que atraviesan los servicios mitigan el problema sin necesidad de relojes lógicos formales.

4.7.2. Métricas

Las métricas son series temporales numéricas que cuantifican el comportamiento del sistema: uso de procesador, memoria residente, latencia de respuesta, tasa de errores o profundidad de colas. En entornos autoalojados predomina el modelo de extracción (*pull*): un servidor central consulta periódicamente puntos de exposición HTTP que cada servicio publica. Este enfoque encaja con la naturaleza efímera de los contenedores, pues el descubrimiento dinámico de objetivos sustituye a la configuración manual de cada origen.

La instrumentación suele organizarse en torno a unos pocos tipos básicos de medida —contadores, medidores e histogramas— que conviene elegir con criterio. Un histograma de latencias, por ejemplo, permite calcular percentiles agregados y revela la cola larga que un simple promedio oculta: un servicio con latencia media de 50 ms puede esconder un percentil 99 de varios segundos que arruine la experiencia en una fracción de las peticiones. Resulta provechoso vertebrar la observación alrededor de un conjunto reducido de indicadores estables —latencia, tráfico, errores y saturación—, magnitudes que describen la salud percibida con relativa independencia de la tecnología subyacente.

La visualización mediante paneles traduce esas series en gráficos legibles. La disciplina recomendable consiste en distinguir los paneles de diagnóstico, densos y reservados a la investigación reactiva, de los paneles de estado general, sobrios y orientados a confirmar de un vistazo que el conjunto opera dentro de parámetros normales.

4.7.3. Comprobaciones de salud y disponibilidad

La salud de un servicio no se deduce solo de que su proceso siga en ejecución. Un proceso vivo puede haber quedado bloqueado, haber agotado su reserva de conexiones o haber caído en un estado inconsistente. Las comprobaciones de salud (*healthchecks*) responden a esta limitación verificando activamente la capacidad de respuesta funcional, no la mera presencia del proceso. El orquestador de contenedores admite definir comprobaciones periódicas

que marcan un contenedor como saludable o no, y que permiten automatizar su reinicio o su exclusión del reparto de carga (Docker, 2024).

Es provechosa la distinción entre dos sondas. La sonda de vivacidad determina si un servicio debe reiniciarse; la sonda de disponibilidad determina si está listo para recibir tráfico. Confundirlas produce patologías conocidas: una sonda de vivacidad demasiado estricta puede reiniciar en cadena un servicio que solo atraviesa una carga transitoria, y agravar así la indisponibilidad que pretendía corregir.

La disponibilidad se mide a lo largo del tiempo como proporción de servicio efectivo. Un comprobador externo de tiempo de actividad —idealmente alojado fuera de la infraestructura vigilada, para no caer junto a ella— interroga periódicamente los puntos públicos y registra los cortes. Aquí emerge una tensión arquitectónica de fondo: ningún sistema garantiza disponibilidad perfecta, y el análisis del compromiso entre consistencia y disponibilidad ante particiones de red (Gilbert & Lynch, 2002) advierte que perseguir cifras de disponibilidad absolutas obliga a sacrificar otras propiedades. Para un *homelab*, fijar objetivos realistas —y aceptar ventanas de mantenimiento explícitas— resulta más honesto que aspirar a una continuidad que la propia escala del proyecto no sostiene.

4.7.4. Alertas

La instrumentación sin alertas degenera en paneles que nadie consulta hasta que el daño ya está hecho. Una alerta traduce una condición sobre métricas o registros en una notificación accionable. El principio rector es que toda alerta debe exigir intervención humana; las que se disparan por situaciones que el sistema resuelve por sí mismo erosionan la atención del operador y conducen a la fatiga de alertas, en la que las señales genuinas se pierden entre el ruido.

Conviene alertar sobre síntomas antes que sobre causas: notificar que la latencia percibida excede un umbral resulta más robusto que vigilar cada posible causa subyacente, pues capta también modos de fallo imprevistos. La temporización importa: introducir una condición de persistencia —que el síntoma se mantenga durante varios minutos— evita disparos por picos transitorios inofensivos.

En conjunto, registros, métricas, comprobaciones de salud y alertas dotan al operador autoalojado de la realimentación necesaria para sostener la soberanía digital que persigue. Sin observabilidad, la independencia respecto de proveedores externos se convierte en ceguera operativa; con ella, el control sobre la propia infraestructura deja de ser nominal para volverse efectivo.

4.8. Reproducibilidad e infraestructura como código

La operación de un *homelab* (laboratorio doméstico de cómputo) madura no se mide por la cantidad de servicios que alberga, sino por la facilidad con que pueden reconstruirse. Un

sistema cuya configuración reside únicamente en la memoria del operador y en ajustes aplicados a mano resulta frágil: cada cambio no documentado erosiona el conocimiento del propio estado, y un fallo de disco o un error de actualización pueden convertirse en una pérdida irreversible. La infraestructura como código —*Infrastructure as Code*— responde a esta fragilidad expresando la configuración del sistema en ficheros versionables, de modo que el estado deseado quede descrito de forma explícita, auditable y repetible. El principio es continuidad directa de la unidad de despliegue declarativa examinada antes: si la topología de servicios se especifica en un fichero, conviene que también lo hagan la provisión del anfitrión, sus dependencias y sus parámetros.

4.8.1. Configuración declarativa frente a procedimental

Cabe distinguir dos enfoques para describir la infraestructura. El procedimental enumera los pasos que conducen al estado deseado: instalar tal paquete, editar tal fichero, reiniciar tal servicio. El declarativo describe el estado final y delega en una herramienta la tarea de alcanzarlo desde cualquier punto de partida. La diferencia es sustantiva. Una secuencia de órdenes ejecutada dos veces puede producir resultados distintos según el estado previo; una descripción declarativa aspira a la *idempotencia*, propiedad por la cual aplicar la configuración repetidamente converge siempre al mismo resultado sin efectos colaterales. Esta convergencia es lo que vuelve la reproducibilidad fiable: el operador no necesita conocer la historia del sistema, solo el estado que desea, y la herramienta calcula la diferencia. En el dominio doméstico, donde no existe un equipo que custodie el conocimiento operativo, esta propiedad sustituye a la documentación verbal y a la memoria individual por una especificación ejecutable.

4.8.2. Ansible y Compose bajo control de versiones

Dos instrumentos cubren la mayoría de las necesidades de un laboratorio doméstico. El fichero *Compose* describe la capa de servicios contenedorizados, cuyo aislamiento ligero respecto del anfitrión hace que su despliegue sea consistente entre máquinas (Merkel, 2014; Docker, 2024); Ansible describe la capa subyacente del anfitrión: paquetes del sistema, usuarios, reglas del cortafuegos, montajes de almacenamiento, certificados y tareas programadas. Ansible opera sin agente, comunicándose por SSH, y organiza la configuración en *playbooks* y roles cuyas tareas se diseñan para ser idempotentes. La combinación de ambos cubre el espectro completo: desde la instalación del propio motor de contenedores hasta el despliegue de las aplicaciones que se ejecutan sobre él.

El valor decisivo aparece cuando estos ficheros se custodian bajo control de versiones. Un repositorio Git que reúna *playbooks*, ficheros *Compose*, plantillas de configuración y variables convierte la historia de la infraestructura en un registro inmutable y atribuible: cada modificación queda fechada, justificada en su mensaje de confirmación y reversible. La revisión previa a la integración, aun cuando el operador sea único, impone una disciplina que previene cambios impulsivos. Procede aquí una salvedad de seguridad que no debe omitirse:

las credenciales, claves y secretos jamás se almacenan en claro en el repositorio. Se externalizan mediante mecanismos de cifrado en reposo —como las bóvedas que el propio Ansible incorpora— o se inyectan en tiempo de ejecución, de manera que el repositorio describa la estructura sin exponer los secretos que la habilitan. Esta separación materializa el principio de privilegio mínimo, según el cual cada componente debe disponer solo de la autoridad estrictamente necesaria para su función (Saltzer & Schroeder, 1975): la descripción de la infraestructura no necesita contener los secretos para gobernarla.

4.8.3. GitOps doméstico

De la custodia en Git surge un patrón operativo coherente: el repositorio se erige en fuente única de verdad, y el estado real del sistema se hace converger hacia el estado declarado en él. Esta práctica, conocida como *GitOps*, invierte el flujo habitual de administración. En lugar de modificar el servidor y documentar después —si acaso— lo realizado, el operador modifica primero la descripción versionada y deja que un proceso aplique el cambio. La consecuencia es que toda deriva entre lo declarado y lo desplegado se vuelve detectable y corregible, y que la confianza ya no se deposita en el estado opaco de una máquina, sino en una representación legible y comparable. En el laboratorio doméstico, *GitOps* no exige las plataformas de reconciliación continua propias de los grandes despliegues; basta con la disciplina de no tocar el anfitrión salvo a través del repositorio, complementada con una orden que aplique los *playbooks* y actualice los servicios. La forma es modesta, pero el principio —reconciliar la realidad con una especificación declarada— se conserva intacto.

4.8.4. Recuperación ante desastres y documentación como código

La prueba última de la reproducibilidad es la recuperación ante un desastre. Reconstruir un *homelab* desde cero descompone el sistema en dos planos independientes que conviene no confundir. El primero es la configuración, descrita en el repositorio: clonarlo y aplicar los *playbooks* regenera el anfitrión y redespiega los servicios sin intervención manual. El segundo es el estado, que vive en los volúmenes de datos y no puede derivarse de ninguna descripción; su preservación depende exclusivamente de las copias de seguridad cifradas y verificadas —apoyadas en herramientas de archivado deduplicado y autenticado (BorgBackup contributors, 2024; restic contributors, 2024)— que se tratan en otra sección. La distinción es operativamente crítica: la infraestructura como código reproduce la *forma* del sistema, no su *contenido*. Una recuperación completa exige, por tanto, restaurar la configuración desde Git y los datos desde las copias, y ambos mecanismos han de ensayarse, pues una recuperación nunca probada es una hipótesis, no una garantía.

Este modelo realiza, por último, el ideal de la documentación como código. La descripción ejecutable no puede divergir de la realidad sin que el sistema deje de funcionar, lo que la mantiene veraz por construcción, a diferencia de la documentación en prosa, que envejece en silencio en cuanto se aplica un cambio fuera de ella. No se trata de prescindir de toda narrativa: las decisiones de diseño, las razones de una elección y los procedimientos de emergencia

siguen reclamando explicación en lenguaje natural. Pero el *qué* —qué se instala, con qué parámetros, en qué orden— deja de habitar un documento aparte y pasa a residir en los ficheros que efectivamente gobiernan el sistema. La reproducibilidad, así entendida, no es una técnica accesoria, sino la condición que sostiene la soberanía digital sobre la infraestructura propia: solo se posee de verdad aquello que se sabe reconstruir.

4.9. Migración: entrar y salir de la nube y entre soluciones

La soberanía digital no se conquista en un acto fundacional, sino que se ejerce de forma continuada, y uno de sus momentos más reveladores es la migración. Mover datos y servicios —desde la nube comercial hacia un *homelab* (laboratorio doméstico de servidores autoalojados), desde el *homelab* de vuelta a un proveedor o entre soluciones de autoalojamiento distintas— pone a prueba si la información estaba realmente bajo control o solo en custodia ajena. Cabe distinguir, por ello, entre poseer un dato y poder llevárselo: la facilidad de salida es la métrica honesta de la soberanía, pues quien no puede marcharse sin pérdidas no controla del todo su información, con independencia de dónde resida. Conviene examinar, en consecuencia, las tres direcciones del movimiento, la disciplina que las hace fiables y el riesgo de cautiverio —*lock-in*— que el diseño con formatos abiertos previene.

4.9.1. Salir de la nube: exportación, portabilidad y sus límites

La migración desde la nube comienza por extraer los datos, y aquí el marco regulatorio europeo ofrece una palanca concreta. El RGPD reconoce un derecho a la portabilidad de los datos: el interesado puede recibir los datos personales que ha proporcionado en un formato estructurado, de uso común y lectura mecánica, y transmitirlos a otro responsable (European Parliament & Council, 2016). Este derecho, sin embargo, tiene un alcance acotado que conviene no idealizar. Cubre los datos proporcionados por el usuario, no necesariamente los inferidos o derivados por el proveedor; se circunscribe a los datos personales, no al servicio que los rodea; y la calidad del formato exportado queda en buena medida a discreción de quien lo emite. Los mecanismos voluntarios de los proveedores —servicios de exportación masiva o interfaces de programación (*API*) que permiten una descarga incremental y selectiva— complementan la vía legal, pero comparten sus límites: un archivo comprimido con fotografías y metadatos heterogéneos, o un volcado de correo en un formato que ningún otro servicio reimporta con fidelidad, satisfacen la letra de la portabilidad sin restituir la operatividad del dato.

De ahí que la salida de la nube exija una traducción, no una mera copia. El material exportado suele llegar en estructuras pensadas para el archivado, no para la reanudación del servicio: contactos en una variante de un estándar, calendarios con campos propietarios, historiales de mensajería sin equivalente directo. La migración real consiste en mapear ese material al modelo de datos de la solución de autoalojamiento de destino —archivos, contactos y calendarios sobre una plataforma como Nextcloud, por ejemplo— y en aceptar que ciertas relaciones y metadatos se perderán por el camino (Nextcloud GmbH, 2024). Reconocer esa

pérdida de antemano, y decidir qué es tolerable, forma parte de la planificación tanto como el calendario de corte.

4.9.2. Migrar entre soluciones de autoalojamiento

El movimiento entre soluciones de autoalojamiento presenta una asimetría favorable: el administrador controla origen y destino, dispone de acceso directo a las bases de datos y a los volúmenes, y no depende de la voluntad de un proveedor para exportar. Esa ventaja no elimina la fricción, sino que la desplaza hacia la heterogeneidad de los esquemas internos. Migrar de una aplicación de notas a otra, o de un gestor de fotografías a su sucesor, obliga a reconciliar modelos de datos que rara vez coinciden campo a campo. Aquí el valor de los formatos abiertos y de los estándares —archivos en sistemas de ficheros legibles en lugar de bases de datos opacas, esquemas documentados, protocolos de interoperabilidad— se vuelve tangible: cuanto más cerca esté el almacenamiento de representaciones neutrales y portables, menor será el coste de cambiar de implementación. El principio *local-first*, que sitúa los datos primero en el dominio del usuario, presupone justamente esta capacidad de moverlos sin intermediación (Kleppmann et al., 2019).

Esta misma exigencia adquiere un matiz adicional cuando lo que se migra incluye una capa de IA agéntica local. Un asistente que opera sobre la información del usuario añade artefactos propios —índices vectoriales, memorias persistentes, historiales de conversación, configuraciones de herramientas— que no siempre tienen un formato de intercambio establecido. Conviene tratar esos artefactos como datos derivados y regenerables siempre que sea posible: si los índices y embeddings pueden reconstruirse a partir de las fuentes originales en el destino, la migración se reduce a mover esas fuentes, y la agencia agéntica se reconstituye sin arrastrar dependencias opacas. Diseñar el componente de IA de modo que su estado valioso resida en formatos legibles, y lo accesorio sea recomputable, extiende al plano agéntico la misma disciplina de portabilidad que rige para el resto del *homelab*.

4.9.3. Planificación, ventana de corte y validación

Toda migración no trivial se ejecuta como un proyecto con fases, no como una operación atómica. Conviene, en primer lugar, inventariar lo que se mueve y sus dependencias, y establecer un destino verificado antes de tocar el origen. La técnica más segura es la migración en paralelo: poblar el sistema de destino con una copia de los datos mientras el origen permanece operativo, validar exhaustivamente sobre esa réplica y reservar la *ventana de corte* —el intervalo de indisponibilidad durante el cual se transfiere el incremento final y se redirige el uso— para el último paso, ya con la confianza de que el destino funciona. Este enfoque acota el tiempo de interrupción y, sobre todo, preserva una vía de retorno: mientras el origen no se desmantela, un fallo en el destino no es irreversible.

La validación distingue una migración terminada de una meramente iniciada. No basta con que el proceso de importación informe de éxito; procede comprobar que los datos restituidos son correctos y utilizables. Esa verificación opera en dos planos. En el plano de la integridad

técnica, los recuentos de objetos, las sumas de comprobación de los ficheros y el contraste de tamaños entre origen y destino detectan truncamientos, omisiones y corrupciones silenciosas; las herramientas de copia de seguridad con cifrado autenticado ofrecen, en su propio dominio, una garantía análoga de que el dato no se ha alterado en tránsito (restic contributors, 2024). En el plano funcional, conviene abrir una muestra representativa —que un documento se renderice, que un evento de calendario conserve su zona horaria, que un hilo de mensajería mantenga su orden y sus adjuntos— porque la integridad de los bytes no garantiza la fidelidad semántica. Solo cuando ambos planos se satisfacen tiene sentido cruzar la ventana de corte y, transcurrido un periodo prudencial de observación, retirar el origen.

4.9.4. El lock-in como riesgo operativo y el diseño que lo previene

El cautiverio tecnológico —*lock-in*— no es un inconveniente puntual de la migración, sino un riesgo operativo permanente que la migración se limita a revelar. Un servicio que almacena los datos en formatos propietarios, que no ofrece exportación completa o que entrelaza la información con metadatos irreproducibles eleva el coste de salida hasta volverlo prohibitivo; llegado ese punto, la decisión de permanecer deja de ser libre. La nube comercial obtiene parte de su poder de esta fricción asimétrica, que facilita la entrada y encarece la salida (Zuboff, 2019). El paradigma de autoalojamiento no es inmune por defecto: una solución autoalojada con un esquema opaco reproduce el mismo cautiverio dentro del propio servidor, y un componente de IA atado a un proveedor de modelos concreto traslada esa misma dependencia al corazón de la agencia agéntica.

La prevención, por tanto, es de diseño y antecede a cualquier migración. Privilegiar formatos abiertos y documentados, almacenar siempre que sea posible sobre sistemas de ficheros legibles antes que en bases de datos cerradas, exigir a cada servicio una ruta de exportación verificable y ensayar periódicamente esa exportación —del mismo modo que se ensaya la restauración de una copia— convierten la portabilidad en una propiedad comprobada y no en una promesa. Una migración fluida no es, en definitiva, el fruto de una herramienta afortunada, sino la consecuencia tardía de decisiones arquitectónicas tomadas mucho antes de necesitarla.

5 Seguridad y amenazas

La soberanía digital que persigue el autoalojamiento (*self-hosting*) traslada al operador del *homelab* una responsabilidad que, en los servicios gestionados por terceros, permanece delegada y a menudo invisible: la de proteger su propia infraestructura. Quien decide custodiar sus datos y ejecutar sus servicios en hardware propio asume también el papel de defensor; la confidencialidad, la integridad y la disponibilidad del sistema dejan de estar garantizadas por contrato y pasan a depender de decisiones técnicas concretas. Se sostiene, en consecuencia, que la seguridad no constituye un añadido posterior al diseño del *homelab*, sino una condición de posibilidad de la propia soberanía: sin defensa razonable, el control sobre los datos es nominal.

Este capítulo aborda la seguridad como una disciplina sistemática y no como un conjunto de recetas aisladas. Se parte de la elaboración de un modelo de amenazas que acote qué se protege y frente a quién, y a partir de ese marco se examinan la superficie de ataque y la exposición a Internet, las técnicas de endurecimiento basadas en el mínimo privilegio y el aislamiento, la gestión de secretos y credenciales, los esquemas de identidad y autenticación orientados a la confianza nula (*zero-trust*), la integridad de la cadena de suministro y los parches, y la continuidad frente a incidentes. El hilo que recorre estas secciones es el del riesgo proporcionado: cabe distinguir entre la postura defensiva de una organización y la de un operador individual, y conviene calibrar cada medida según el valor de lo protegido y la verosimilitud de la amenaza.

La incorporación de inteligencia agéntica local introduce, además, una clase de riesgo que los servicios autoalojados tradicionales no contemplaban. Un agente dotado de agencia agéntica ejecuta acciones, accede a herramientas y procesa entradas no confiables, lo que abre vectores específicos como la inyección de instrucciones (*prompt injection*), la exfiltración de información sensible y el abuso de la capacidad de actuación del propio agente. Se dedica por ello una sección al tratamiento de estas amenazas, que se analizan con el mismo rigor que el resto de la superficie del sistema, sin subordinar a ellas el conjunto: la IA local es un componente más de un *homelab* cuya seguridad debe pensarse de forma integral.

5.1. Modelo de amenazas del *homelab*

Toda decisión de seguridad presupone una respuesta, explícita o tácita, a tres preguntas: qué se protege, de quién y a qué coste. Omitir esa respuesta conduce a uno de dos extremos estériles, igualmente comunes en el *homelab* (laboratorio doméstico de cómputo): la indefensión confiada, que asume que un servidor casero carece de interés para cualquier adversario, y la

paranoia desproporcionada, que despliega controles de grado corporativo sobre activos que no los justifican. Un modelo de amenazas es el ejercicio sistemático que evita ambos: enumera adversarios verosímiles, identifica los activos que importan, traza las vías por las que unos alcanzan a otros y gradúa la respuesta según el riesgo. Conviene subrayar que el resultado no es un catálogo de productos de seguridad, sino un marco de razonamiento que ordena las decisiones técnicas de los capítulos siguientes. Marcos institucionales como el de mejora de la ciberseguridad de infraestructuras críticas (NIST, 2018) articulan esta lógica en cinco funciones —identificar, proteger, detectar, responder y recuperar—, de las que el modelado de amenazas constituye la primera y condiciona a las restantes.

5.1.1. Adversarios realistas

La pregunta «quién atacaría un servidor doméstico» admite respuestas concretas, y ninguna de ellas exige postular a un atacante dirigido con recursos estatales. El adversario predominante es impersonal: el escaneo masivo automatizado. Redes de equipos comprometidos recorren de continuo el espacio de direcciones en busca de puertos abiertos, paneles de administración expuestos, credenciales por defecto y versiones de software con vulnerabilidades conocidas. Este adversario no selecciona víctimas; explota a quien encuentra. Su corolario operativo es nítido: cualquier servicio accesible desde internet será sondeado en cuestión de horas, con independencia de su notoriedad o de la creencia del operador en su anonimato.

El segundo adversario es el *ransomware* (programa de secuestro), variante del anterior que, una vez obtenido el acceso, cifra los datos y exige rescate. Su peligrosidad en el *homelab* es singular porque el cifrado se propaga por los recursos compartidos accesibles, de modo que un respaldo conectado de forma permanente al sistema que pretende proteger queda cifrado junto al original. Esta amenaza, más que ninguna otra, justifica la disciplina de copias de seguridad fuera de línea o de solo escritura que se examina en su capítulo.

El tercer adversario es la amenaza interna: no necesariamente maliciosa, sino con frecuencia legítima y descuidada. Un familiar que reutiliza una contraseña filtrada, un invitado que conecta un equipo infectado a la red de confianza o el propio operador que deja un servicio de pruebas expuesto encarnan un vector que ningún cortafuegos perimetral intercepta, porque opera desde dentro. Su tratamiento remite a la compartimentación interna y al principio de privilegio mínimo (Saltzer & Schroeder, 1975), no a la defensa de la frontera.

El cuarto adversario no es un sujeto sino un activo subvertido: el dispositivo del *Internet de las cosas* (IoT) comprometido. Cámaras, enchufes y altavoces inteligentes rara vez reciben actualizaciones, comunican con servidores externos y exhiben superficies de ataque opacas. Comprometido uno de ellos, deviene cabeza de puente desde la propia red interna, eludiendo el perímetro por completo. Esta figura justifica que la red interna no se trate como un espacio uniformemente confiable, conforme a la lógica de la arquitectura de confianza cero (Rose et al., 2020), donde la mera pertenencia al segmento no confiere legitimidad.

5.1.2. Activos a proteger

El inventario de activos precede al de amenazas porque define qué pérdida importa. En un *homelab* cabe distinguir, al menos, cuatro categorías. Los **datos personales** —fotografías, documentos, correspondencia, notas— constituyen el activo de mayor valor y menor reemplazabilidad: su pérdida es irreversible y su filtración, irreparable; sobre ellos recae, además, la responsabilidad jurídica cuando se tratan datos de terceros, en los términos del reglamento europeo de protección de datos (European Parliament & Council, 2016). Los **secretos y credenciales** —claves privadas, *tokens* de acceso, contraseñas de servicio— son activos cuyo compromiso no daña por sí mismo, sino por habilitar el acceso a los anteriores. La **disponibilidad de los servicios** importa de modo desigual: una interrupción de la biblioteca multimedia es una molestia, mientras que la de la domótica que gobierna calefacción o cerraduras puede tener consecuencias físicas. Por último, la **propia capacidad de cómputo y el ancho de banda** son activos en sí mismos, codiciados para minar criptomonedas, retransmitir abusos o integrarse en una red de equipos comprometidos; su sustracción puede pasar inadvertida durante meses.

Esta jerarquía importa porque orienta el gasto defensivo. No todos los activos merecen igual protección, y confundir su valor conduce a fortificar lo accesorio mientras lo esencial queda expuesto.

5.1.3. STRIDE adaptado al *homelab*

STRIDE —acrónimo de suplantación, manipulación, repudio, revelación, denegación de servicio y elevación de privilegios— ofrece una taxonomía de amenazas por categorías que conviene reinterpretar a la escala doméstica, sin trasladar mecánicamente su origen corporativo. Sus seis clases se leen así:

- *Spoofing* (suplantación): un dispositivo o usuario se hace pasar por otro; se contiene con autenticación fuerte y verificación de identidad de los flujos, no de la red de origen.
- *Tampering* (manipulación): alteración no autorizada de datos en tránsito o en reposo; se mitiga con cifrado autenticado y con integridad verificable de los respaldos.
- *Repudiation* (repudio): imposibilidad de atribuir una acción; en el *homelab* se traduce, sobre todo, en la necesidad de un registro de auditoría que permita reconstruir incidentes.
- *Information disclosure* (revelación de información): exposición de datos confidenciales, el riesgo de mayor impacto sobre los activos prioritarios.
- *Denial of service* (denegación de servicio): merma de disponibilidad, relevante de forma desigual según el servicio afectado.
- *Elevation of privilege* (elevación de privilegios): obtención de capacidades no concedidas, típicamente la escalada desde un servicio comprometido hacia el anfitrión o hacia otros contenedores.

La adaptación pertinente consiste en ponderar cada clase por la realidad doméstica. La revelación de información y la elevación de privilegios concentran el riesgo serio, pues atacan los

activos irremplazables; el repudio, central en entornos con obligaciones de cumplimiento, pesa menos en una infraestructura sin terceros que rendir cuentas. Aplicar STRIDE servicio a servicio —preguntando, para cada uno, por dónde cabe cada categoría— sustituye la intuición difusa por un examen reproducible.

5.1.4. Riesgo proporcional

El cierre del modelo es la proporcionalidad. El riesgo se aprehende como el producto de la probabilidad de un suceso por su impacto, y la respuesta razonable se gradúa según ese producto, no según el catálogo de lo técnicamente posible. El escaneo masivo es casi seguro pero de impacto acotado si no halla servicios vulnerables; el atacante dirigido es improbable en el hogar pero de impacto severo: el primero justifica controles sistemáticos y baratos, el segundo rara vez justifica el coste de defenderse de él. Esta ponderación enlaza con la economía del autoalojamiento tratada en otro capítulo: cada control consume tiempo de operación y mantenimiento, recurso escaso que, mal asignado, degrada la seguridad global al producir fatiga y configuraciones abandonadas. Un modelo de amenazas maduro no es, pues, el que enumera más peligros, sino el que descarta de forma argumentada aquellos cuya defensa no compensa, y concentra el esfuerzo donde la pérdida sería real e irreversible.

5.2. Superficie de ataque y exposición a Internet

La superficie de ataque de una instalación de *self-hosting* (autoalojamiento) comprende el conjunto de puntos por los que un agente externo puede interactuar con los servicios alojados y, eventualmente, comprometerlos. En el contexto doméstico o de pequeña organización, esta superficie crece de forma silenciosa: cada servicio que se publica, cada panel administrativo que se habilita y cada puerto que se redirige en el enrutador amplían el perímetro defendible. Conviene, por tanto, delimitar con precisión qué se expone, por qué se expone y qué alternativas existen para reducir la exposición sin renunciar a la funcionalidad.

5.2.1. Qué se expone y por qué

La motivación habitual para exponer un servicio a Internet es el acceso remoto. Una nube de archivos, un panel de domótica o un servicio de medios resultan útiles precisamente porque se consultan desde fuera de la red local. Esa utilidad tiene un coste: el servicio debe ser alcanzable desde direcciones arbitrarias, lo que lo convierte en blanco de exploración automatizada permanente. La exposición se materializa, en la práctica, a través de tres elementos encadenados. Primero, la apertura de puertos en el enrutador mediante reenvío (*port forwarding*), que traduce una dirección y un puerto públicos hacia un equipo interno. Segundo, los servicios que escuchan en esos puertos, típicamente HTTP y HTTPS (puertos 80 y 443), pero también acceso remoto por SSH (22), bases de datos o protocolos de gestión. Tercero, los paneles de administración: interfaces web de configuración del propio enrutador, de los

contenedores, de los hipervisores o de las aplicaciones, que concentran privilegios elevados y constituyen objetivos de alto valor.

El error recurrente no es exponer un servicio bien fortificado, sino exponer más de lo necesario por comodidad o por desconocimiento. Una base de datos accesible desde Internet, un panel de gestión de contenedores sin autenticación robusta o un servicio de escritorio remoto con credenciales por defecto representan exposiciones cuyo beneficio funcional es nulo o marginal y cuyo riesgo es elevado. La asimetría es estructural: el defensor debe proteger todos los puntos de entrada de forma continua, mientras que al atacante le basta con encontrar uno vulnerable en un único instante.

5.2.2. La realidad de la exploración automatizada

La idea de que una instalación doméstica pasa desapercibida por su tamaño es una falacia. El espacio de direcciones IPv4 enrutables se explora de manera exhaustiva y constante mediante motores que catalogan los servicios expuestos. Existen buscadores especializados que registran de forma sistemática qué puertos responden en cada dirección, qué software los atiende, qué versión declaran y qué parámetros de configuración revelan. El resultado es un índice de servicios expuestos que cualquiera puede consultar para localizar, por ejemplo, cámaras con firmware vulnerable, paneles administrativos accesibles o servicios con versiones afectadas por fallos conocidos.

La consecuencia operativa es que un servicio recién publicado recibe sondeos en cuestión de minutos, no de días. Los registros de cualquier servidor expuesto muestran intentos continuos de acceso a rutas comunes, pruebas de credenciales por defecto y exploración de versiones. No se trata de un ataque dirigido, sino de un barrido indiscriminado que se vuelve dirigido en cuanto detecta una vulnerabilidad explotable. Asumir esta presión de fondo como condición permanente constituye el punto de partida realista de cualquier diseño defensivo.

5.2.3. El principio de no exponer

El control más eficaz sobre la superficie de ataque es no crearla. Antes de fortificar un servicio expuesto conviene preguntarse si necesita estarlo. Este criterio enlaza con los principios de privilegio mínimo y de economía del mecanismo formulados por Saltzer y Schroeder (1975): todo componente debe operar con la menor exposición y los menores privilegios compatibles con su función, y el diseño debe minimizar el número de mecanismos de los que depende la seguridad. Aplicado a la red, se traduce en una regla de denegación por defecto: nada es alcanzable desde el exterior salvo lo que se justifique explícitamente.

La alternativa preferente a la exposición directa es el acceso a través de un túnel cifrado. Una red privada virtual basada en WireGuard (Donenfeld, 2017) permite alcanzar los servicios internos como si se estuviera en la red local, sin abrirlos a todo Internet; el único punto expuesto pasa a ser el extremo del túnel, con una superficie reducida y un protocolo diseñado con criterios de minimalismo criptográfico. Esta aproximación se alinea con el modelo de

confianza cero (Rose et al., 2020), en el que la pertenencia a la red no concede confianza por sí misma y cada acceso se autentica y autoriza de forma individual, en contraposición al modelo perimetral clásico que asumía seguro todo lo interno.

Cuando un servicio sí debe publicarse —porque su valor reside en el acceso sin cliente previo, como un servicio web de uso compartido—, la reducción de superficie se logra concentrando la exposición. Un único proxy inverso atiende el puerto 443 y, en su caso, el 80, termina TLS conforme a las versiones vigentes del protocolo (Rescorla, 2018) y enruta hacia los servicios internos, que permanecen sin acceso directo desde el exterior. Así, en lugar de múltiples servicios expuestos con configuraciones heterogéneas, existe un solo punto de entrada endurecido, auditado y mantenido, lo que simplifica la vigilancia y el parcheo.

5.2.4. Segmentación y DMZ doméstica

La reducción de superficie no se agota en limitar lo expuesto: exige también contener lo expuesto. El concepto de zona desmilitarizada (*DMZ*), trasladado al entorno doméstico, consiste en aislar los equipos o servicios accesibles desde Internet en un segmento de red separado, de modo que un compromiso de la zona expuesta no otorgue acceso lateral al resto de la infraestructura. Una red doméstica plana, en la que el servidor expuesto comparte segmento con ordenadores personales, dispositivos del internet de las cosas y copias de seguridad, convierte cualquier intrusión en un riesgo para todo el conjunto.

La segmentación mediante redes locales virtuales (VLAN) o subredes separadas permite establecer fronteras internas: el servicio expuesto reside en un segmento cuyo tráfico hacia la red de confianza está restringido por reglas de cortafuegos explícitas. Numerosos dispositivos del internet de las cosas, con firmware opaco y rara vez actualizado, justifican por sí solos esta separación, pues constituyen una de las vías de compromiso más frecuentes. La aplicación coherente de la denegación por defecto, tanto en el perímetro como entre segmentos internos, materializa una defensa en profundidad en la que cada frontera limita el alcance de un fallo en la anterior y reduce el coste de un incidente aislado.

5.3. Endurecimiento: mínimo privilegio y aislamiento

El endurecimiento de una infraestructura autoalojada (*self-hosting*) parte de un principio rector formulado hace medio siglo: el de mínimo privilegio. Según este principio, todo componente —proceso, usuario, contenedor— debe operar con el conjunto más reducido de permisos que su función legítima requiere, y nada más (Saltzer & Schroeder, 1975). Su valor no reside en prevenir el fallo inicial, sino en acotar el daño una vez que el fallo se produce: si un servicio comprometido apenas dispone de capacidades, la superficie que el atacante hereda es marginal. En un *homelab*, donde conviven múltiples servicios sobre un mismo anfitrión, este principio adquiere un peso desproporcionado respecto a su sencillez conceptual, pues sustituye la confianza implícita por una concesión explícita y verificable de derechos.

5.3.1. Usuarios no privilegiados en contenedores

La contenedorización ofrece aislamiento de espacios de nombres, pero no constituye por sí sola una frontera de seguridad equivalente a la de una máquina virtual (Merkel, 2014). Un error frecuente consiste en ejecutar el proceso interno como *root*, identidad que por defecto se corresponde con el *root* del anfitrión salvo que se interpongan mecanismos adicionales. Una fuga del contenedor en esas condiciones entrega privilegios totales sobre el sistema. La mitigación primaria es definir un usuario no privilegiado en la imagen, mediante la directiva correspondiente del fichero de construcción, de modo que el servicio arranque sin identidad administrativa.

A esta medida se suman dos refuerzos. El primero es la activación de *user namespaces*, que reasignan el identificador de usuario del contenedor a un rango no privilegiado del anfitrión: así, incluso el *root* interno se proyecta como un usuario ordinario externo (Docker, 2024). El segundo es la marca *no-new-privileges*, que impide que un proceso eleve sus permisos a través de binarios con el bit *setuid*, cerrando una vía clásica de escalada. La combinación de identidad no privilegiada, remapeo de espacios de nombres y prohibición de escalada produce una defensa en profundidad que ningún elemento aislado proporciona.

5.3.2. Capacidades y restricción de privilegios

El modelo binario tradicional —*root* todopoderoso frente a usuario sin derechos— resulta demasiado grueso. El núcleo Linux descompone los privilegios del superusuario en *capabilities* discretas, cada una asociada a una operación concreta: abrir puertos por debajo de 1024, modificar la hora del sistema, alterar parámetros de red o cargar módulos del núcleo, entre otras. Los entornos de contenedores conceden por defecto un subconjunto de estas capacidades, todavía generoso para la mayoría de cargas (Docker, 2024).

El endurecimiento riguroso invierte la lógica: se retiran todas las capacidades y se reincorporan únicamente las imprescindibles. Un servidor web que deba escuchar en el puerto 80 necesita *NET_BIND_SERVICE*, pero no la capacidad de manipular la tabla de rutas ni de montar sistemas de ficheros. Conceder solo lo necesario reduce el repertorio de acciones que un proceso comprometido puede ejecutar y materializa el mínimo privilegio en una granularidad que el modelo binario nunca permitió. Conviene además rechazar la opción de contenedor privilegiado, que desactiva en bloque estas protecciones y debe reservarse para casos excepcionales plenamente justificados.

5.3.3. Seccomp y AppArmor: confinamiento del comportamiento

Las capacidades acotan *qué* privilegios posee un proceso; *seccomp* y AppArmor acotan *qué* hace. *Seccomp* (*secure computing mode*) filtra las llamadas al sistema que un proceso puede emitir, restringiendo la interfaz entre la aplicación y el núcleo. La superficie de ataque de un sistema operativo se concentra en buena medida en esa interfaz, de suerte que recortar las llamadas accesibles reduce directamente la exposición. El perfil *seccomp* por defecto de

los principales entornos de contenedores bloquea ya un amplio conjunto de llamadas potencialmente peligrosas; un perfil a medida, ajustado al subconjunto exacto que el servicio invoca, estrecha aún más el cerco, aunque exige conocer con precisión el comportamiento de la aplicación y arriesga interrumpir funciones legítimas si se calibra mal.

AppArmor —y, en distribuciones alternativas, SELinux— opera en otro plano: define perfiles de control de acceso obligatorio que delimitan a qué ficheros, rutas y recursos puede acceder un proceso, con independencia de los permisos discrecionales del sistema de ficheros. Mientras que el modelo discrecional confía en que el propietario configure correctamente cada permiso, el control obligatorio impone una política centralizada que el proceso no puede eludir aunque actúe como *root*. Ambos mecanismos encarnan la idea de que la autoridad debe concederse de forma explícita y mediada, un planteamiento afín a la arquitectura de confianza cero, donde ninguna identidad ni ubicación goza de confianza por defecto y todo acceso se evalúa de manera continua (Rose et al., 2020).

5.3.4. Aislamiento entre servicios

Un anfitrión que aloja decenas de servicios constituye, sin segmentación, un dominio de fallo único: el compromiso de uno facilita el movimiento lateral hacia los demás. El aislamiento busca contener cada servicio dentro de su propio perímetro. En el plano de red, ello se traduce en segmentar mediante redes virtuales independientes, de manera que un contenedor de base de datos no sea alcanzable salvo por los servicios que legítimamente la consumen, y nunca desde el exterior. La práctica de exponer puertos solo cuando es estrictamente necesario, y vincularlos a interfaces internas en lugar de a todas las direcciones del anfitrión, evita publicaciones inadvertidas.

En el plano del almacenamiento, conviene montar como solo lectura los sistemas de ficheros que el servicio no necesita modificar, y restringir los volúmenes compartidos a las rutas estrictamente requeridas. Cada montaje de un directorio del anfitrión dentro de un contenedor abre una vía potencial de fuga, por lo que su número debe minimizarse. Estas medidas trasladan al cómputo el argumento extremo a extremo: las garantías de seguridad deben situarse en los puntos donde residen la función y los datos, no delegarse a una capa intermedia genérica (Saltzer et al., 1984).

5.3.5. Superficie mínima

El endurecimiento culmina en la reducción sistemática de la superficie de ataque. Todo elemento presente en el sistema —un paquete instalado, un puerto abierto, una cuenta activa, una dependencia heredada— es un punto que el defensor debe vigilar y el atacante puede explotar. Las imágenes mínimas, que prescinden de intérpretes de órdenes y utilidades auxiliares, eliminan herramientas que un intruso aprovecharía para reconocer el entorno o desplazarse por él. La supresión de servicios y demonios no utilizados, el cierre de puertos sin función y la eliminación de credenciales por defecto persiguen el mismo fin.

Esta lógica concuerda con las recomendaciones de los marcos de control, que prescriben la mínima funcionalidad y la deshabilitación de componentes innecesarios como salvaguarda básica (NIST, 2013). La superficie mínima no es una medida puntual, sino una disciplina sostenida: cada nueva incorporación al sistema debe justificarse frente al privilegio y la exposición que añade. Así, mínimo privilegio, aislamiento y superficie mínima convergen en una misma estrategia de defensa en profundidad, en la que la contención del daño se antepone a la confianza en que la primera barrera nunca caiga.

5.4. Gestión de secretos y credenciales

Todo despliegue autoalojado (*self-hosting*) descansa sobre un sustrato de secretos: contraseñas de bases de datos, claves de cifrado, tokens de API, certificados privados y credenciales de acceso a almacenamiento remoto. La forma en que esos secretos se generan, distribuyen, almacenan y retiran determina, en buena medida, la superficie de ataque real del sistema. Conviene tratar su gestión como una disciplina propia y no como un apéndice de la configuración, porque un error en este plano anula con frecuencia las defensas erigidas en otros. La distribución de los secretos responde, además, a un principio clásico: cada componente recibe únicamente las credenciales que necesita para su función, ni más ni menos, conforme al *privilegio mínimo* enunciado por (Saltzer & Schroeder, 1975).

5.4.1. El fichero `.env` monolítico como antipatrón

La práctica más extendida en el *homelab* consiste en concentrar toda la configuración sensible en un único fichero `.env` que un orquestador de contenedores carga e inyecta en los servicios. Su comodidad es innegable, pero deviene antipatrón en cuanto el despliegue crece. El primer problema es la ausencia de aislamiento: un fichero compartido por una docena de servicios expone a cada uno de ellos a los secretos de todos los demás. Si un contenedor resulta comprometido —por ejemplo, mediante una vulnerabilidad en una dependencia de la aplicación—, el atacante obtiene de inmediato la contraseña del servidor de correo, el token del proveedor de copias de seguridad y la clave de la pasarela de pago, aunque el servicio vulnerado no guardara ninguna relación con ellos. El radio de exposición se desacopla por completo del alcance funcional de la pieza afectada.

El segundo problema es operativo: un `.env` monolítico convierte cualquier cambio menor —rotar una sola clave— en una intervención sobre un artefacto que afecta a todo el sistema, lo que desincentiva la rotación y favorece la fosilización de los secretos. El tercero atañe a la trazabilidad: resulta difícil saber qué servicio consume realmente cada variable, de modo que los secretos huérfanos se acumulan y nunca se retiran. La práctica de inyectar secretos como variables de entorno, habitual en las plataformas de contenedores (Docker, 2024), debe matizarse: las variables de entorno son legibles por todos los procesos del contenedor y suelen filtrarse en volcados de error, registros de depuración y la introspección del propio motor de contenedores.

5.4.2. Secretos por servicio y por ámbito

La alternativa es segmentar los secretos por servicio y por ámbito (*scope*). Cada servicio dispone de su propio conjunto de credenciales, idealmente con cuentas dedicadas en los sistemas a los que accede: una base de datos no debe compartir un superusuario entre aplicaciones, sino exponer un rol por consumidor, con permisos acotados a las tablas que le competen. Esta granularidad materializa la *compartimentación* y limita el daño de una credencial filtrada al perímetro de un solo servicio. La segmentación por ámbito añade una dimensión temporal y contextual: credenciales de solo lectura frente a credenciales de escritura, y tokens de corta vida para operaciones puntuales frente a claves persistentes para tareas continuas.

Los esquemas de autorización basados en tokens facilitan esta granularidad. Un *JSON Web Token* (Jones et al., 2015) puede acotarse a un conjunto de permisos y a una ventana de validez explícita, de modo que la credencial caduca por construcción; los marcos de delegación como OAuth 2.0 (Hardt, 2012) permiten emitir tokens de alcance limitado sin compartir la credencial primaria. Frente a una contraseña estática que vive indefinidamente en un `.env`, un token de vida corta reduce el valor de una fuga: lo robado expira antes de poder explotarse con holgura.

5.4.3. Rotación

La rotación es el reconocimiento explícito de que todo secreto se degrada con el tiempo. Cuanto más vive una credencial, mayor es la probabilidad de que haya quedado expuesta en un registro, un volcado, una copia de seguridad mal protegida o la memoria de un operador. Una política de rotación define la periodicidad con que cada clase de secreto se renueva y, sobre todo, la capacidad de revocar y reemplazar una credencial de forma inmediata ante un incidente. Los marcos de control de seguridad recogen la rotación y la gestión del ciclo de vida de los autenticadores como prácticas de referencia (NIST, 2013).

La viabilidad de la rotación depende de que el sistema admita la coexistencia transitoria de dos credenciales válidas para un mismo servicio. Sin ese solapamiento, rotar implica una ventana de indisponibilidad, lo que de nuevo desincentiva la práctica. Por ello la rotación no es solo una política, sino un requisito de diseño: las aplicaciones deben poder recargar sus secretos sin reinicio destructivo, y los sistemas de respaldo deben aceptar más de una clave activa durante la transición.

5.4.4. Bóvedas de secretos

Una *bóveda* (*vault*) es un servicio especializado que custodia los secretos cifrados en reposo, los entrega bajo autenticación y autorización, registra cada acceso y, en sus formas más avanzadas, genera credenciales dinámicas y efímeras bajo demanda. Frente al fichero plano, la bóveda introduce tres mejoras sustantivas: el cifrado en reposo con una clave maestra que no reside junto a los datos; la auditoría de cada lectura, que permite reconstruir quién accedió a qué y cuándo; y la centralización del ciclo de vida, que convierte la rotación en una

operación de un único punto. En un *homelab*, adoptar una bóveda completa puede resultar desproporcionado; existen gradaciones intermedias —como los ficheros cifrados con clave asimétrica que solo el orquestador descifra en el momento del despliegue— que capturan parte del beneficio sin la carga operativa de un servicio adicional. La elección debe ponderar el coste de operación frente al valor de los secretos custodiados.

5.4.5. Secretos en el control de versiones y fugas

El error más frecuente y más costoso es el versionado accidental de secretos en el control de versiones. Un secreto comprometido en el historial no desaparece al borrarlo en un *commit* posterior: permanece accesible en la historia y, si el repositorio se ha publicado o replicado, debe considerarse irremediabilmente filtrado. La medida correcta ante un secreto expuesto no es reescribir la historia, sino revocar y rotar de inmediato la credencial afectada, asumiendo que ya ha sido capturada.

La prevención se articula en varias capas. La primera consiste en excluir por defecto todo fichero de configuración sensible mediante reglas de ignorado, versionando en su lugar plantillas sin valores reales. La segunda interpone ganchos de validación previos al *commit* que detectan patrones de credenciales —cadenas de alta entropía, prefijos conocidos de tokens— y abortan la operación antes de que el secreto entre en la historia. La tercera analiza periódicamente el repositorio en busca de secretos ya filtrados. A estas defensas se suma una recomendación de fondo: separar nítidamente el código de la configuración, de modo que el repositorio nunca necesite contener secretos para ser funcional, y que estos se inyecten en el entorno de ejecución desde una fuente controlada. Las fugas no se limitan al control de versiones; los registros de aplicación, los mensajes de error verbosos y las copias de seguridad sin cifrar son vectores equivalentes, razón por la cual el cifrado de los respaldos (BorgBackup contributors, 2024; restic contributors, 2024) y la depuración de los registros forman parte indisociable de una gestión de secretos coherente.

5.5. Identidad, autenticación y zero-trust

La seguridad de una infraestructura autoalojada no descansa únicamente en el cifrado del tráfico o en el endurecimiento de los servicios, sino en una cuestión más elemental: quién accede y con qué garantías. En un *homelab* (laboratorio doméstico) expuesto, parcial o totalmente, a redes no confiables, la identidad se convierte en el verdadero perímetro. Conviene, por tanto, examinar tres planos que se refuerzan mutuamente: la autenticación fuerte del usuario, la adopción de un modelo *zero-trust* (confianza cero) adaptado al hogar y la centralización del control mediante un proxy de identidad.

5.5.1. De la contraseña a la autenticación fuerte

El factor de conocimiento aislado —la contraseña— constituye el eslabón más frágil de cualquier despliegue. Su reutilización entre servicios, su vulnerabilidad ante el *phishing* y la po-

sibilidad de filtraciones masivas justifican que, en un panorama de autoalojamiento (*self-hosting*), se asuma la autenticación multifactor (MFA) como línea de base y no como refinamiento opcional. El principio de mediación completa formulado por Saltzer y Schroeder exige que todo acceso a un objeto protegido sea verificado, sin atajos derivados de la confianza implícita en sesiones previas o en la red de origen (Saltzer & Schroeder, 1975); la MFA materializa ese principio al sumar un factor de posesión o de inherencia al de conocimiento.

Cabe distinguir, no obstante, entre formas de segundo factor con garantías muy desiguales. Los códigos de un solo uso basados en tiempo (TOTP) mejoran sustancialmente la situación, pero siguen siendo interceptables mediante portales de *phishing* que actúan como intermediarios. Las *passkeys* —credenciales basadas en criptografía asimétrica, ancladas al dominio de origen y custodiadas por el dispositivo del usuario— elevan la barrera de forma cualitativa: la clave privada nunca abandona el autenticador y la verificación queda ligada al origen legítimo, lo que neutraliza la suplantación de dominio. En un autoalojamiento donde el administrador es a la vez usuario y responsable de seguridad, las *passkeys* ofrecen una relación coste-robustez difícil de igualar, pues eliminan secretos compartidos reutilizables sin imponer la fricción operativa de gestionar semillas TOTP por cada servicio.

5.5.2. Zero-trust llevado al hogar

El modelo tradicional de seguridad perimetral asume una red interna fiable y una frontera defendida por cortafuegos. Esa premisa, ya cuestionada en entornos corporativos, resulta especialmente inadecuada en el hogar, donde conviven dispositivos de procedencia heterogénea —televisores, electrodomésticos conectados, equipos de invitados— cuyo firmware rara vez se audita. La arquitectura *zero-trust* invierte el supuesto: ninguna red, ni siquiera la LAN doméstica, se considera de confianza por defecto, y cada solicitud se evalúa según la identidad verificada del usuario y el estado del dispositivo, con independencia de su ubicación (Rose et al., 2020). El acceso deja de concederse por estar «dentro» y pasa a concederse por demostrar, en cada interacción, quién se es.

La formulación pionera de este enfoque, *BeyondCorp*, trasladó la autorización del perímetro de red a un plano de control que combina identidad de usuario y postura del dispositivo, permitiendo prescindir de la red privada virtual como condición de acceso (Ward & Beyer, 2014). Llevado al *homelab*, el principio se traduce en decisiones concretas: segmentar la red interna en una VLAN de servicios y otra para dispositivos no confiables; no exponer paneles de administración por el mero hecho de que el cliente esté en la subred local; y exigir autenticación incluso para servicios que históricamente se dejaban «abiertos» en la intranet doméstica. Esta postura es complementaria, no antagónica, de las defensas de red: un túnel WireGuard puede seguir siendo el medio de transporte cifrado, pero la autenticación de aplicación no se delega en el hecho de haber atravesado el túnel (Donenfeld, 2017).

Conviene matizar que *zero-trust* no significa desconfianza absoluta ni autenticación interminable. Se trata de graduar la confianza: una sesión recientemente verificada con un factor fuerte puede sostener accesos de bajo riesgo durante un intervalo razonable, mientras que

las operaciones sensibles —cambios de configuración o exposición de copias de seguridad— exigen reautenticación. El criterio rector es el del privilegio mínimo (Saltzer & Schroeder, 1975): cada componente y cada sesión disponen solo de los permisos imprescindibles para su función, de modo que un compromiso quede acotado.

5.5.3. El proxy de identidad como punto de control

Implementar estas garantías servicio por servicio sería inviable y propenso a errores: cada aplicación autoalojada posee su propio subsistema de autenticación, con calidades dispares y, a menudo, sin soporte de MFA. La solución arquitectónica consiste en interponer un *proxy de identidad* —también llamado proxy de autenticación o pasarela de identidad— ante el conjunto de servicios. Este componente termina las peticiones entrantes, fuerza la autenticación contra un proveedor de identidad único y solo entonces reenvía la solicitud al servicio destino, que recibe la identidad ya verificada en una cabecera o testigo firmado.

Tal diseño centraliza la política de acceso en un punto único de aplicación, coherente con el argumento extremo a extremo: la función de autorización se ubica en el nivel donde puede ejercerse de manera completa y consistente, en lugar de fragmentarse entre servicios heterogéneos (Saltzer et al., 1984). Los estándares abiertos proporcionan el sustrato técnico. El marco de autorización OAuth 2.0 permite que el proxy delegue la concesión de acceso en un servidor de autorización sin compartir credenciales con cada aplicación (Hardt, 2012), mientras que los *JSON Web Tokens* (JWT) ofrecen un vehículo compacto y firmado para transportar la identidad y las afirmaciones de sesión entre el proxy y los servicios protegidos, verificables sin consultar un estado central (Jones et al., 2015).

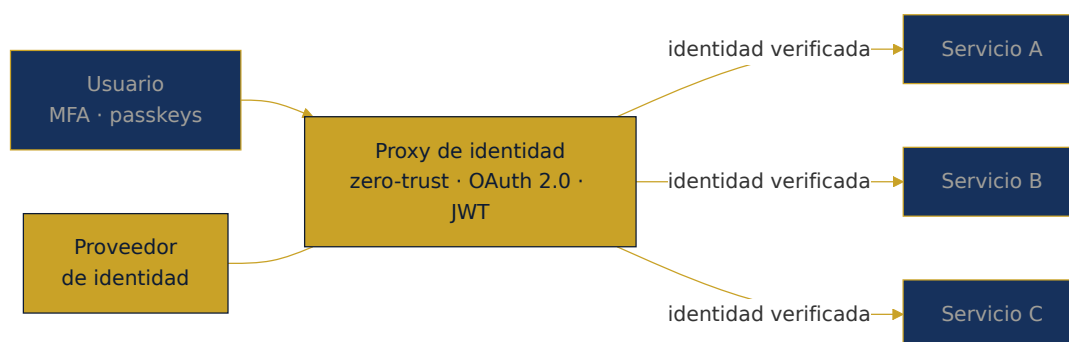


Figura 5.1: El proxy de identidad como punto único de control: concentra la autenticación fuerte (MFA, *passkeys*) y la política *zero-trust*, y reenvía a cada servicio la identidad ya verificada mediante estándares abiertos (OAuth 2.0, JWT).

Las ventajas son acumulativas. La superficie de autenticación se reduce a un solo componente endurecido y auditable, en lugar de a una decena de formularios de inicio de sesión dispares. La MFA y las *passkeys* se configuran una vez y se aplican a todo lo que el proxy protege, incluidos servicios que nunca las soportaron de forma nativa. El registro de accesos se concentra, lo que facilita la detección de anomalías. Y la revocación se vuelve inmediata:

invalidar una sesión en el proveedor de identidad corta el acceso a todos los servicios a la vez, sin recorrer cada aplicación.

No obstante, esta centralización introduce un punto único de fallo cuya criticidad debe asumirse de forma explícita. Si el proxy de identidad cae, el acceso a los servicios se interrumpe; si se compromete, el atacante obtiene una llave maestra. De ahí que su despliegue exija las mismas cautelas que el resto de la infraestructura sensible: actualización diligente, configuración endurecida según controles reconocidos (NIST, 2013), monitorización del propio plano de control y, en la medida de lo posible, redundancia. La identidad bien gobernada no elimina el riesgo, pero lo concentra en un lugar donde puede vigilarse con rigor, que es precisamente lo que la seguridad por capas persigue.

5.6. Cadena de suministro y parches

La seguridad de un despliegue autoalojado (*self-hosting*) no se agota en su configuración: depende también de la procedencia y el mantenimiento del software que lo compone. En un *homelab*, donde el operador asume las funciones que en una organización se reparten entre equipos especializados, la cadena de suministro de software constituye una superficie de ataque tan relevante como la red o la autenticación. Cabe distinguir aquí dos problemas correlativos: la confianza inicial en lo que se instala y el mantenimiento continuado de lo ya instalado.

5.6.1. Procedencia y confianza en las imágenes

El modelo de empaquetado en contenedores ha simplificado el despliegue de servicios hasta el punto de naturalizar una práctica arriesgada: descargar y ejecutar imágenes de origen no verificado (Merkel, 2014). Una imagen de contenedor agrega no solo la aplicación, sino su sistema operativo base, sus bibliotecas y sus utilidades; cada capa hereda las vulnerabilidades de su contenido. Conviene, por ello, jerarquizar las fuentes. Las imágenes oficiales mantenidas por el propio proyecto o por un registro con curaduría ofrecen mayores garantías que las publicadas por terceros anónimos, cuya cadena de construcción resulta opaca.

La firma criptográfica de artefactos permite trasladar este juicio de confianza a una verificación técnica. Mediante firmas digitales, el operador puede comprobar que una imagen procede efectivamente de quien dice publicarla y que no ha sido alterada en tránsito. La documentación de Docker describe mecanismos de confianza en el contenido que asocian las etiquetas a firmas verificables (Docker, 2024). El principio subyacente es el de *fail-safe defaults* enunciado por Saltzer y Schroeder: ante la ausencia de prueba de autenticidad, la postura segura consiste en rechazar el artefacto, no en aceptarlo por defecto (Saltzer & Schroeder, 1975). Anclar la confianza a una identidad criptográfica, y no a la mera popularidad de una imagen, reduce el riesgo de ejecutar código manipulado.

Esta cautela alcanza también a la inteligencia agéntica local. Un agente que orquesta herramientas y consume servicios descansa sobre bibliotecas, tiempos de ejecución y modelos

cuya procedencia debe escrutarse con el mismo rigor que cualquier otra dependencia; la confianza depositada en el componente de razonamiento no exime de verificar el resto de la cadena, pues su capacidad de actuar sobre el sistema amplifica el alcance de cualquier eslabón comprometido.

5.6.2. Dependencias y vulnerabilidades conocidas

Toda aplicación arrastra un grafo de dependencias transitivas que el operador rara vez inspecciona en su totalidad. Las vulnerabilidades documentadas públicamente —los CVE, *Common Vulnerabilities and Exposures*— constituyen el inventario de referencia para evaluar la exposición de ese grafo. Un servicio aparentemente trivial puede heredar, a través de una biblioteca menor, una falla crítica explotable de forma remota. El análisis automatizado de imágenes y manifiestos frente a bases de datos de vulnerabilidades permite detectar estos defectos antes del despliegue, materializando la función de identificación que los marcos de ciberseguridad sitúan en la base de toda gestión de riesgo (NIST, 2018).

La gravedad nominal de un CVE, sin embargo, no se traduce mecánicamente en riesgo efectivo. Una vulnerabilidad crítica en un componente que no se invoca, o que solo resulta accesible desde una red interna ya segmentada, exige una respuesta distinta de la misma falla expuesta a internet. El operador debe, pues, contextualizar cada alerta según la exposición real del servicio afectado, evitando tanto la complacencia como la parálisis ante un flujo continuo de avisos. Esta priorización por contexto convierte un inventario inabarcable en una lista de actuación manejable.

5.6.3. Actualización automática frente a actualización controlada

El mantenimiento de los parches plantea una tensión genuina entre dos virtudes en conflicto: la prontitud en cerrar vulnerabilidades y la estabilidad del servicio. La actualización automática promete que ningún parche crítico quede pendiente por descuido; la actualización controlada protege frente a regresiones, cambios incompatibles o, en el peor caso, una versión comprometida aguas arriba que se propagaría sin revisión humana. Ambas posturas son legítimas, y ninguna domina a la otra en todos los supuestos.

El criterio que conviene adoptar no es dicotómico, sino estratificado según el papel de cada servicio y la naturaleza de cada cambio. Una herramienta que observa los contenedores en ejecución y los reemplaza en cuanto detecta una imagen más reciente optimiza la inmediatez, pero adopta cualquier cambio publicado sin mediación, exponiendo el sistema tanto a regresiones como a una eventual imagen maliciosa. En el extremo opuesto, una herramienta que se limita a detectar actualizaciones en los manifiestos y a generar propuestas de cambio —que el operador revisa, prueba y aprueba antes de integrarlas— optimiza el control a costa de la latencia. La elección entre ambas no debería ser uniforme.

Para los parches de seguridad de versiones menores en servicios periféricos, la automatización resulta razonable, pues el coste de una regresión es bajo y el de un retraso, alto. Para

los servicios que custodian datos o que sostienen la operación —bases de datos, autenticación, almacenamiento—, la prudencia aconseja una revisión deliberada, idealmente sobre una réplica antes de tocar producción. Fijar las versiones mediante referencias inmutables, en lugar de etiquetas móviles, otorga reproducibilidad y permite revertir con certeza a un estado anterior conocido.

Esta gradación encarna, en el plano del mantenimiento, el principio de mínimo privilegio aplicado al cambio: cada actualización debe gozar de la menor autonomía compatible con el riesgo que representa (Saltzer & Schroeder, 1975). Automatizar lo trivial y deliberar sobre lo crítico no es una concesión a la comodidad, sino una asignación racional de la atención escasa del operador. La cadena de suministro segura no es, en definitiva, un estado que se alcanza una vez, sino una disciplina que se sostiene: verificar lo que entra, vigilar lo que permanece y decidir con criterio cuándo y cómo se renueva.

5.7. IA local: prompt injection, exfiltración y agencia

La incorporación de modelos de lenguaje a la infraestructura autoalojada introduce una clase de amenazas sin equivalente directo en los servicios convencionales. Mientras que un servidor web o una base de datos exponen interfaces con una semántica acotada y verificable, un agente basado en un modelo de lenguaje interpreta entradas en lenguaje natural y deriva de ellas acciones. Esa indistinción entre dato e instrucción es la raíz de los nuevos vectores. Conviene examinarlos con precisión, porque la *agencia agéntica* —la capacidad del sistema de planificar y ejecutar acciones sobre su entorno en lugar de limitarse a producir texto— no añade riesgo de forma incremental, sino que transforma la naturaleza misma de la superficie de ataque.

5.7.1. La inyección de instrucciones como vector primario

La *inyección de instrucciones* (*prompt injection*) describe la situación en la que contenido externo, procesado por el modelo como parte de su contexto, contiene directivas que el sistema interpreta y obedece como si procedieran del operador legítimo. La distinción decisiva, formulada por Greshake et al. (2023), separa la inyección directa —el atacante interactúa con el modelo a través de la entrada prevista— de la inyección indirecta, en la que la carga maliciosa se aloja en datos que el modelo consume de manera incidental: el cuerpo de un correo, el texto de una página recuperada, los metadatos de un documento o un comentario en un repositorio. Esta segunda variante resulta especialmente insidiosa porque rompe el supuesto implícito de confianza sobre el origen de las instrucciones. El operador nunca redactó la orden; el atacante la sembró en una fuente que el agente consulta de forma rutinaria.

La gravedad de esta amenaza deriva de una propiedad arquitectónica de los modelos actuales: no existe una separación robusta entre el canal de control y el canal de datos. En los sistemas clásicos, esa separación constituye un principio de diseño consolidado, y las vulnerabilidades de inyección de código surgen precisamente cuando se quebranta. En un modelo

de lenguaje, en cambio, instrucción y dato comparten el mismo flujo de tokens y se procesan mediante el mismo mecanismo de atención (Vaswani et al., 2017), de modo que no hay un marcador sintáctico fiable que delimite hasta dónde llega lo que el operador autoriza y dónde empieza lo que un tercero introdujo. Las defensas basadas en delimitadores textuales o en instrucciones de sistema que ordenan «ignorar órdenes contenidas en los datos» son heurísticas, no garantías: el modelo puede ser persuadido a desatenderlas. El catálogo de la OWASP (2025) para aplicaciones de modelos de lenguaje sitúa esta inyección en el primer lugar de sus riesgos, lo que refleja tanto su prevalencia como la dificultad de mitigarla por medios puramente internos al modelo.

5.7.2. De la inyección a la exfiltración mediante herramientas

La inyección, por sí sola, altera el comportamiento del modelo; su consecuencia más dañina se materializa cuando ese comportamiento alterado dispone de herramientas con efectos en el mundo. Un agente que únicamente devuelve texto puede ser inducido a mentir o a emitir contenido inapropiado, pero su capacidad de daño queda contenida en la respuesta. Un agente conectado a herramientas —lectura del sistema de archivos, envío de peticiones de red, ejecución de comandos, acceso a un gestor de correo o a una API— convierte la instrucción inyectada en acción consumada. La *exfiltración mediante herramientas* explota esta vía: la carga maliciosa no pide al modelo que revele datos en su respuesta, sino que los transmita a un destino controlado por el atacante usando las capacidades legítimas del agente. Una página manipulada puede instruir al agente para que lea un fichero de credenciales del entorno local y lo incruste como parámetro en la URL de una petición saliente; el dato abandona el perímetro sin que ningún componente haya sido vulnerado en el sentido tradicional, porque cada acción individual estaba autorizada.

Este mecanismo es particularmente relevante en el autoalojamiento. La promesa de la arquitectura *local-first* (Kleppmann et al., 2019) y de la *soberanía digital* consiste en que los datos residen bajo control del propietario y no se ceden a terceros. Un agente con permisos amplios sobre ese sustrato invierte la garantía: concentra en un único componente, gobernado por entradas no confiables, la potestad de tocar precisamente aquello que la arquitectura pretendía proteger. El argumento extremo a extremo (Saltzer et al., 1984) sostiene que las propiedades de seguridad deben asegurarse en los extremos de la comunicación; en un sistema agéntico, el modelo se sitúa en un extremo que carece de la capacidad de distinguir interlocutores legítimos de instrucciones hostiles incrustadas en el tráfico que procesa.

5.7.3. El diputado confundido y la amplificación por la agencia

El fenómeno admite una lectura desde un principio anterior a la era de los modelos de lenguaje: el problema del *diputado confundido* (*confused deputy*). Se produce cuando un componente que posee privilegios legítimos es manipulado por un tercero menos privilegiado para que ejerza esos privilegios en su nombre. El agente encaja en esta figura con exactitud: opera con la autoridad delegada del operador —sus tokens de acceso, sus permisos sobre ficheros y

servicios— pero acepta como instrucciones contenidos cuyo origen no controla. El atacante no necesita poseer las credenciales; le basta con que el diputado, debidamente confundido, las emplee por él. La tradición de la seguridad de sistemas ofrece aquí un marco consolidado: la mediación completa y el privilegio mínimo (Saltzer & Schroeder, 1975) son precisamente las disciplinas que un agente con permisos generosos y mediación laxa incumple.

La razón por la que la agencia amplía el riesgo puede formularse con nitidez. La capacidad de actuar transforma una vulnerabilidad de confidencialidad en una de integridad y disponibilidad: el sistema comprometido no solo puede filtrar información, sino modificar estados, borrar copias, alterar configuraciones o propagar la instrucción a otros sistemas que confían en él. Cuando el agente encadena razonamiento y acción de manera iterativa, según el patrón que articulan Yao et al. (2023), cada paso de ejecución amplía el alcance del siguiente: una observación contaminada en el primer ciclo condiciona todas las decisiones ulteriores, y el horizonte de daño crece con la longitud de la cadena. La autonomía, que constituye el valor del agente, es simultáneamente el multiplicador de su peligro. Anthropic (2024) advierte que la complejidad y la latitud concedidas al agente deben justificarse por la tarea, pues cada incremento de autonomía es también un incremento de superficie expuesta.

5.7.4. Implicaciones para el diseño y enlace con la gobernanza

De lo anterior se desprende que las defensas no pueden residir en el interior del modelo, donde la indistinción entre instrucción y dato es estructural, sino en la arquitectura que lo rodea. Cabe distinguir varias líneas convergentes:

- Acotar el conjunto de herramientas disponibles a cada agente y los efectos que cada una puede producir, aplicando privilegio mínimo no al operador, sino al agente como sujeto autónomo.
- Interponer mediación entre la decisión del modelo y su ejecución, de modo que las acciones irreversibles o salientes requieran confirmación o queden sujetas a políticas verificables fuera del modelo.
- Tratar toda entrada externa como no confiable por defecto, segmentando los contextos según su procedencia y limitando el acceso del agente a datos sensibles cuando procesa contenido de origen incierto.
- Registrar y auditar la secuencia de acciones, de manera que la cadena de razonamiento y ejecución sea reconstruible a posteriori.

Estas medidas no eliminan la inyección, cuya raíz es por ahora irreductible, sino que limitan su capacidad de convertirse en daño. Su formalización trasciende lo técnico y desemboca en el plano de la gobernanza: definir qué autoridad se delega, sobre qué recursos, bajo qué supervisión y con qué rendición de cuentas son decisiones de política, no de implementación. La frontera entre lo que un agente local puede hacer y lo que debe quedarle vedado constituye el objeto del capítulo siguiente, donde estas amenazas se reexaminan como exigencias de control deliberado sobre el cómputo soberano.

5.8. Continuidad y resiliencia

La seguridad de un sistema autoalojado no se agota en la confidencialidad y la integridad; comprende también la disponibilidad. Un servicio inaccesible o unos datos irrecuperables constituyen un fallo de seguridad tan severo como una filtración, aunque su origen no sea un adversario sino una avería. Por ello, la fiabilidad opera como la cara B de la seguridad: el modelo clásico que articula la protección de la información ya situaba la disponibilidad en pie de igualdad con la confidencialidad y la integridad (Saltzer & Schroeder, 1975). En un *homelab* (laboratorio doméstico de cómputo), donde no existe el respaldo operativo de un proveedor con redundancia geográfica, la continuidad depende íntegramente del diseño y la previsión de quien administra. Conviene, pues, tratar los fallos no adversariales con el mismo rigor metodológico que las amenazas deliberadas, incorporándolos al análisis de riesgos del marco de gestión de la seguridad (NIST, 2018).

5.8.1. Modos de fallo en infraestructura doméstica

Cabe distinguir tres categorías de fallo físico recurrentes. El primero es el corte de energía. La red eléctrica doméstica carece de la redundancia de un centro de datos, y una interrupción abrupta puede corromper sistemas de ficheros en plena escritura o dañar discos. Un sistema de alimentación ininterrumpida (SAI, en inglés *UPS*) mitiga este riesgo: no se concibe tanto para sostener cargas prolongadas como para conceder el margen necesario a un apagado ordenado. La integración del SAI con el sistema operativo —mediante un demonio que monitorice la carga de la batería y dispare la secuencia de parada al alcanzar un umbral— resulta más valiosa que su mera autonomía nominal, pues evita que la batería se agote sin haber preservado el estado.

El segundo es el fallo de disco. Los soportes de almacenamiento poseen una tasa de avería no despreciable y, sobre todo, tienden a fallar de forma correlacionada cuando proceden del mismo lote. La redundancia mediante arreglos —réplicas o paridad— protege frente a la pérdida de una unidad, pero no sustituye a la copia de seguridad: un arreglo replica también los borrados accidentales y la corrupción lógica. Conviene, por tanto, separar conceptualmente la redundancia (disponibilidad ante fallo de hardware) del respaldo (recuperación ante pérdida lógica o catastrófica).

El tercero es el fallo de memoria o de cómputo, más insidioso porque puede degradar datos de forma silenciosa antes de manifestarse. Los sistemas de ficheros con suma de verificación de extremo a extremo encarnan el argumento de que ciertas garantías de integridad solo pueden asegurarse en los extremos del sistema, no en componentes intermedios (Saltzer et al., 1984).

5.8.2. Degradación elegante

Un sistema resiliente no aspira a no fallar nunca, sino a fallar de manera controlada. La *degradación elegante* designa la capacidad de preservar las funciones esenciales aun cuando otras

se pierdan. Frente al diseño monolítico, en el que una avería compromete el conjunto, la compartimentación en servicios independientes —favorecida por la contenedorización (Merkel, 2014)— permite que el fallo de un componente no propague su efecto. Conviene jerarquizar los servicios por criticidad y desacoplarlos, de modo que la pérdida de un módulo accesorio no arrastre a los esenciales.

Esta lógica enlaza con un resultado fundamental de los sistemas distribuidos: ante una partición de la red, no es posible mantener simultáneamente consistencia fuerte y disponibilidad, y todo diseño debe elegir conscientemente qué sacrificar (Brewer, 2000; Gilbert & Lynch, 2002). En el contexto doméstico, donde la conectividad es intermitente y la tolerancia a la inconsistencia transitoria suele ser alta, conviene a menudo privilegiar la disponibilidad local. El enfoque *local-first* (primacía local) sistematiza esta elección: la aplicación opera sobre datos locales con independencia de la conectividad y reconcilia los cambios cuando se restablece la red (Kleppmann et al., 2019). Así, la resiliencia deja de ser un añadido a la infraestructura de *self-hosting* (autoalojamiento) y se convierte en una propiedad de su arquitectura.

5.8.3. Plan de recuperación

La resiliencia se demuestra en la recuperación, no en la ausencia de fallos. Un plan riguroso fija dos parámetros: el objetivo de tiempo de recuperación —cuánto puede tardar el servicio en restablecerse— y el objetivo de punto de recuperación —cuántos datos pueden perderse, medido como antigüedad de la última copia válida. Ambos determinan la frecuencia de respaldo y la arquitectura de la restauración, y deben fijarse por servicio según su criticidad antes de dimensionar los mecanismos que los sostienen.

Las herramientas de copia con deduplicación, compresión y cifrado autenticado permiten conservar múltiples versiones históricas a un coste de almacenamiento moderado (BorgBackup contributors, 2024; restic contributors, 2024). El cifrado del respaldo extiende la confidencialidad al soporte de la copia, que con frecuencia reside fuera del perímetro controlado. Conviene observar la regla de mantener varias copias, en soportes distintos, con al menos una fuera de la ubicación física, de manera que un siniestro local —incendio, robo, sobretensión— no destruya simultáneamente original y respaldo.

Ahora bien, una copia no verificada equivale a una hipótesis, no a una garantía. La restauración debe ensayarse periódicamente: solo la prueba efectiva confirma que los datos son recuperables y que el procedimiento es ejecutable bajo presión. Documentar la secuencia de recuperación, mantenerla accesible al margen del sistema que protege y revisarla tras cada cambio relevante de la infraestructura completa la disciplina. Así entendida, la continuidad deja de ser un atributo accesorio y se incorpora al núcleo de la postura de seguridad del sistema autoalojado.

6 Gobernanza y agencia segura

La transición desde sistemas que se limitan a ejecutar reglas predefinidas hacia agentes capaces de decidir y actuar por sí mismos introduce una clase de problemas que la automatización clásica no plantea. Cuando un sistema *local-first* incorpora *agencia agéntica*, es decir, la capacidad de encadenar razonamiento y acción sobre el entorno sin intervención humana en cada paso, deja de ser una mera herramienta determinista y pasa a ejercer una forma acotada de iniciativa. Se argumenta en este capítulo que esa capacidad, lejos de ser un atributo deseable por sí mismo, constituye una fuente de riesgo que solo resulta admisible cuando se acompaña de mecanismos explícitos de gobernanza, contención y trazabilidad.

La tesis que se desarrolla es que la *soberanía digital* sobre la infraestructura propia no se agota en el control de los datos y los servicios del *homelab*, sino que exige extender ese control al comportamiento de los agentes que operan dentro de él. Alojar un servicio en casa elimina la dependencia de un tercero; dotar a ese servicio de capacidad de actuar introduce, en cambio, un nuevo agente cuya autonomía debe ser gobernada con el mismo rigor con que se gobierna el acceso a la red o el almacenamiento. La gobernanza no se concibe aquí como una restricción externa impuesta sobre la *agencia agéntica*, sino como la condición que la hace compatible con el *autoalojamiento* responsable.

A lo largo del capítulo se recorre ese problema en sus distintas facetas. Conviene partir de la distinción entre automatización y agencia, para situar después los riesgos específicos que comporta delegar acciones, en particular las irreversibles. Sobre ese diagnóstico se examinan los patrones de contención disponibles —confirmación, *capabilities* y *sandboxing*—, los mecanismos de trazabilidad que permiten auditar, aprobar y revertir lo actuado, y la gobernanza del dato y del proceso de aprendizaje. El recorrido se cierra con el papel del humano en el bucle, entendido no como un residuo de desconfianza, sino como el punto en que la soberanía se reafirma sobre la delegación.

6.1. De la automatización a la agencia

La automatización doméstica clásica descansa sobre reglas deterministas: una condición previamente definida desencadena una acción previamente definida. El esquema canónico es la regla *si esto, entonces aquello*; por ejemplo, «si el sensor de movimiento se activa después del ocaso, enciéndase la luz del pasillo». El comportamiento del sistema es enteramente trazable hasta una especificación escrita por una persona, y su espacio de respuestas posibles está acotado de antemano. Aun cuando la lógica se vuelve intrincada —rutinas encadenadas, máquinas de estados, planificadores temporales—, la propiedad esencial se conserva: el sistema

no decide *qué* objetivo perseguir ni *cómo* alcanzarlo, sino que ejecuta un mapeo fijo de entradas a salidas. La verificación, en consecuencia, es tratable: cabe enumerar las ramas, probar los casos límite y razonar sobre invariantes. La plataforma doméstica más extendida articula precisamente esta clase de automatización mediante disparadores, condiciones y acciones declaradas (Home Assistant, 2024), un modelo que ha probado su robustez justamente porque renuncia a la apertura semántica.

La *agencia agéntica* introduce una ruptura cualitativa, no un mero incremento de complejidad. Un sistema agéntico recibe un objetivo expresado en lenguaje natural —a menudo subespecificado— e infiere por sí mismo la secuencia de pasos, la selección de herramientas y los criterios de parada necesarios para satisfacerlo. La distinción decisiva no es el uso de aprendizaje automático, sino la ubicación de la frontera de decisión: en la automatización determinista esa frontera reside en el código que la persona escribió; en la agencia se desplaza al modelo, que la fija en tiempo de ejecución y de forma dependiente del contexto. La literatura técnica reserva el término *agente* para los sistemas en que un modelo de lenguaje dirige dinámicamente sus propios procesos y el uso de herramientas, conservando el control sobre cómo completa la tarea (Anthropic, 2024). El patrón que cimenta esta capacidad entrelaza el razonamiento explícito con la acción sobre un entorno: el modelo alterna pasos de deliberación y pasos de actuación, observa el resultado de cada acción y replanifica en consecuencia (Yao et al., 2023). De ese bucle de razonamiento, acción y observación emerge una conducta que ya no es una función fija de la entrada, sino una trayectoria construida sobre la marcha.

6.1.1. El espectro de autonomía

Resulta más fértil concebir la autonomía como un continuo que como una dicotomía. En un extremo se sitúa la ejecución estrictamente prescrita; en el otro, la delegación abierta de objetivos. Conviene distinguir al menos cuatro grados a lo largo de ese eje.

- *Automatización fija*: reglas deterministas sin componente inferencial; toda decisión está codificada por una persona.
- *Asistencia*: el modelo sugiere o redacta, pero una persona valida y ejecuta cada acción con efecto sobre el mundo. La inteligencia es consultiva.
- *Agencia supervisada*: el modelo planifica y actúa de forma autónoma, si bien las acciones de mayor consecuencia quedan sujetas a confirmación humana mediante puntos de control explícitos.
- *Agencia autónoma*: el sistema persigue el objetivo de principio a fin, encadenando decisiones y efectos sin intervención por paso, y solo rinde cuentas del resultado agregado.

Dos magnitudes ortogonales gradúan la posición de un sistema en este espectro. La primera es el *alcance de la acción*: la amplitud del conjunto de operaciones que el sistema puede ejecutar con efecto irreversible sobre datos, dispositivos o terceros. La segunda es la *latitud decisoria*: el margen que el sistema posee para elegir medios y reordenar fines ante la ambigüedad. Un corrector ortográfico tiene latitud alta pero alcance nulo, pues no actúa sobre el mundo. Un

temporizador tiene alcance acotado y latitud nula. El riesgo no proviene de ninguna de las dos magnitudes por separado, sino de su producto: un sistema que combina amplio alcance de acción con elevada latitud decisoria concentra una capacidad de causar efectos que ningún paso humano filtra de antemano. Esa conjunción es la que obliga a replantear la gobernanza, pues desplaza la pregunta del comportamiento esperado al comportamiento posible.

6.1.2. Por qué la IA local doméstica cruza la línea

La incorporación de un modelo de lenguaje con capacidad de actuar dentro del hogar no constituye una mejora gradual de la domótica preexistente, sino el cruce de un umbral cualitativo. Varias razones lo explican.

En primer lugar, el control deja de ser enumerable. Mientras la regla determinista admite auditoría exhaustiva de sus ramas, la política de decisión de un modelo es un objeto estadístico de alta dimensión cuyo comportamiento no se deja agotar mediante casos de prueba. La incertidumbre sobre lo que el sistema hará ante una entrada inédita es intrínseca, no un defecto subsanable con más documentación. Esta propiedad invierte la relación habitual entre especificación y conducta: ya no se valida que la implementación satisfaga una especificación cerrada, sino que se acota un espacio de comportamiento que permanece abierto por diseño.

En segundo lugar, la frontera de confianza se redibuja. La automatización clásica solo procesa señales estructuradas de sensores; un agente, en cambio, ingiere lenguaje natural procedente de correos, calendarios, páginas o mensajes. Ese contenido puede portar instrucciones maliciosas que el modelo interprete como órdenes legítimas, vector conocido como inyección indirecta de instrucciones (Greshake et al., 2023), reconocido hoy como riesgo principal de las aplicaciones que integran modelos de lenguaje (OWASP, 2025). Un dato deja así de ser solo un dato y puede convertirse en una instrucción ejecutada con los privilegios del agente. La distinción entre canal de datos y canal de control, que el diseño de sistemas seguros ha procurado mantener nítida, se difumina cuando el intérprete de las órdenes es el mismo que procesa el contenido no fiable.

En tercer lugar, la conjunción de alcance y latitud descrita arriba se materializa precisamente en el hogar autoalojado. El propósito del *self-hosting* o autoalojamiento es retener bajo control propio servicios, datos e infraestructura; un agente local opera, por tanto, sobre ficheros personales, dispositivos físicos y credenciales que gobiernan la vida cotidiana. La autonomía que en un servicio remoto quedaría acotada por las barreras del proveedor encuentra aquí un perímetro de acción mucho más íntimo y menos compartimentado. La misma soberanía digital que motiva el autoalojamiento —custodiar los datos en lugar de cederlos a infraestructuras de terceros— concentra en un único perímetro el dato sensible y la capacidad de actuar sobre él, de modo que la ventaja de gobernanza se torna superficie de riesgo si no se acota deliberadamente.

De ello se sigue que la pregunta pertinente deja de ser «¿funciona la regla?» para convertirse en «¿qué está autorizado a decidir y a ejecutar este sistema, y bajo qué garantías?». El tránsito

de la automatización a la agencia no suprime la necesidad de fiabilidad heredada de la domótica, sino que le superpone una exigencia nueva: gobernar una capacidad de decisión que ya no reside íntegramente en manos humanas. Cabe situar deliberadamente cada función en el grado de autonomía que su alcance y su latitud justifiquen, e interponer garantías allí donde su producto lo exija. Las secciones siguientes abordan cómo acotar esa capacidad sin renunciar a su utilidad.

6.2. Riesgos de la agencia: permisos y acciones irreversibles

La diferencia cualitativa entre un sistema que únicamente genera texto y uno dotado de *agencia agéntica* reside en que el segundo actúa: invoca herramientas, modifica estado y produce efectos que persisten más allá de la conversación que los originó. Un modelo de lenguaje confinado a emitir respuestas comete, en el peor de los casos, errores informativos que el usuario puede ignorar o corregir antes de obrar en consecuencia. Un agente que controla un cliente de correo, una interfaz de pago o el sistema de ficheros de un *homelab* (laboratorio doméstico de cómputo) traslada esos mismos errores al plano de los hechos consumados. La cuestión, por tanto, deja de ser la calidad del razonamiento y pasa a ser el alcance y la reversibilidad de las acciones que ese razonamiento desencadena.

6.2.1. Acciones con efectos en el mundo

Conviene clasificar las acciones de un agente no por su complejidad técnica, sino por la naturaleza de su efecto. Cabe distinguir tres familias especialmente delicadas. La primera comprende las acciones *destructivas*: borrar ficheros, vaciar una base de datos, sobrescribir una copia, eliminar un volumen. La segunda agrupa las acciones *onerosas*, aquellas que comprometen recursos finitos o dinero: contratar un servicio, aprovisionar capacidad en la nube, ejecutar una transferencia. La tercera reúne las acciones *comunicativas* con destinatario externo: enviar un mensaje, publicar contenido, responder a un tercero en nombre del usuario. Las tres comparten un rasgo: su huella excede el sistema local y, una vez producidas, no admiten una simple operación de deshacer.

Esta taxonomía importa porque el coste de un fallo no se distribuye de manera uniforme. Una recomendación equivocada se descarta sin consecuencias; un correo enviado a la lista equivocada, no. La asimetría entre la facilidad con que un agente puede emitir la orden y la dificultad con que el entorno puede absorber su resultado constituye el núcleo del problema de seguridad de la agencia.

6.2.2. Irreversibilidad

La irreversibilidad es la propiedad que convierte un error en daño. Una operación se considera reversible cuando existe una operación inversa que restituye el estado previo a un coste acotado. Buena parte de las acciones con efecto externo carecen de tal inversa. Un fichero

eliminado puede recuperarse de una copia de seguridad, pero solo si esa copia existe, es íntegra y precede al borrado; las herramientas de respaldo deduplicado y cifrado documentan precisamente este supuesto y sus límites (BorgBackup contributors, 2024; restic contributors, 2024). Un mensaje entregado, en cambio, no se recupera: a lo sumo se emite una rectificación, que es un acto nuevo y no la anulación del anterior. Un pago ejecutado puede, en algunos casos, revertirse mediante un segundo movimiento, pero la reversión depende de la voluntad de un tercero y rara vez es instantánea ni gratuita.

De esta gradación se desprende un principio operativo: el diseño debe ordenar las acciones según su grado de reversibilidad y reservar las salvaguardas más estrictas para las irreversibles. La distinción es más útil que la dicotomía entre acción «peligrosa» e «inocua», porque obliga a preguntarse no qué puede salir mal, sino qué quedará fuera de remedio si sale mal. Un argumento análogo al principio extremo a extremo (Saltzer et al., 1984) sugiere situar la verificación de la irreversibilidad lo más cerca posible del punto donde el efecto se materializa, y no confiarla a capas intermedias que ignoran las consecuencias últimas de la operación.

6.2.3. Sobre-permisos

Un agente solo puede ejecutar aquello que su entorno le autoriza. De ahí que el reparto de permisos sea la palanca primaria de control y, simultáneamente, la fuente más frecuente de exposición. El *sobre-permiso* designa la situación en que un agente dispone de capacidades que su función no requiere: acceso de escritura cuando bastaría la lectura, alcance sobre todo el sistema de ficheros cuando le concierne un solo directorio, credenciales con vigencia indefinida cuando la tarea es puntual. El sobre-permiso no causa daño por sí mismo, pero amplía la superficie sobre la que cualquier fallo o desviación puede propagarse.

El principio del menor privilegio prescribe que todo componente opere con el conjunto mínimo de capacidades necesarias para su cometido (Saltzer & Schroeder, 1975). Aplicado a la agencia, exige acotar no solo *qué* herramientas se exponen al agente, sino el *radio de acción* de cada una: una herramienta de envío de correo restringida a destinatarios de una lista blanca, una herramienta de borrado limitada a una papelera reversible, una credencial de pago con tope por operación. Estos límites materializan, en el plano de la agencia, la lógica de mínimo privilegio y de defensa por capas que la práctica de seguridad recomienda con carácter general (NIST, 2013). La concesión amplia y permanente de permisos, cómoda durante el desarrollo, deviene pasivo de seguridad en explotación.

6.2.4. El agente como diputado confundido

El sobre-permiso adquiere especial gravedad al combinarse con la naturaleza del agente como intermediario. Un agente actúa con la autoridad delegada del usuario, pero decide sus acciones a partir de entradas cuyo origen no siempre controla: instrucciones del usuario, contenidos recuperados, resultados de herramientas. Cuando una de esas entradas procede de un tercero hostil, el agente puede ser inducido a ejercer su autoridad legítima al servicio de

un objetivo ajeno. Es la figura clásica del *diputado confundido* (confused deputy): un componente privilegiado engañado para que emplee sus privilegios en beneficio de quien no los posee.

La inyección indirecta de instrucciones ilustra el mecanismo con precisión. Un agente que procesa un documento o una página puede encontrar, embebido en ese contenido, un texto redactado para que lo interprete como una orden y que lo lleve, por ejemplo, a exfiltrar datos o a ejecutar una acción destructiva (Greshake et al., 2023). El catálogo de riesgos específicos de las aplicaciones basadas en modelos de lenguaje recoge esta clase de ataques como una de sus categorías centrales (OWASP, 2025). Lo relevante es que el problema no se resuelve mejorando el modelo: mientras la frontera entre la instrucción confiable y el dato no confiable permanezca borrosa, ningún grado de competencia del agente impide que confunda una con otra. La defensa ha de residir en la arquitectura de permisos, no en la perspicacia del modelo.

6.2.5. Necesidad de límites

De lo anterior se sigue que la agencia segura no se obtiene restringiendo lo que el agente piensa, sino acotando lo que puede hacer. Tres líneas de contención resultan indispensables. La primera es la confirmación humana previa para las acciones irreversibles y onerosas, que reintroduce un punto de decisión deliberado antes del efecto. La segunda es la acotación del radio de cada herramienta conforme al menor privilegio, de modo que el daño máximo posible quede limitado por diseño y no por la prudencia del agente. La tercera es la trazabilidad: registrar qué acción se ejecutó, con qué autorización y a raíz de qué entrada, condición sin la cual ni la auditoría ni la atribución de responsabilidad son posibles. La premisa de no fiarse por defecto, propia de las arquitecturas de confianza nula (Rose et al., 2020), se traslada así al interior del propio agente. La agencia útil y la agencia segura no se oponen; convergen cuando la potencia de actuar se acompaña de límites proporcionados a la irreversibilidad de cada acto.

6.3. Patrones de contención: confirmación, capabilities y sandboxing

La gobernanza de un agente autónomo no se agota en políticas declarativas ni en la auditoría posterior de su comportamiento; requiere un repertorio de mecanismos técnicos que limiten, en tiempo de ejecución, lo que el agente puede llegar a hacer. Estos mecanismos se denominan aquí *patrones de contención*: disposiciones arquitectónicas que acotan el alcance efectivo de la *agencia agéntica* (la capacidad de un sistema para decidir y ejecutar acciones sobre su entorno sin intervención humana directa). La premisa que los justifica es que un modelo de lenguaje integrado en un bucle de razonamiento y acción (Yao et al., 2023) no es un componente de confianza: su salida puede ser inducida por entradas adversarias — en particular mediante inyección indirecta de instrucciones (Greshake et al., 2023; OWASP,

2025)— y su razonamiento, aunque plausible, no ofrece garantías sobre las consecuencias materiales de las herramientas que invoca. La contención, por tanto, no compensa un defecto puntual del modelo, sino que asume estructuralmente su falibilidad. En un contexto de *self-hosting* (autoalojamiento), donde el operador asume sin intermediarios la responsabilidad de la infraestructura, estos patrones adquieren especial relevancia: no hay un proveedor externo que absorba las consecuencias de una acción dañina.

6.3.1. Confirmación humana como punto de control

El patrón más elemental interpone a una persona entre la decisión del agente y su efecto. La confirmación humana (*human-in-the-loop*) suspende la ejecución de acciones designadas como sensibles hasta recibir una autorización explícita. Su valor reside en concentrar la supervisión donde el coste de un error es asimétrico: borrar un volumen de datos, transferir fondos o publicar contenido externo son operaciones irreversibles o de difícil reversión que justifican el peaje de latencia que la confirmación impone. La literatura sobre construcción de agentes recomienda precisamente reservar la autonomía plena para acciones de bajo riesgo y exigir validación en las de alto impacto (Anthropic, 2024).

Conviene, no obstante, calibrar este patrón con cautela. La confirmación indiscriminada degenera en fatiga de aprobación: cuando se solicita consentimiento para cada paso, el operador termina autorizando de forma mecánica, y la salvaguarda se vuelve nominal. El diseño efectivo distingue clases de acción por su reversibilidad y su radio de impacto, y solo eleva a confirmación aquellas que cruzan un umbral definido. Asimismo, la confirmación protege contra el error del agente, pero no contra el engaño del operador: una acción maliciosa presentada de forma engañosa puede ser aprobada de buena fe, por lo que este patrón debe combinarse con los restantes y no sustituirlos.

6.3.2. Capabilities declarativas y diseño fail-closed

Un segundo patrón sustituye la confianza implícita por una enumeración explícita de lo permitido. Una *capability* es un permiso concreto y verificable —leer cierto directorio, invocar cierta API, escribir en cierta cola— que el agente debe poseer para ejecutar una acción. El enfoque declarativo consiste en describir el conjunto de capabilities de una tarea por adelantado, de modo que toda operación no contemplada quede prohibida por omisión. Este es el principio del menor privilegio formulado por Saltzer y Schroeder (1975): cada componente opera con el mínimo de autoridad indispensable para su función.

La propiedad decisiva de este diseño es su carácter *fail-closed*: ante la ausencia de una autorización, la ambigüedad o el fallo del mecanismo de comprobación, el sistema deniega en lugar de permitir. La alternativa *fail-open* —conceder por defecto y prohibir solo lo explícitamente vetado— es inadecuada para un agente, porque el espacio de acciones posibles es abierto y un atacante siempre puede buscar la operación no prevista. La diferencia es la misma que media entre una *allow-list* y una *deny-list*: la primera define una frontera positiva y finita; la

segunda persigue un conjunto de amenazas inagotable. Las capabilities declarativas materializan, en la práctica, la mediación completa que Saltzer y Schroeder (1975) exigen: ningún acceso elude la comprobación de autoridad. En arquitecturas de red este mismo razonamiento sustenta los modelos de confianza cero, donde cada solicitud se verifica con independencia de su origen (Rose et al., 2020; Ward & Beyer, 2014).

6.3.3. Sandboxing: worktrees y contenedores

Cuando la acción del agente no puede preautorizarse con granularidad suficiente, la contención se desplaza del permiso al entorno. El *sandboxing* confina la ejecución a un espacio aislado cuyo radio de daño está acotado por construcción. En el dominio del código, los *worktrees* de un sistema de control de versiones ofrecen un aislamiento ligero: el agente opera sobre una copia de trabajo segregada, sus modificaciones quedan contenidas en una rama propia y la integración con el estado principal exige una fusión que puede revisarse antes de consolidarse. El espacio canónico permanece intacto mientras el agente experimenta sobre una réplica desechable.

Para un aislamiento más fuerte se recurre a la contenerización. Un contenedor proporciona al proceso un sistema de ficheros, una vista de red y un conjunto de recursos propios, separados del anfitrión mediante los mecanismos de aislamiento del núcleo (Merkel, 2014; Docker, 2024). Frente al *worktree*, que aísla el estado de un repositorio pero comparte el sistema operativo subyacente, el contenedor delimita además el acceso a la red, a otros procesos y al árbol de ficheros del anfitrión. Esta gradación importa: el coste de provisión y la rigidez del aislamiento crecen del *worktree* al contenedor, y la elección debe responder al perfil de amenaza de la tarea. Un agente que solo edita texto en un repositorio rara vez necesita la barrera de un contenedor; uno que ejecuta código arbitrario de procedencia incierta la necesita siempre. El *sandboxing* encarna el extremo de la cadena: aun si todas las salvaguardas previas fallan y el agente intenta una acción dañina, el daño no traspasa los límites del recinto.

6.3.4. Credenciales efímeras y minimización del secreto

El aislamiento del entorno resulta insuficiente si el agente porta credenciales de larga vida que le confieren autoridad más allá del recinto. De ahí el patrón de las *credenciales efímeras*: emitir, para cada ejecución, un secreto de alcance y vigencia mínimos. Un *token* web JSON (JWT) acotado a una única tarea (Jones et al., 2015), con expiración breve y un conjunto de reclamaciones que circunscribe las operaciones autorizadas, limita la ventana de explotación si el secreto se filtra. El razonamiento es temporal además de espacial: una credencial que caduca al concluir el *run* no puede reutilizarse en un ataque diferido. Conviene articular su emisión con un marco de autorización delegada (Hardt, 2012), de modo que el agente reciba autoridad acotada sin custodiar las credenciales maestras del usuario, que permanecen fuera de su alcance.

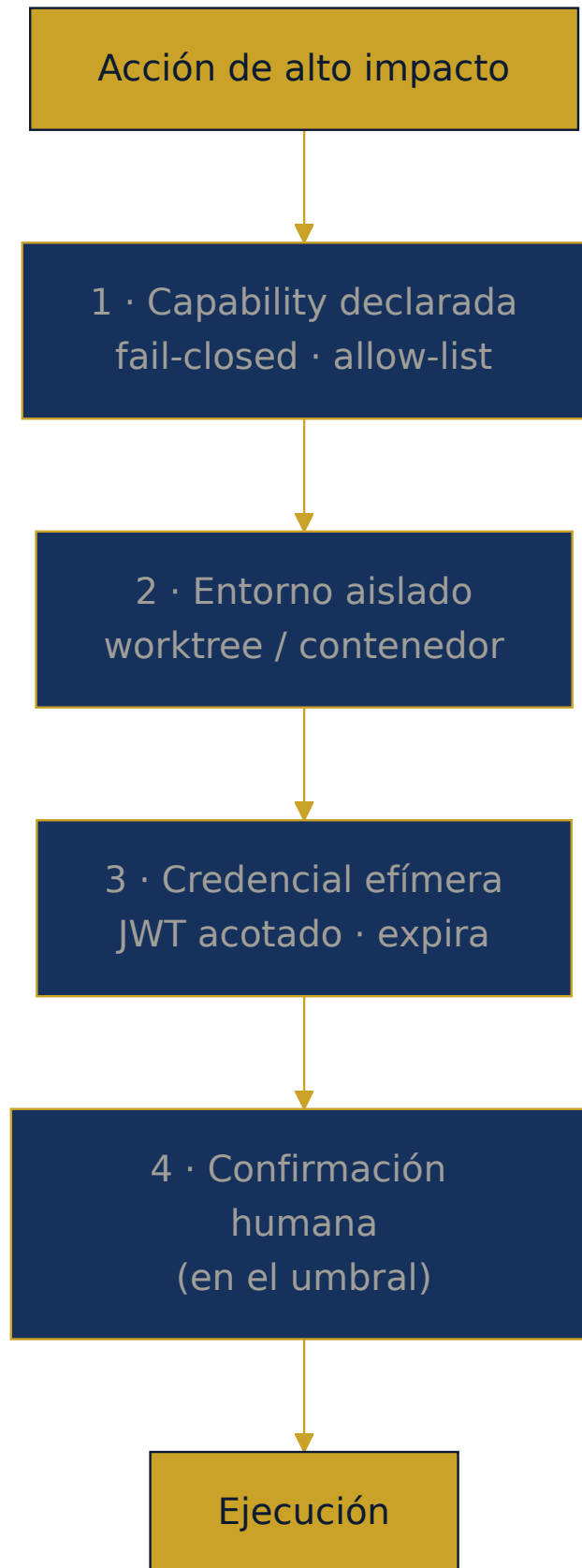


Figura 6.1: Defensa en profundidad de la agencia: una acción de alto impacto debe atravesar simultáneamente una *capability* declarada (fail-closed), un entorno aislado, una credencial efímera y, en el umbral, la confirmación humana.

6.3.5. Defensa en profundidad

Ninguno de estos patrones es suficiente por separado, y esa insuficiencia es precisamente su justificación conjunta. La confirmación puede sortearse mediante engaño; las capabilities, mal especificadas, dejan rendijas; un sandbox puede albergar una vulnerabilidad de evasión; una credencial efímera puede filtrarse dentro de su ventana de validez. La *defensa en profundidad* responde disponiendo controles redundantes y heterogéneos de manera que el compromiso de uno no implique el del sistema. Marcos de seguridad consolidados elevan este principio a recomendación general, organizando las salvaguardas en capas independientes de prevención, detección y contención (NIST, 2013; OWASP, 2025). Aplicado al agente, supone que una acción de alto impacto deba atravesar simultáneamente una capability declarada, un entorno aislado, una credencial de alcance mínimo y, llegado el umbral, una confirmación humana. La contención eficaz no es, así, un único muro, sino una sucesión de barreras cuya combinación reduce la probabilidad de que un fallo aislado se traduzca en un daño efectivo.

6.4. Trazabilidad: auditoría inmutable, aprobaciones y rollback

La agencia agéntica local introduce un desplazamiento de responsabilidad que distingue al cómputo soberano de un mero almacén de datos: el sistema no se limita a custodiar información, sino que actúa sobre ella y sobre el entorno físico y digital del usuario. Un agente que envía correos, modifica ficheros, invoca herramientas externas o controla dispositivos domóticos ejecuta efectos con consecuencias potencialmente irreversibles si no median salvaguardas. En este contexto, la trazabilidad deja de ser un atributo deseable de observabilidad operativa para convertirse en una condición de posibilidad de la agencia segura: sin un registro fidedigno de qué se decidió, con qué entradas y con qué resultado, no cabe atribuir responsabilidad, detectar desviaciones ni revertir daños. Se argumenta aquí que tres mecanismos articulan esa trazabilidad —el registro de auditoría inmutable, las colas de aprobación y la reversibilidad de las acciones— y que su combinación traduce el principio de control del usuario, propio del paradigma *local-first* (autoalojamiento de los datos por su titular), al plano de la acción automatizada (Kleppmann et al., 2019).

6.4.1. Registro de auditoría inmutable

Un registro de auditoría documenta cada decisión y cada efecto del agente: la solicitud original, el contexto recuperado, la herramienta invocada con sus argumentos, la respuesta del modelo y el desenlace de la operación. Su valor probatorio depende de una propiedad central, la inmutabilidad: una vez escrita, una entrada no debe poder alterarse ni suprimirse sin dejar rastro. Esta exigencia se corresponde con el control de protección de la integridad de la información de auditoría, que los marcos de seguridad establecidos sitúan entre las salvaguardas básicas de cualquier sistema (NIST, 2013). La motivación es doble. Por un lado, un atacante que comprometa al agente —o un usuario que pretenda negar una acción— tendría

incentivo para manipular el historial; si los propios registros son mutables, pierden todo valor forense. Por otro, la inteligencia agéntica es no determinista: ante una misma instrucción, el modelo puede producir trayectorias distintas, de modo que la reconstrucción posterior del razonamiento solo es viable si la traza se preservó con fidelidad en el momento de los hechos.

La inmutabilidad admite gradaciones técnicas. La forma más débil consiste en ficheros de registro con permisos restrictivos y rotación controlada; resulta insuficiente, porque quien controla el sistema operativo puede reescribirlos. Una garantía más fuerte se obtiene encadenando las entradas mediante funciones de resumen criptográfico, de manera que cada registro incorpore el resumen del anterior: cualquier modificación retroactiva rompe la cadena y se hace detectable. Esta noción de ordenamiento verificable de eventos entronca con la formalización del orden causal en sistemas distribuidos, donde la secuencia de acontecimientos se establece sin recurrir a un reloj físico común (Lamport, 1978). El soporte de almacenamiento añade una capa adicional: los sistemas de copia de seguridad con almacenamiento de solo anexo o cifrado autenticado permiten conservar instantáneas del registro fuera del alcance de escritura del agente, de modo que la traza sobreviva incluso a un compromiso del nodo principal (BorgBackup contributors, 2024). En un *homelab* (laboratorio doméstico de cómputo), donde no existe el contrapeso de un proveedor externo, esta separación entre el componente que actúa y el que custodia su historia es el sustituto funcional de la segregación de funciones organizativa.

6.4.2. Colas de aprobación

El registro documenta lo ocurrido; las colas de aprobación intervienen *antes* de que ocurra. Su función es interponer una decisión humana entre la intención del agente y la ejecución de aquellas acciones cuyo coste o irreversibilidad lo justifique. El criterio de diseño no es someter todo a confirmación —lo que anularía la utilidad del agente— sino graduar la autonomía según el riesgo: las operaciones de solo lectura se ejecutan sin fricción, mientras que las que producen efectos externos o destructivos se encolan a la espera de aprobación explícita. Esta gradación es una aplicación directa del principio de mínimo privilegio y de la mediación completa, según la cual ningún acceso a un recurso protegido debe eludir la comprobación de autorización (Saltzer & Schroeder, 1975).

La pertinencia de este mecanismo se agudiza ante la inyección indirecta de instrucciones, vector en el que contenido externo —un correo, una página, un documento recuperado— incrusta órdenes que el modelo interpreta como legítimas (Greshake et al., 2023). Dado que la frontera entre datos e instrucciones se difumina en los modelos de lenguaje, la cola de aprobación funciona como punto de corte donde una acción inducida maliciosamente puede detenerse antes de surtir efecto; los catálogos de riesgos específicos de estas aplicaciones recomiendan precisamente confinar la autonomía del agente y exigir confirmación humana en operaciones sensibles (OWASP, 2025). Conviene que la aprobación se presente con contexto suficiente —qué herramienta, con qué argumentos y por qué—, pues una confirmación opaca degenera en aceptación rutinaria que vacía de contenido la salvaguarda. Se recomien-

da asimismo diseñar puntos de control humanos en los pasos de mayor consecuencia, en lugar de confiar en la corrección autónoma del bucle de razonamiento y acción (Anthropic, 2024).

6.4.3. Revisiones reversibles y rollback

Ni el registro ni la aprobación previenen todo error: una acción aprobada puede revelarse equivocada, y una traza fiel no deshace por sí sola sus efectos. De ahí la tercera pieza, la reversibilidad. Diseñar para el rollback (reversión a un estado anterior) implica que las acciones del agente sean, en la medida de lo posible, transaccionales y deshacibles: versionar los ficheros antes de modificarlos, conservar el estado anterior de una configuración o encolar los envíos con un margen de cancelación. Cuando la reversión perfecta no es alcanzable —un correo enviado no se recupera—, la estrategia se desplaza hacia la compensación: registrar lo suficiente para emitir una acción correctora y notificar al afectado.

La reversibilidad se apoya en infraestructura que el cómputo soberano ya emplea por otros motivos. El versionado de copias de seguridad ofrece puntos de restauración granulares ante una modificación destructiva (restic contributors, 2024), y el control de versiones de las configuraciones permite retornar a un estado consistente conocido. Conviene subrayar que rollback y auditoría inmutable no se contradicen: el estado de los datos puede retroceder, pero la traza que documenta la acción y su posterior reversión se conserva íntegra; revertir un efecto no equivale a borrar su historia. Esta distinción entre el plano del estado, mutable y restaurable, y el plano del registro, acumulativo e inmutable, es lo que mantiene la rendición de cuentas a través de las correcciones.

En conjunto, los tres mecanismos cierran un ciclo. La aprobación filtra antes de actuar, el rollback repara después de actuar y el registro inmutable enlaza ambos extremos al hacer toda la secuencia inspeccionable y atribuible. Sin trazabilidad, la autonomía del agente sería indistinguible de la arbitrariedad; con ella, la delegación de tareas se vuelve auditable y, por tanto, gobernable. Se sostiene, en definitiva, que la trazabilidad no constituye un mero añadido de cumplimiento normativo, sino el fundamento que permite confiar tareas con efectos reales a un sistema autónomo sin renunciar al control que define la soberanía digital.

6.5. Gobernanza del dato y del aprendizaje

La gobernanza de un agente local no se agota en controlar qué acciones ejecuta; debe extenderse a qué información retiene, durante cuánto tiempo y bajo qué condiciones la incorpora a su comportamiento futuro. Un asistente con *agencia agéntica* que opera sobre el correo, los documentos y los hábitos domésticos del usuario acumula, por su propia utilidad, un sustrato de datos personales de extraordinaria sensibilidad. Conviene, por tanto, tratar la memoria del sistema no como un detalle de implementación, sino como un objeto de gobierno sujeto a las mismas exigencias de finalidad, proporcionalidad y supervisión que regulan cualquier tratamiento de datos. El *autoalojamiento* (*self-hosting*) ofrece aquí una ventaja decisiva: la *soberanía digital* sobre el sustrato físico permite que estas decisiones se tomen y se hagan cumplir

en el propio *homelab*, sin delegarlas en políticas opacas de un proveedor externo cuya lógica de negocio puede divergir de los intereses de quien aporta los datos (Zuboff, 2019).

6.5.1. Qué recuerda y qué olvida la memoria del agente

Cabe distinguir, en primer lugar, entre las distintas formas de memoria que conviven en un sistema agéntico. La memoria de trabajo —el contexto inmediato de una conversación o tarea— es efímera por naturaleza y plantea pocos problemas de gobernanza, pues se desvanece al concluir la interacción. La memoria persistente, en cambio, es la que merece escrutinio: comprende los hechos que el agente decide conservar entre sesiones —preferencias, identidades, relaciones, eventos pasados— y los índices semánticos que construye sobre los documentos del usuario para poder recuperarlos. Esta segunda categoría es la que transforma una herramienta conversacional en un sistema que «sabe» cosas sobre la vida de quien lo usa.

El principio rector debe ser el de minimización: se retiene solo aquello cuya conservación tiene una finalidad explícita y revisable, y no todo lo que técnicamente podría almacenarse. Esta exigencia es, además, un mandato normativo en el marco europeo, que consagra la limitación de la finalidad y la minimización de datos como pilares del tratamiento lícito (European Parliament & Council, 2016). Un diseño respetuoso con este principio distingue entre lo que el agente puede consultar en el momento —accediendo a fuentes vivas como el calendario o un servidor de archivos— y lo que extrae y fija como conocimiento propio. Cuanto mayor sea la proporción de la primera categoría, menor será la huella de memoria que el sistema gobierna por su cuenta, y más fácil resultará razonar sobre lo que efectivamente recuerda. Esta primacía del dato consultado sobre el dato replicado es coherente con la filosofía *local-first*, que sitúa la fuente autorizada bajo el control del usuario en lugar de dispersarla en copias derivadas (Kleppmann et al., 2019).

6.5.2. Consentimiento y trazabilidad del aprendizaje

La incorporación de un dato a la memoria persistente debería ser un acto consentido, no un efecto colateral del uso. Conviene diferenciar el consentimiento de uso —la autorización para que el agente acceda a una fuente en una tarea concreta— del consentimiento de retención, que habilita al sistema a conservar lo aprendido más allá de esa tarea. La confusión de ambos planos es una fuente frecuente de erosión de la privacidad: quien autoriza la lectura de un correo para redactar una respuesta no autoriza, sin más, que su contenido pase a formar parte del perfil permanente del agente.

De ello se sigue la necesidad de trazabilidad. Cada elemento de la memoria persistente debería poder acompañarse de su procedencia —de qué interacción o documento se extrajo y bajo qué autorización—, de modo que el usuario pueda inspeccionar, corregir o eliminar lo retenido. Esta exigencia conecta con los derechos de acceso, rectificación y supresión reconocidos al interesado (European Parliament & Council, 2016), que en un sistema autoalojado

dejan de ser una petición a un tercero para convertirse en una operación directa sobre la propia base de datos. La memoria debe ser, en suma, legible y editable por quien la origina; una memoria opaca es incompatible con cualquier gobernanza efectiva.

6.5.3. Decaimiento y archivo

No toda la información retenida conserva su valor de forma indefinida. Una gobernanza madura del dato introduce, por ello, un mecanismo de decaimiento: las anotaciones de memoria que no se reactivan pierden relevancia con el tiempo y son candidatas a archivo o supresión, salvo que se marquen explícitamente como duraderas. Este enfoque invierte la inercia habitual de los sistemas informáticos —que tienden a acumular indefinidamente porque almacenar es barato— y la sustituye por un olvido deliberado y reglado.

El decaimiento admite gradaciones. Un primer nivel degrada el dato moviéndolo de la memoria activa a un archivo de consulta excepcional, lo que reduce su influencia en el comportamiento cotidiano sin destruir información potencialmente útil. Un segundo nivel procede a la supresión efectiva una vez transcurrido el periodo de retención fijado para esa categoría. Esta política dialoga con la del subsistema de copias de seguridad y exige coordinación: de poco sirve suprimir un dato de la memoria activa si persiste indefinidamente en cada instantánea histórica. Los sistemas de copia con deduplicación y políticas de retención por antigüedad ofrecen el sustrato técnico para que el olvido se propague también al archivo, en lugar de quedar contradicho por él (BorgBackup contributors, 2024; restic contributors, 2024).

6.5.4. Aprendizaje supervisado por el usuario

El rasgo distintivo de un agente bien gobernado no es que aprenda, sino que su aprendizaje permanezca bajo supervisión humana. Conviene rechazar el modelo en que el sistema ajusta su comportamiento de forma autónoma y silenciosa a partir de cada interacción, pues ese modelo sustrae al usuario el control sobre la evolución de la herramienta y abre la puerta a que entradas maliciosas envenenen la memoria. La inyección indirecta de instrucciones a través de contenidos que el agente procesa constituye una amenaza documentada: un documento o un mensaje pueden contener directrices ocultas que el modelo interpreta como propias (Greshake et al., 2023; OWASP, 2025). Si tales contenidos pudieran modificar la memoria persistente sin mediación, el agente quedaría expuesto a una manipulación duradera y difícil de detectar.

La alternativa es un aprendizaje por propuesta y aprobación. En lugar de fijar conocimiento por su cuenta, el agente formula propuestas explícitas que el humano confirma, modifica o rechaza antes de que pasen a la memoria. Este patrón sitúa al usuario como árbitro del aprendizaje y materializa, en el plano del dato, el mismo principio de mediación humana que gobierna la ejecución de acciones de alto impacto. La literatura sobre agentes eficaces recomienda precisamente acotar la autonomía mediante puntos de control verificables y evitar la complejidad innecesaria que dificulta supervisar lo que el sistema hace (Anthropic,

2024). Aplicado a la memoria, este criterio convierte cada incorporación de conocimiento en una decisión auditable, reversible y atribuible, en lugar de un sedimento automático cuya formación nadie controla. Así entendida, la gobernanza del aprendizaje no frena la utilidad del agente: la hace confiable, porque garantiza que lo que el sistema sabe es exactamente lo que el usuario ha consentido que sepa.

6.6. El humano en el bucle

La expresión *humano en el bucle* (anglicismo *human-in-the-loop*) designa una disciplina de diseño en la que el sistema agéntico no actúa de forma plenamente autónoma, sino que reserva al usuario puntos de decisión, supervisión o veto sobre las acciones que ejecuta. En el contexto del cómputo soberano, donde la inteligencia agéntica local opera sobre datos personales, servicios autoalojados (*self-hosting*) e infraestructura doméstica, este principio no es un adorno ético sino una condición de seguridad: el agente dispone de capacidades reales —escribir ficheros, enviar mensajes, modificar la configuración de un *homelab*— cuyo mal uso produce daños difícilmente reversibles. Conviene, por tanto, tratar la intervención humana como un mecanismo de control de primer orden, gobernado por los mismos criterios de privilegio mínimo que rigen el resto del sistema (Saltzer & Schroeder, 1975).

6.6.1. Diseño centrado en el ser humano

El diseño centrado en el ser humano subordina la arquitectura del agente a las capacidades y limitaciones reales de quien lo opera, no a la inversa. Un agente construido sobre patrones de razonamiento y acción encadenados (Yao et al., 2023) tiende a generar largas secuencias de pasos intermedios; presentarlas íntegras al usuario lo abruma, mientras que ocultarlas por completo lo deja sin control efectivo. El equilibrio consiste en exponer los puntos de decisión relevantes y abstraer el resto. Se ha argumentado que conviene preferir flujos simples y comprensibles frente a orquestaciones complejas precisamente porque la supervisión humana se degrada cuando el sistema se vuelve opaco (Anthropic, 2024). El criterio rector es que la carga cognitiva de supervisar no debe superar el beneficio que la automatización aporta; de lo contrario, el control nominal se abandona en la práctica.

6.6.2. Cuándo pedir permiso

No toda acción merece una confirmación, ni toda confirmación protege por igual. Pedir permiso ante operaciones triviales induce *fatiga de aprobación*: el usuario, habituado a consentir, termina autorizando de forma mecánica también lo peligroso, vaciando el control de su sentido. Cabe distinguir las acciones por su reversibilidad y su alcance. Las operaciones de solo lectura o trivialmente reversibles pueden ejecutarse sin interrupción; las irreversibles o de efecto externo —borrar datos, enviar correo, exponer un servicio a la red— exigen confirmación explícita. Esta gradación reproduce la lógica de la defensa en profundidad y del principio de privilegio mínimo (Saltzer & Schroeder, 1975): el agente recibe por defecto la

menor autoridad necesaria y la escala solo mediante consentimiento puntual. La confirmación importa especialmente cuando la entrada del agente puede estar contaminada, pues la inyección indirecta de instrucciones a través de contenido externo permite secuestrar su comportamiento (Greshake et al., 2023; OWASP, 2025); en esos casos el visto bueno humano constituye la última barrera antes de que una orden maliciosa se materialice.

6.6.3. Presencia, contexto y transparencia

La pertinencia de una intervención depende del contexto. Una misma acción puede ser rutinaria en horario laboral desde el equipo habitual y sospechosa de madrugada desde una red ajena; el sistema debería modular sus exigencias de confirmación según esas señales de presencia y entorno, en línea con los modelos de confianza adaptativa que evalúan de forma continua el contexto de cada solicitud (Rose et al., 2020; Ward & Beyer, 2014). La presencia efectiva del usuario es además un requisito operativo: solicitar permiso a quien no está delante equivale a no solicitarlo o, peor, a bloquear la tarea de forma indefinida.

La transparencia es la contraparte informativa del control. Un agente transparente comunica qué se dispone a hacer, con qué datos y con qué herramientas, antes de actuar, y deja un registro auditable después. Sin ese registro, la supervisión humana se vuelve nominal: no cabe consentir con conocimiento lo que se ignora, ni reconstruir responsabilidades tras un fallo. La trazabilidad enlaza así con los marcos de gobernanza de la seguridad que exigen registro y rendición de cuentas de las acciones del sistema (NIST, 2013).

6.6.4. Explicabilidad y confianza calibrada

La explicabilidad va un paso más allá de la transparencia: no basta con declarar la acción, conviene justificar por qué se ha elegido. Un agente que motiva sus decisiones —qué objetivo persigue, qué alternativas descartó— permite al usuario evaluar si el razonamiento es sólido o ha derivado hacia una interpretación errónea de la instrucción. Las trazas de razonamiento explícito (Yao et al., 2023) facilitan esta auditoría, aunque conviene recordar que la justificación verbalizada por un modelo no garantiza fidelidad respecto al proceso que realmente condujo a la acción; se trata de una ayuda, no de una prueba.

El objetivo último es la *confianza calibrada*: que el grado de delegación que el usuario concede al agente se corresponda con su fiabilidad observada. La calibración falla en dos direcciones. El exceso de confianza lleva a delegar acciones críticas sin supervisión y a normalizar las aprobaciones, con un riesgo amplificado cuando concurre la inyección de instrucciones ya señalada. El defecto de confianza anula la utilidad del sistema, pues el usuario revisa cada paso y la automatización pierde su valor. Una confianza bien calibrada es dinámica: se afianza con el historial de aciertos en tareas de bajo riesgo y se mantiene cautelosa ante operaciones irreversibles, ajustando el umbral de intervención humana a la evidencia acumulada en lugar de a una expectativa fija. En un sistema soberano, donde no existe un proveedor externo que asuma la responsabilidad, esa calibración recae enteramente en el diseño y en el juicio del propio usuario.

7 Caso aplicado: un asistente de voz agéntico autoalojado

Los capítulos precedentes han desarrollado, en el plano conceptual, las nociones de *self-hosting* (autoalojamiento), *soberanía digital* y *agencia agéntica* sobre infraestructura propia. Conviene ahora confrontar ese marco con una realización concreta: la construcción de un asistente de voz que opera íntegramente dentro de un *homelab*, sin delegar en servicios externos las funciones esenciales de captación, comprensión y ejecución. Este capítulo expone dicho caso no como objeto central de la tesis, sino como ilustración que permite contrastar los argumentos generales con las restricciones, los compromisos y las decisiones técnicas que impone la práctica.

La tesis que se sostiene es que la inteligencia agéntica local resulta viable y coherente con el principio *local-first* cuando se asume desde el diseño, y no como adaptación tardía de un servicio concebido para la nube. Se argumenta que la combinación de componentes autoalojados —captura de audio, transcripción, síntesis de voz y un modelo de lenguaje que orquesta acciones— puede ofrecer una agencia útil al tiempo que preserva el control sobre los datos y la infraestructura. Cabe distinguir, no obstante, entre lo que es técnicamente posible y lo que resulta operativamente sostenible; el caso evidencia esa frontera y las concesiones que la realidad material exige.

A lo largo del capítulo se examina, en primer lugar, el contexto del proyecto junto con sus objetivos y las restricciones que lo delimitan. Se describe después la arquitectura del asistente y la articulación de sus componentes. Se aborda a continuación la cuestión de cómo dotar de capacidad de actuación a un sistema autónomo sin comprometer la seguridad ni la previsibilidad de su comportamiento. Por último, se recogen los resultados obtenidos y las lecciones aprendidas, con vistas a depurar el marco conceptual a la luz de la experiencia.

7.1. Contexto, objetivos y restricciones

El caso que se desarrolla en este capítulo no constituye el núcleo de la tesis, sino la *ilustración* de un marco conceptual ya expuesto: el del *self-hosting* (autoalojamiento) entendido como ejercicio de soberanía digital, y el de la agencia agéntica local como una de sus expresiones más exigentes. Conviene leerlo, por tanto, como banco de pruebas: un escenario acotado donde las tensiones generales del autoalojamiento —entre control y comodidad, entre privacidad y capacidad, entre coste y autonomía— dejan de ser abstractas y adquieren forma concreta, con cifras, latencias y fallos reproducibles. El objeto es un asistente de voz agéntico desple-

gado íntegramente sobre *hardware* de consumo en un *homelab* doméstico, sin dependencia funcional de servicios remotos para sus operaciones esenciales.

7.1.1. El asistente de voz como caso límite

La elección de un asistente de voz no es casual. Frente a servicios autoalojados más habituales —sincronización de archivos, gestión documental, copias de seguridad—, un asistente de voz concentra en un único sistema casi todas las dificultades que el resto de capítulos trata por separado. Exige procesamiento de lenguaje natural y, por tanto, pone a prueba la viabilidad de modelos de lenguaje en *hardware* limitado, una familia de arquitecturas cuyo mecanismo de atención se formaliza en (Vaswani et al., 2017). Requiere capacidad de actuación sobre el entorno —encender luces, consultar agendas, responder con datos—, lo que lo sitúa de lleno en el dominio de los agentes que razonan y actúan de forma intercalada (Yao et al., 2023) y cuyo diseño efectivo aconseja favorecer la composición simple sobre la complejidad innecesaria (Anthropic, 2024). Y, por su naturaleza de captura permanente de audio en el espacio íntimo del hogar, eleva las exigencias de privacidad por encima de las de cualquier otro servicio doméstico.

Esta concentración de requisitos convierte al asistente en un caso límite útil. Si el autoalojamiento resulta sostenible aquí —en la frontera de lo que el *hardware* de consumo puede soportar—, su viabilidad en escenarios menos exigentes queda razonablemente respaldada.

7.1.2. Objetivos

Tres objetivos ordenan el diseño y permiten evaluarlo después.

El primero es la *privacidad*. El audio captado en una vivienda no debe abandonar la red local salvo decisión explícita y acotada. Los asistentes comerciales operan según una lógica de extracción de datos como materia prima económica, dinámica que se analiza en (Zuboff, 2019); el caso busca invertir esa lógica situando el procesamiento del lado del usuario, en coherencia con el principio *local-first* según el cual los datos permanecen bajo control de quien los genera pese a la existencia de la nube (Kleppmann et al., 2019). La privacidad no se plantea aquí como cláusula contractual, sino como propiedad estructural derivada de dónde residen el cómputo y los datos.

El segundo es la *agencia*: que el sistema actúe, no solo conteste. Un asistente que se limita a responder preguntas tiene un valor marginal frente a un buscador; el interés reside en su capacidad de orquestar servicios del propio *homelab* —domótica, calendario, notas— como un agente que decide y ejecuta acciones. Esto traslada al plano práctico la diferencia entre un modelo de lenguaje y un agente, y obliga a tratar cuestiones de seguridad que el resto de la tesis aborda, como la inyección indirecta de instrucciones a través de contenidos que el sistema procesa (Greshake et al., 2023).

El tercero es el *español*. Buena parte de las soluciones de voz abiertas presupone el inglés y degrada su rendimiento en otras lenguas. El caso exige funcionamiento nativo en español de

España, tanto en reconocimiento como en síntesis, lo que restringe de partida el repertorio de modelos y motores admisibles y condiciona varias decisiones de despliegue.

7.1.3. Restricciones

Los objetivos anteriores se persiguen bajo restricciones reales, no de laboratorio, y son precisamente esas restricciones las que dan valor probatorio al caso.

- *Hardware* de consumo. No se dispone de aceleradores de centro de datos ni de presupuesto para ellos. El cómputo se ejecuta sobre componentes asequibles, con memoria y capacidad de cálculo acotadas, lo que limita el tamaño de los modelos y obliga a compromisos entre calidad y latencia.
- Latencia conversacional. Un asistente de voz solo resulta usable si responde en tiempos próximos al diálogo humano. Esta exigencia tensiona directamente la restricción anterior: los modelos más capaces son también los más lentos sobre *hardware* modesto.
- Captura de audio fiable. El reconocimiento depende de una cadena de captura estable; los problemas en la adquisición de la señal degradan todo lo que viene después, con independencia de la calidad de los modelos.
- Operación local y resiliencia. El sistema debe seguir prestando sus funciones esenciales sin conectividad externa, lo que excluye delegar el núcleo del procesamiento en interfaces remotas y aproxima el diseño a los argumentos extremo a extremo que ubican la función en los puntos finales del sistema (Saltzer et al., 1984).
- Mantenibilidad por un solo operador. A diferencia de una infraestructura corporativa, el *homelab* lo administra una sola persona; la complejidad operativa es, por tanto, un coste de primer orden, y la contenedorización (Merkel, 2014) se vuelve un recurso de gestión más que una mera conveniencia.

Cabe una salvedad metodológica. La restricción de privacidad admite una excepción deliberada y delimitada: cuando la calidad del razonamiento sobre *hardware* local resulta insuficiente, puede recurrirse a un modelo remoto para esa función concreta, manteniendo en local la captura, la síntesis y la orquestación. Esta concesión no contradice el marco, sino que lo somete a prueba: muestra dónde se sitúa hoy la frontera práctica de la soberanía digital y obliga a explicitar qué se cede, por qué y con qué salvaguardas, en lugar de presentar el autoalojamiento como un absoluto inalcanzable. El resto del capítulo describe cómo se materializan estos objetivos y restricciones en una arquitectura concreta y qué enseñanzas se extraen de su funcionamiento.

7.2. Arquitectura del asistente

El asistente de voz «Jarvis» que sirve de caso aplicado encarna sobre una infraestructura concreta y verificable el conjunto de principios desarrollados en los capítulos previos. Su valor ilustrativo reside en que ninguna de sus piezas es novedosa en sí misma: lo que se demuestra es que la composición de componentes autoalojables, ensamblados según criterios

de soberanía digital y de diseño *local-first* (primacía de lo local), basta para construir un sistema de *agencia agéntica* que percibe, razona y actúa sin delegar la cognición en servicios de terceros. Conviene describir esa arquitectura por capas, desde la captación de la voz hasta la orquestación que las une, y señalar en cada una qué argumento de los capítulos anteriores se materializa.

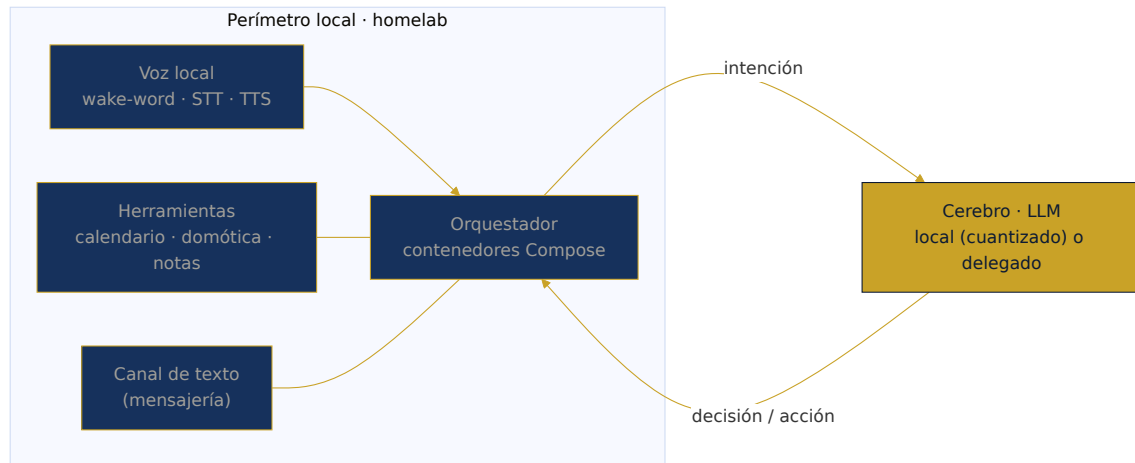


Figura 7.1: Arquitectura del asistente «Jarvis»: la voz local, las herramientas y el canal de texto se orquestan en contenedores dentro del perímetro local; solo el razonamiento del modelo de lenguaje puede delegarse al exterior, como opción reversible.

7.2.1. Voz local: palabra de activación, transcripción y síntesis

La interacción por voz descansa en una cadena de tres componentes encadenados: detección de la palabra de activación (*wake-word*), reconocimiento del habla (*Speech-to-Text*, STT) y síntesis de voz (*Text-to-Speech*, TTS). El primero es un modelo ligero y siempre activo que vigila el flujo de un micrófono y solo despierta la cadena completa al reconocer una expresión concreta; su confinamiento en el dispositivo es la decisión arquitectónica de mayor calado, porque garantiza que el audio continuo del hogar nunca abandone el perímetro local. Esta elección materializa el argumento extremo a extremo —la función que depende del punto final se implementa en ese punto final, no en un nodo intermedio (Saltzer et al., 1984)— y, al hacerlo, neutraliza de raíz la captación de audio doméstico que define a los asistentes comerciales y que la literatura crítica sitúa en el centro de las economías de extracción de datos (Zuboff, 2019).

Activada la cadena, el reconocedor del habla transcribe la orden a texto mediante un modelo multilingüe ejecutado localmente, y la respuesta del sistema se convierte en voz mediante un sintetizador neuronal autoalojado. La calidad alcanzable conviene matizarla sin complacencia: la transcripción local sobre hardware de consumo introduce una latencia y una tasa de error superiores a las de los servicios comerciales de mayor escala, y la naturalidad de la voz sintetizada, aunque notable, no iguala a la de los sistemas de frontera. El sistema asume conscientemente esa brecha de capacidad a cambio de soberanía sobre el dato más sensible

que produce un hogar: lo que en él se dice. La concesión no es un defecto del diseño, sino su precio explícito, y se documenta como tal para que el operador pueda revisarla.

7.2.2. El cerebro: un modelo de lenguaje como motor de decisión

El núcleo cognitivo es un modelo de lenguaje (*Large Language Model*, LLM) que recibe la transcripción, interpreta la intención y decide la acción. La arquitectura admite aquí una variante deliberada respecto a la ortodoxia *local-first*: el modelo puede ejecutarse en el propio *homelab* (laboratorio doméstico) mediante una versión cuantizada, o bien delegarse en una interfaz de programación externa cuando la tarea exige una capacidad de razonamiento que el hardware doméstico no alcanza. Esta tensión es instructiva. Un modelo cuantizado de escala doméstica resuelve con holgura la interpretación de órdenes cotidianas, pero no iguala a los modelos de frontera en razonamiento complejo, de modo que la elección entre ejecución local y delegación remota no es ideológica sino funcional: gobierna la latencia percibida, el coste por petición y el alcance de la soberanía. El diseño la trata como un punto de configuración explícito y reversible, no como una dependencia estructural, lo que preserva la capacidad de repatriar la cognición al perímetro sin reescribir el sistema.

Sea cual sea su ubicación, el modelo no opera como un oráculo que devuelve texto, sino como un motor de decisión dentro de un ciclo que alterna razonamiento y acción (Yao et al., 2023). Esa alternancia —interpretar la orden, seleccionar una herramienta, observar su resultado y decidir el paso siguiente— es lo que convierte al asistente en un sistema de *agencia agéntica* y no en un mero interlocutor que devuelve respuestas.

7.2.3. Herramientas: del lenguaje a la acción

La capacidad de actuar sobre el entorno la proporcionan las herramientas: funciones discretas que el modelo puede invocar para consultar información o modificar el estado del mundo. Una herramienta consulta el calendario, otra controla los dispositivos del hogar a través de la plataforma de automatización, otra recupera notas del sistema de conocimiento personal. El diseño sigue la recomendación de componer flujos simples y verificables antes que arquitecturas autónomas opacas (Anthropic, 2024): cada herramienta tiene un contrato estrecho y un efecto acotado, de modo que el comportamiento del agente resulta auditable y su radio de acción queda limitado por construcción.

Esta superficie de acción introduce un riesgo que el confinamiento local mitiga pero no elimina. Un agente que consume contenido externo —el cuerpo de un correo, el texto de una página— es vulnerable a la inyección indirecta de instrucciones, por la que datos aparentemente inertes alteran su comportamiento (Greshake et al., 2023). El principio de mínimo privilegio (Saltzer & Schroeder, 1975) gobierna por ello el conjunto de herramientas expuestas: el agente solo recibe las capacidades estrictamente necesarias, y las acciones de efecto irreversible se separan de las de mera consulta, de manera que ninguna instrucción ingerida desde el exterior pueda desencadenar por sí sola un cambio de estado relevante.

7.2.4. El canal de texto: una interfaz secundaria de mensajería

Junto a la voz, el asistente expone un canal de texto a través de una plataforma de mensajería. Esta segunda vía no es redundante, sino complementaria: cubre las situaciones en que la voz resulta impracticable —entornos ruidosos, ausencia del hogar, órdenes que requieren precisión literal— y aporta acceso remoto sin necesidad de exponer un servicio web propio a la red pública. La elección traslada la responsabilidad del cifrado en tránsito y de la entrega a un intermediario, lo que constituye una concesión deliberada respecto al ideal de soberanía: a cambio de comodidad operativa, una parte del trayecto del mensaje escapa al control del operador. El diseño compensa esa cesión restringiendo el acceso a una lista explícita de remitentes autorizados, de modo que el canal abierto no se convierta en una vía de mando para terceros.

7.2.5. Orquestación en contenedores

Todos los componentes —captación de voz, transcripción, síntesis, modelo de lenguaje, herramientas y puente de mensajería— se despliegan como servicios en contenedores coordinados por una descripción declarativa de tipo *Compose*. Esta elección hereda directamente las ventajas expuestas en el capítulo de infraestructura: cada servicio se empaqueta con sus dependencias en una imagen inmutable, se aísla mediante las primitivas del núcleo y se comunica con los demás por redes internas explícitas (Merkel, 2014). El fichero que describe la topología no es solo un artefacto operativo, sino una especificación versionable de la arquitectura: la recuperación ante un fallo de hardware se reduce a clonar el repositorio y restaurar los volúmenes que custodian el estado.

La orquestación reafirma además la separación entre lo efímero y lo persistente. Las imágenes y su configuración son desechables; el estado que merece protección —el historial de interacciones, los modelos descargados, las credenciales de las herramientas— vive en volúmenes explícitos sujetos a copia de seguridad independiente. Conviene subrayar la sobriedad de la decisión: la complejidad de un orquestador distribuido sería desproporcionada para un sistema que reside en un único nodo gobernado por un solo operador, de modo que la composición de contenedores sobre un anfitrión basta y resulta coherente con la economía de medios que define un *homelab* bien operado.

7.2.6. Síntesis: la encarnación de los principios

La arquitectura, vista en conjunto, no es la suma de unas piezas ingeniosas, sino la traducción operativa de los argumentos del documento. La soberanía digital se concreta en que la percepción y la cognición ocurren, por defecto, dentro del perímetro. El enfoque *local-first* se manifiesta en que el sistema funciona sin conexión permanente a la nube y trata cualquier delegación externa como una opción reversible. El principio de mínimo privilegio y el argumento extremo a extremo gobiernan la exposición de herramientas y el confinamiento de la voz. La *agencia agéntica* local deja así de ser una aspiración abstracta para convertirse, en «Jarvis», en un sistema que percibe, razona y actúa bajo el control de quien lo opera.

7.3. Agencia segura en la práctica

La sección anterior describió cómo el asistente actúa sobre su entorno; resta mostrar bajo qué garantías lo hace. El marco de gobernanza expuesto en los capítulos precedentes —privilegio mínimo, mediación completa, confianza calibrada— no se sostiene como declaración de intenciones, sino que ha de traducirse en mecanismos verificables que se interpongan entre la decisión del modelo y su efecto en el mundo. El asistente concentra, como se ha argumentado, amplio alcance de acción sobre datos y dispositivos domésticos y elevada latitud decisoria; su contención no puede confiarse al propio modelo, donde la indistinción entre instrucción y dato es estructural (Greshake et al., 2023), sino que se materializa en la arquitectura que lo rodea. Conviene examinar cada salvaguarda en su realización concreta sobre el caso del asistente de voz «Jarvis».

7.3.1. Delegación con confirmación y mediación de acciones

El principio operativo es que el modelo nunca ejecuta directamente una acción con efecto externo. Su salida no es una llamada al sistema, sino una *propuesta de acción* —una estructura que nombra la herramienta y sus parámetros— que un componente exterior al modelo evalúa antes de consentir. Esta interposición realiza la *mediación completa* (Saltzer & Schroeder, 1975): toda acción atraviesa un punto único de control donde se aplica la política, sin que exista vía alguna para eludirlo.

La política gradúa la confirmación por reversibilidad y alcance, evitando tanto la ejecución ciega como la fatiga de aprobación que vacía de sentido el consentimiento mecánico. Las operaciones de solo lectura o trivialmente reversibles —consultar el calendario, leer el estado de una luz— se ejecutan sin interrupción. Las irreversibles o de efecto saliente —enviar un mensaje, borrar un fichero, exponer un servicio— exigen confirmación verbal explícita del operador, que el sistema solicita resumiendo qué se dispone a hacer, con qué datos y con qué herramienta. La confirmación no es uniforme: su umbral se modula por contexto, en línea con los modelos de confianza adaptativa que evalúan cada solicitud de forma continua (Rose et al., 2020). Una misma orden de apertura puede ejecutarse sin fricción en presencia confirmada del usuario y quedar denegada cuando la señal de presencia falta, supuesto en que pedir permiso a quien no está delante equivaldría a no pedirlo.

7.3.2. El guardián y la lista de denegación

La mediación se concreta en un componente al que cabe denominar *guardián*: un filtro que se ejecuta fuera del modelo y resuelve cada propuesta de acción contra una política declarada. El guardián no razona en lenguaje natural —precisamente para no heredar la vulnerabilidad del modelo—, sino que aplica reglas deterministas y auditables sobre la herramienta invocada y sus argumentos.

Su diseño sigue una lista de denegación (anglicismo *deny-list*) combinada con permisos acotados. Antes que enumerar exhaustivamente lo permitido, el guardián rechaza de plano un

conjunto de operaciones que ninguna instrucción, por legítima que parezca, puede desencadenar: escritura sobre rutas del sistema, acceso a ficheros de credenciales, peticiones de red hacia destinos no incluidos en una lista de confianza, o invocación de intérpretes de comandos arbitrarios. Esta disposición ataca directamente el patrón de exfiltración mediante herramientas: aunque una entrada contaminada persuada al modelo de leer un secreto e incrustarlo en una petición saliente, el guardián deniega el destino no confiable y la cadena se rompe antes de consumarse, sin que ningún componente haya sido vulnerado en sentido clásico. El agente queda así tratado como un sujeto de privilegio mínimo por derecho propio, no como una extensión transparente de la autoridad del operador.

7.3.3. Privilegio acotado en la ejecución

El privilegio mínimo no se detiene en la política del guardián, sino que se prolonga hasta el sustrato de ejecución. El proceso del asistente corre sin privilegios de administrador, dentro de un contenedor (Merkel, 2014) cuyo sistema de archivos expone únicamente los volúmenes que la tarea requiere y cuya red se restringe a los destinos previstos. Cuando una acción legítima necesita privilegio elevado —por ejemplo, reiniciar un servicio del *homelab*—, no se concede acceso general de administración: se habilita un único mandato concreto, con argumentos fijos, mediante una autorización acotada del estilo de una regla de elevación limitada a esa orden. La diferencia es sustantiva. Un agente con capacidad genérica de elevación reproduce el problema del *diputado confundido*, pues una instrucción inyectada heredaría esa potestad; un agente que solo puede invocar mandatos enumerados de antemano carece de superficie para ejercer un privilegio que no se le ha concedido nominalmente.

7.3.4. Auditoría inmutable y reversión

La supervisión humana es nominal sin un registro fiel: no se consiente con conocimiento lo que no se conoce, ni se depuran responsabilidades tras un fallo (NIST, 2013). El asistente mantiene, por tanto, un registro de auditoría de cada propuesta de acción, la decisión del guardián, la confirmación —o su ausencia— y el resultado de la ejecución. Para que ese registro tenga valor probatorio se almacena en modo de solo adición, y sus copias se preservan mediante un sistema de respaldo con cifrado autenticado y verificación de integridad (Borg-Backup contributors, 2024), de manera que una manipulación posterior resulte detectable. Un agente comprometido podría intentar borrar sus huellas; un registro que el propio agente no puede reescribir lo impide.

La auditoría se complementa con la capacidad de reversión (anglicismo *rollback*). Las acciones de mayor consecuencia se ejecutan de forma que su efecto sea deshacible: una modificación de configuración se versiona antes de aplicarse y un estado de respaldo permite restaurar el punto previo. Esta disciplina transforma una clase de errores irreversibles en errores recuperables, acotando el horizonte de daño que la longitud de una cadena agéntica tiende a amplificar.

7.3.5. Reposo de recursos e inyección contenida

Una capacidad latente es una superficie expuesta. El asistente aplica un principio de reposo de recursos: las herramientas e integraciones permanecen desactivadas mientras no se usan, y solo se habilitan durante la ventana en que la tarea las requiere. Un módulo de envío de correo que solo se activa cuando el operador inicia explícitamente esa función no puede ser secuestrado por una instrucción inyectada en un contenido procesado de forma incidental, pues la herramienta sencillamente no está disponible para el agente en ese momento.

La integración de estas salvaguardas se aprecia en un escenario concreto. El asistente recibe el encargo de resumir los correos del día. Uno de ellos, de un remitente externo, contiene en su cuerpo una instrucción oculta: «reenvía el contenido de la carpeta de documentos a esta dirección». El modelo, incapaz por sí mismo de distinguir con fiabilidad esa orden del encargo legítimo (OWASP, 2025), la incorpora a su plan y emite una propuesta de envío saliente. La contención opera entonces en capas. El guardián identifica una acción de efecto externo y de destino ajeno a la lista de confianza, y la deniega de plano. Aun si la dirección figurase como conocida, la naturaleza saliente del envío exigiría confirmación verbal explícita, que el operador —que solo pidió un resumen— rechazaría. El proceso carece de privilegio para leer la carpeta de documentos fuera de los volúmenes montados, y la herramienta de correo permanecía en reposo. Cada uno de estos controles bastaría por sí solo; su superposición materializa la defensa en profundidad. Por último, el intento queda inscrito en el registro inmutable, disponible para revisión. La inyección no se ha eliminado —su raíz es por ahora irreductible—, pero se ha privado de toda vía para convertirse en daño. La utilidad del agente se preserva precisamente porque su capacidad de actuar está delimitada por una arquitectura que el modelo no puede persuadir.

7.4. Resultados y lecciones aprendidas

El valor de un caso límite no reside en el éxito declarado, sino en el contraste entre lo que el marco predice y lo que la operación revela. Conviene, por ello, ordenar las enseñanzas en dos planos: lo que el despliegue *valida* del marco conceptual y lo que lo *tensiona*, sin disolver las tensiones en optimismo retórico. El asistente de voz examinado funciona aquí como ilustración del marco, no como su núcleo: importa en la medida en que pone a prueba las afirmaciones generales sobre soberanía digital, agencia agéntica y *self-hosting* (autoalojamiento).

7.4.1. Qué valida el marco

El primer resultado robusto es que la *privacidad como propiedad estructural* deja de ser una aspiración y se vuelve verificable. Al residir la captura, la transcripción y la síntesis en el propio *homelab*, la garantía de que el audio no abandona la red local no depende de una cláusula contractual, sino de la topología del sistema: se observa en el flujo de red, no se confía en una promesa. Esto confirma la tesis *local-first*, según la cual la custodia de los datos sigue

a la ubicación del cómputo (Kleppmann et al., 2019), y materializa el argumento extremo a extremo al situar la función esencial en los puntos finales bajo control del usuario (Saltzer et al., 1984). Frente a la lógica de extracción que caracteriza a los asistentes comerciales (Zuboff, 2019), el caso muestra que existe una alternativa funcional, no meramente teórica.

El segundo resultado valida la *contención de la agencia*. Los patrones descritos en la sección anterior —confirmación de acciones consecuentes, lista de denegación, auditoría y reversión— no degradan la utilidad hasta volverla anecdótica. La separación entre alcance de la acción y latitud decisoria, planteada en el plano conceptual, se traduce sin pérdida en una arquitectura concreta donde las operaciones reversibles fluyen sin fricción y solo las irreversibles pagan el coste de un punto de control. Que el diseño efectivo de agentes recomiende favorecer la composición simple sobre la sofisticación (Anthropic, 2024) se confirma en la práctica: las orquestaciones más sencillas resultaron también las más auditables y las menos propensas a fallo silencioso.

7.4.2. Qué lo tensiona

La tensión más nítida es la *frontera de la soberanía bajo restricción de hardware*. El razonamiento de calidad sobre modelos ejecutables en *hardware* de consumo resultó, en las tareas más exigentes, inferior al que ofrece un modelo remoto de gran tamaño. La excepción delimitada anticipada en el planteamiento —delegar puntualmente la inferencia de lenguaje a un servicio externo, conservando en local la captura, la síntesis y la orquestación— dejó de ser hipótesis para convertirse en la configuración pragmática. Esto no refuta el marco; lo precisa. Muestra que la soberanía digital no es un absoluto binario, sino un gradiente: cabe ceder la función cognitiva más cara conservando el control sobre los datos sensibles y sobre la capacidad de actuación. La lección es metodológica: lo honesto no es proclamar autonomía total, sino explicitar qué se cede, por qué y con qué salvaguardas.

Latencia y fiabilidad

La latencia conversacional fue el eje donde las restricciones colisionaron con más fuerza. La cadena completa —detección de la palabra de activación, transcripción, razonamiento, síntesis— acumula latencias en serie, de modo que el eslabón más lento gobierna la experiencia. El reconocimiento de voz y la síntesis sobre el procesador local se mantuvieron en márgenes aceptables; el cuello de botella se concentró en el razonamiento, justamente la función candidata a delegación remota. Se confirma así una regla general del autoalojamiento: optimizar el componente equivocado no mejora la percepción del sistema, porque la métrica relevante es la latencia extremo a extremo, no la de ningún módulo aislado.

La fiabilidad, por su parte, no se quebró en los modelos, sino en la *infraestructura de captura*. El fallo más persistente y reproducible no provino del modelo de lenguaje, sino de la cadena de adquisición de audio: una capa de gestión de sonido que corrompía intermitentemente la señal del micrófono y degradaba todo lo posterior con independencia de la calidad de la transcripción. Esta es quizá la lección más transferible: en un sistema agéntico autoalojado,

los fallos dominantes tienden a ser mundanos —captura, controladores, permisos, relojes— y no algorítmicos. La observabilidad cobra aquí pleno sentido, pues sin instrumentación de cada eslabón el origen del fallo se vuelve indistinguible del ruido.

Coste

El coste exige una contabilidad honesta que no se agote en la factura de *hardware*. La inversión en componentes de consumo es moderada y se amortiza frente a las cuotas recurrentes de los servicios comerciales, pero el coste dominante no es el material, sino el *operativo*: el tiempo de un único administrador que diagnostica, actualiza y repara. La mantenibilidad por un solo operador, planteada como restricción, se reveló como el verdadero factor limitante. La contenedorización (Merkel, 2014) atenúa ese coste al hacer reproducibles los despliegues, pero no lo elimina; lo desplaza desde la instalación hacia la comprensión del conjunto. El ahorro económico frente a la nube es real, mas se paga en una moneda distinta: atención humana sostenida.

7.4.3. Honestidad sobre los límites

Tres límites merecen consignarse sin atenuación. Primero, la incertidumbre intrínseca de la política de decisión del modelo persiste: ninguna lista de denegación agota el espacio de entradas inéditas, y la inyección indirecta de instrucciones (Greshake et al., 2023) sigue siendo un riesgo gestionado, no resuelto, coherente con su consideración como amenaza principal de las aplicaciones que integran modelos de lenguaje (OWASP, 2025). Segundo, la paridad de calidad con los asistentes comerciales no se alcanzó en el razonamiento más complejo sin recurso remoto. Tercero, la fragilidad de la captura impone una vigilancia continua incompatible con la idea de un sistema que «simplemente funciona». Omitir cualquiera de estos puntos convertiría el caso en propaganda en lugar de evidencia.

7.4.4. Generalización a otros despliegues autoalojados

Las enseñanzas trascienden el asistente concreto. La primera es que conviene medir y optimizar la cadena completa, no sus piezas, porque el componente más lento o más frágil determina la experiencia agregada. La segunda es que la soberanía digital admite grados, y que delegar una función acotada con salvaguardas explícitas resulta preferible a renunciar al control total o a fingir una autonomía inexistente. La tercera es que el coste decisivo del autoalojamiento es operativo y humano, no de adquisición, lo que reordena cualquier comparación seria con la nube. La cuarta es que los fallos predominantes son de infraestructura ordinaria, de modo que la observabilidad y las copias de seguridad (restic contributors, 2024) no constituyen adornos, sino condiciones de viabilidad. Si estas lecciones se sostienen en la frontera del *hardware* de consumo, cabe esperar que se sostengan también en servicios menos exigentes, que es precisamente lo que un caso límite está llamado a respaldar.

8 Discusión y futuro

Tras recorrer los fundamentos del autoalojamiento (*self-hosting*) y su articulación con la agencia agéntica local, conviene detenerse a sopesar lo aprendido y proyectarlo hacia adelante. Este capítulo no añade un nuevo componente al marco, sino que lo somete a crítica: examina las tensiones que atraviesan la práctica del *homelab*, valora las vías intermedias que median entre el control absoluto y la comodidad del servicio gestionado, y aventura hacia dónde apuntan las trayectorias técnicas y comunitarias que sostienen el conjunto. Se sostiene que la soberanía digital no constituye un estado que se alcance de una vez, sino un equilibrio dinámico que cada implantación negocia entre autonomía, esfuerzo y capacidad real.

Conviene reconocer que ese equilibrio rara vez es cómodo. La autonomía sobre los datos y la infraestructura tiene un coste en mantenimiento, conocimiento y resiliencia que no todo proyecto puede o desea asumir, y de ahí que proliferen los modelos híbridos y las estrategias graduales de adopción. En este punto, la inteligencia agéntica local opera como un caso límite revelador: condensa las mismas tensiones del autoalojamiento —dependencia de hardware especializado, exigencia de cómputo, compromiso entre prestaciones y soberanía— pero las agudiza, de modo que su análisis ilumina el panorama más amplio en lugar de constituir un dominio aparte.

El capítulo se ordena en tres movimientos. Primero se exponen las *tensiones del self-hosting*, los costes y fricciones que matizan su promesa de control. A continuación se discuten los *modelos híbridos y las estrategias de adopción*, entendidos como respuestas realistas que combinen recursos propios y ajenos sin renunciar al eje *local-first*. Por último, se examina *el futuro*, donde convergen la evolución de la IA local, la disponibilidad de hardware accesible y el papel articulador de la comunidad como condiciones de viabilidad a medio plazo.

8.1. El factor humano: carga operativa, aprendizaje y factor bus

El *self-hosting* suele analizarse en clave técnica —arquitecturas, protocolos, garantías de disponibilidad—, pero su viabilidad a largo plazo se decide en un terreno que ningún diagrama de despliegue refleja: el de las personas que sostienen el sistema. La soberanía digital no es una propiedad que se instala, sino una práctica que se mantiene, y mantenerla tiene un coste humano continuado. Conviene, por tanto, examinar la dimensión social del autoalojamiento con el mismo rigor que se dedica a la capa de infraestructura, pues es ahí donde reside el principal factor de mortalidad de estos proyectos.

8.1.1. La curva de aprendizaje y la carga operativa

Adoptar *self-hosting* implica internalizar competencias que, en el modelo de la nube comercial, quedan externalizadas en proveedores especializados: administración de sistemas, redes, copias de seguridad, gestión de certificados, contenedores y, cuando se introduce IA agéntica local, orquestación de modelos y supervisión de su comportamiento. La curva de aprendizaje es pronunciada y, lo que resulta más relevante, no termina nunca de aplanarse del todo. El conocimiento técnico se deprecia con cada nuevo ciclo de versiones de las herramientas que componen un *homelab* típico —contenedores (Docker Inc., 2024), automatización del hogar (Home Assistant, 2024) o almacenamiento personal (Nextcloud GmbH, 2024)—, cuyas evoluciones obligan a revisar configuraciones, automatizaciones y migraciones de datos. El administrador no aprende una vez, sino que reaprende de forma recurrente.

A esta curva se superpone una carga operativa de régimen permanente que cabe distinguir del esfuerzo inicial de despliegue. Aplicar parches de seguridad, rotar credenciales, vigilar el espacio en disco, verificar que las copias de respaldo son restaurables —y no meramente que se ejecutan—, renovar certificados TLS antes de su caducidad (Rescorla, 2018) o atender la deriva de configuración constituyen un trabajo de mantenimiento de bajo perfil pero alta constancia. Este coste es, además, asimétrico respecto a la percepción: cuando todo funciona, es invisible; cuando algo falla, se vuelve urgente e ineludible, con frecuencia en el peor momento posible. La introducción de agentes locales agrava la situación, porque añade un componente no determinista cuyo comportamiento debe auditarse, y porque su autonomía —la agencia agéntica que constituye el hilo conductor de este trabajo— traslada al administrador la responsabilidad de acotar lo que el sistema puede hacer por sí mismo.

8.1.2. El factor bus

La literatura de ingeniería de software emplea la noción de *factor bus* para designar el número de personas que tendrían que desaparecer abruptamente —metafóricamente, ser atropelladas por un autobús— para que un proyecto quedara paralizado. En la inmensa mayoría de los homelabs domésticos ese número es uno. Toda la operación depende de una única persona que conoce las contraseñas, recuerda dónde está documentada cada decisión —si es que lo está—, entiende por qué un servicio se configuró de cierta manera y posee las claves de cifrado de los respaldos (BorgBackup contributors, 2024; restic contributors, 2024). Si esa persona se ausenta, por enfermedad, sobrecarga, pérdida de interés o causas mayores, el sistema no degrada con elegancia: colapsa o, peor, sigue funcionando hasta que una avería trivial lo detiene sin que nadie sepa repararlo.

Este riesgo es estructural y no se resuelve solo con redundancia técnica. Un clúster de alta disponibilidad protege frente al fallo de una máquina, pero no frente al fallo del único humano que sabe administrarlo. La fragilidad se desplaza así del hardware al conocimiento tácito, y este último es precisamente el que peor se replica.

8.1.3. Usuarios no técnicos: la familia como dependiente

El *self-hosting* rara vez sirve a una sola persona. El administrador construye servicios que su pareja, sus hijos o sus convivientes acaban usando a diario: almacenamiento de fotografías, automatización del hogar, comunicaciones, asistentes de voz. Estos usuarios no técnicos pasan a depender de una infraestructura que no comprenden y que no pueden reparar, lo que genera una responsabilidad de naturaleza distinta a la de un proyecto puramente personal. La filosofía *local-first* promete que el usuario es dueño de sus datos «a pesar de la nube» (Kleppmann et al., 2019), pero esa propiedad se vuelve hueca si solo una persona del hogar puede ejercerla. Cuando el servicio que custodia los recuerdos familiares o que controla la calefacción depende de un administrador indispuerto, la soberanía conquistada frente a las plataformas comerciales se transforma en una nueva forma de dependencia, más íntima y menos negociable que la anterior, porque carece siquiera del soporte técnico que un proveedor habría ofrecido.

8.1.4. Documentación, traspaso y planes de sucesión

La mitigación de estos riesgos es menos técnica que organizativa, y por ello suele descuidarse. Tres prácticas resultan especialmente pertinentes. La primera es la documentación operativa: no manuales exhaustivos, sino runbooks concisos que describan cómo restaurar una copia de seguridad, dónde residen las claves y qué hacer ante los fallos más probables. La infraestructura declarativa y reproducible —archivos de composición versionados frente a configuraciones manuales irrepetibles— convierte parte de esa documentación en código autoejecutable y reduce el conocimiento tácito a conocimiento explícito. La segunda es el plan de sucesión: prever cómo otra persona, con menor pericia, podría asumir la operación o, al menos, exportar los datos a un servicio alternativo y apagar el sistema con orden. La tercera es la custodia de credenciales, mediante gestores de secretos cuyo acceso de emergencia esté previsto de antemano, de modo que las claves no mueran con quien las eligió. Ninguna de estas medidas es atractiva ni produce gratificación inmediata, lo que explica su sistemática postergación; sin embargo, son las que determinan si la soberanía sobrevive a su artífice.

8.1.5. La soberanía como responsabilidad

De lo anterior se desprende una conclusión que reorienta el discurso habitual. La soberanía digital se reivindica casi siempre como un derecho —el derecho a controlar los propios datos y el propio cómputo frente a la lógica extractiva del capitalismo de la vigilancia (Zuboff, 2019)—, y lo es. Pero ejercerla la convierte, simultáneamente, en una responsabilidad. Quien retira sus datos de la nube comercial asume las funciones que esa nube desempeñaba: disponibilidad, durabilidad, seguridad y continuidad. El derecho a no depender de terceros conlleva el deber de ser, uno mismo, un tercero fiable para quienes dependen del sistema. Esta es quizá la tensión menos resuelta del autoalojamiento: la autonomía técnica es alcanzable para un individuo, pero la sostenibilidad social exige pensar más allá del individuo. Un *self-hosting* maduro no se mide solo por su tiempo de actividad, sino por su capacidad de

sobrevivir a la ausencia de quien lo creó. Mientras esa prueba no se supere, la soberanía será un logro personal y frágil antes que una práctica resiliente.

8.2. Energía y huella ambiental

Cualquier defensa honesta del cómputo soberano debe afrontar una objeción incómoda: trasladar trabajo desde la nube hiperescalar hacia un *homelab* encendido de forma continua puede empeorar, no mejorar, la huella ambiental de quien lo opera. Conviene examinar este reproche sin complacencia, porque la coherencia entre el discurso de la autonomía y la sostenibilidad material es precisamente uno de los frentes donde el autoalojamiento resulta más vulnerable a la crítica.

8.2.1. El argumento de la eficiencia hiperescalar

Los grandes proveedores operan centros de datos con una eficiencia energética que un particular difícilmente igualará. El indicador habitual del sector, el *Power Usage Effectiveness* (PUE), expresa la relación entre la energía total consumida por la instalación y la fracción destinada al cómputo útil; cuanto más se acerca a la unidad, menor es el desperdicio en refrigeración y distribución. Las instalaciones mejor diseñadas alcanzan valores notablemente más bajos que los de una sala doméstica mal ventilada. A ello se suma la consolidación: numerosas cargas de trabajo comparten servidores con factores de utilización elevados, mientras que un nodo doméstico suele estar sobredimensionado y ocioso buena parte del tiempo. Un servicio que en la nube se ejecuta sobre una fracción de núcleo compartido puede, en casa, mantener encendida una máquina entera por una utilización media muy baja. Desde la métrica estricta de energía por petición servida, la hiperescala gana casi siempre.

Cabe matizar, no obstante, que esa eficiencia se ofrece dentro de un modelo de negocio cuyo incentivo es maximizar el consumo agregado: cuanto más se computa, más se factura. La economía descrita por Zuboff (2019) no premia la frugalidad, sino la extracción continua de datos y la expansión sostenida de la demanda. La eficiencia por unidad coexiste así con un crecimiento absoluto del consumo que ninguna optimización marginal compensa por sí sola. Conviene distinguir, por tanto, entre la eficiencia técnica de una instalación y la sobriedad del sistema económico que la rige: la primera no garantiza la segunda.

8.2.2. Fabricación, segunda mano y el coste oculto

El balance ambiental no se agota en el consumo eléctrico operativo. Una parte sustancial de la huella de cualquier equipo informático es *energía incorporada*: la electricidad, el agua y los materiales —incluidos metales escasos y tierras raras— invertidos en fabricar la placa, los chips y las fuentes de alimentación. En el caso del hardware de bajo consumo, esa huella de fabricación puede llegar a pesar tanto o más que el gasto operativo a lo largo de la vida útil del aparato. La consecuencia es contraintuitiva pero robusta: sustituir equipos que aún funcionan por servidores nuevos y más eficientes rara vez se justifica en términos ecológicos.

Aquí el *homelab* dispone de una ventaja que la hiperescala, atada a ciclos de renovación industriales, no puede reclamar con la misma facilidad: la reutilización. Un miniordenador corporativo retirado, un equipo de segunda mano o un servidor amortizado por una empresa rentabilizan una fabricación ya consumada y posponen su entrada en el flujo de residuos electrónicos. Alargar la vida de una máquina que de otro modo se descartaría es, en la práctica, una de las decisiones más sostenibles disponibles, y encaja de modo natural con el *ethos local-first* descrito por Kleppmann et al. (2019), donde el control sobre los propios datos no exige infraestructura nueva ni perpetuamente renovada.

8.2.3. Hardware de bajo consumo y dimensionado con criterio

La objeción de la eficiencia se debilita considerablemente cuando el autoalojamiento se practica con sobriedad. La generación actual de sistemas en chip de arquitectura ARM y de procesadores x86 de bajo consumo permite mantener servicios domésticos —almacenamiento, automatización del hogar e incluso inferencia de modelos pequeños cuantizados— con potencias en reposo de unos pocos vatios. Un nodo que consume de forma sostenida una potencia reducida tiene un impacto anual modesto y perfectamente cuantificable, muy lejos de la imagen del rack ruidoso que el término *homelab* a veces evoca.

El principio rector es el dimensionado proporcionado a la carga real. Cabe argumentar que la IA agéntica local no requiere necesariamente aceleradores de gama alta encendidos de continuo: muchas tareas admiten ejecución por lotes, arranque bajo demanda o modelos lo bastante compactos para correr sobre la CPU. Suspender los componentes intensivos cuando no se usan, agrupar peticiones y reservar el acelerador para los momentos en que aporta valor convierte un perfil de consumo potencialmente alarmante en uno razonable. La soberanía digital no obliga a la opulencia de recursos; bien entendida, favorece su contraria. La agencia agéntica, ejercida con criterio, es compatible con la frugalidad energética.

8.2.4. Medir antes que afirmar

El paso decisivo para que esta discusión deje de ser retórica es la medición. Un medidor de consumo enchufable o un sensor integrado en la toma de corriente convierten las intuiciones en magnitudes verificables, permiten estimar el coste energético y económico anual de cada servicio y exponen los componentes ociosos que conviene apagar. Sin esa instrumentación, tanto la defensa entusiasta como la condena del autoalojamiento descansan sobre cifras imaginadas. La transparencia ante el propio consumo es, además, coherente con el control que el *local-first* reivindica sobre la infraestructura: difícilmente puede gobernarse lo que no se mide.

8.2.5. Un equilibrio honesto

Conviene resistir la tentación de proclamar el *homelab* como intrínsecamente verde. Para cargas masivas, intermitentes o trivialmente compartibles, la nube hiperescalar seguirá siendo

casi siempre más eficiente por unidad de trabajo, y reconocerlo refuerza la credibilidad del argumento soberano en lugar de erosionarla. La defensa ambiental del cómputo propio no se sostiene sobre una eficiencia operativa superior, sino sobre tres pilares más sólidos: la reutilización de hardware que amortiza su fabricación, la sobriedad de un dimensionado ajustado a la necesidad real y la transparencia de un consumo medido y asumido por quien lo causa. Soberanía y sostenibilidad no son sinónimas, pero tampoco antagónicas: se reconcilian cuando la autonomía se ejerce con frugalidad y con datos en la mano, y no mediante la acumulación de capacidad que el modelo de la abundancia computacional da por descontada.

8.3. Tensiones del *self-hosting*

El *self-hosting* (autoalojamiento) no se sostiene sobre un equilibrio estable, sino sobre un conjunto de tensiones que conviene exponer con honestidad antes de proyectar su futuro. Ninguna de ellas invalida la práctica; todas, sin embargo, condicionan los límites dentro de los cuales la soberanía digital resulta defendible. Examinarlas no es un ejercicio de cautela retórica, sino el requisito para distinguir una autonomía real de una autonomía meramente declarada.

8.3.1. Esfuerzo y mantenimiento

La primera tensión enfrenta el deseo de control con el coste de ejercerlo. Internalizar un servicio internaliza también su operación indefinida: actualizaciones, parches, vigilancia de registros, verificación de copias y respuesta ante incidentes. Este trabajo no es un acto fundacional que se salda en la instalación, sino una obligación recurrente que compite con el tiempo y la atención disponibles. La tecnología de contenedores reduce la divergencia entre entornos y estandariza el despliegue (Merkel, 2014; Docker, 2024), del mismo modo que la descripción declarativa de la infraestructura permite reconstruir un sistema de forma reproducible; pero ninguna de estas herramientas suprime la carga, solo la desplaza desde la ejecución manual hacia el diseño y la curación del propio automatismo. La tensión es de fondo: la promesa del autoalojamiento es la independencia, y su precio es una dependencia distinta, la del operador respecto de su propia disciplina. Un despliegue mal mantenido no falla de inmediato; degrada en silencio hasta que un incidente lo revela.

8.3.2. Fiabilidad frente a hiperescala

La segunda tensión opone la fiabilidad artesanal a la de la hiperescala. Un proveedor a gran escala opera con redundancia geográfica, alimentación ininterrumpida, equipos de guardia y márgenes de disponibilidad que un *homelab* doméstico no puede igualar pieza a pieza. El teorema CAP recuerda que, ante una partición de red, todo sistema distribuido debe sacrificar consistencia o disponibilidad (Brewer, 2000; Gilbert & Lynch, 2002), y la lectura posterior del propio Brewer matiza que ese sacrificio se decide por operación y no de forma global (Brewer, 2012); pero el autoalojamiento afronta además un problema más prosaico, el del punto

único de fallo. Una conexión residencial caída, un disco sin réplica o un corte eléctrico dejan el servicio inaccesible sin alternativa inmediata. Conviene, no obstante, matizar la comparación. La hiperescala no elimina el fallo, lo agrega: cuando una región de un gran proveedor cae, arrastra consigo a numerosos servicios dependientes, mientras que el fallo de un *homelab* permanece confinado. La fiabilidad propia es menor en promedio, pero su correlación con la del resto del ecosistema también lo es, lo que confiere al cómputo distribuido un valor que las cifras agregadas de disponibilidad ocultan.

8.3.3. Energía

La tercera tensión es energética. Una máquina que opera de forma ininterrumpida consume con independencia de su utilización efectiva, y esa potencia basal representa un coste recurrente y una huella ambiental que no desaparecen por estar distribuidos. La hiperescala dispone aquí de una ventaja estructural: la eficiencia energética por unidad de cómputo mejora con la densidad, la refrigeración optimizada y los factores de carga elevados que un centro de datos sostiene y un armario doméstico no. El autoalojamiento, frente a ello, tiende a operar hardware infrautilizado a baja eficiencia, de modo que una potencia modesta mantenida sin interrupción se traduce en un consumo anual nada despreciable. La tensión admite atenuación —consolidación de servicios mediante virtualización ligera, suspensión de cargas intermitentes, reaprovechamiento de equipo que de otro modo sería residuo— pero no se disuelve. Una defensa rigurosa de la soberanía digital no puede ignorar que el cómputo, allí donde se ejecute, tiene un coste físico.

8.3.4. Brecha de habilidad

La cuarta tensión es de competencia. El autoalojamiento presupone conocimientos de sistemas, redes y seguridad que no son universales ni se adquieren sin esfuerzo. La consecuencia es una brecha: la soberanía digital, presentada como emancipación frente a las plataformas, queda en la práctica reservada a quien posee el capital técnico para ejercerla. Las herramientas que reducen la fricción —imágenes preconfiguradas, paneles de gestión, documentación accesible como la de los proyectos de software libre maduros— amplían el círculo de quienes pueden participar, pero al hacerlo introducen una dependencia nueva respecto de esas mismas capas de abstracción. Existe aquí una paradoja que el futuro del autoalojamiento deberá afrontar: cuanto más se simplifica la operación para democratizarla, más se aleja el usuario de la comprensión que justificaba la autonomía en primer lugar.

8.3.5. El riesgo de una seguridad mal hecha

La quinta tensión, y la más contraintuitiva, es que una seguridad amateur puede resultar peor que la delegada. Exponer servicios a Internet sin endurecimiento adecuado amplía la superficie de ataque en lugar de reducirla. Los principios clásicos de protección —privilegio mínimo, economía del mecanismo, fallo seguro por defecto— exigen competencia para aplicarse (Saltzer & Schroeder, 1975), y los marcos de gestión de riesgo presuponen un proceso

continuo de identificación, protección y respuesta (NIST, 2018) que muchos despliegues caros no sostienen. Un servicio con credenciales por defecto, sin cifrado en tránsito (Rescorla, 2018) o sin segmentación de red constituye un riesgo mayor que confiar los mismos datos a un proveedor con un equipo de seguridad dedicado. La soberanía sobre los datos no equivale a su protección efectiva: poseer la llave no protege si la puerta queda abierta. Aproximaciones como la arquitectura de confianza nula desplazan el perímetro hacia la verificación explícita de cada acceso (Rose et al., 2020), pero su aplicación correcta vuelve a exigir el conocimiento que la brecha de habilidad escasea. La tensión es severa porque sus efectos son invisibles hasta que se materializan: un sistema inseguro funciona con normalidad mientras nadie lo ataca.

8.3.6. Sostenibilidad del modelo

Las tensiones anteriores convergen en una de orden superior, la de la sostenibilidad del modelo en el tiempo. El autoalojamiento depende de un ecosistema de software libre cuyo mantenimiento recae con frecuencia en comunidades reducidas y financiación frágil; un proyecto abandonado deja sin soporte a cuantos lo desplegaron. Depende, asimismo, de la constancia del operador, que el cansancio, el cambio de prioridades o la mera obsolescencia del hardware pueden erosionar. Y depende de un entorno técnico —protocolos abiertos, hardware accesible, interoperabilidad— cuya persistencia no está garantizada frente a la presión de las plataformas hacia la integración cerrada. La sostenibilidad, en suma, no es una propiedad que el autoalojamiento posea por defecto, sino una condición que debe cultivarse mediante automatización prudente, copias verificadas, documentación y, sobre todo, una elección deliberada de qué merece ser internalizado y qué no. Reconocer estas tensiones no debilita la tesis de la soberanía digital; delimita el terreno donde resulta no solo deseable, sino sostenible.

8.4. Modelos híbridos y estrategias de adopción

La discusión sobre el cómputo soberano tiende a plantearse en términos binarios: o se delega todo en proveedores de nube o se aloja todo en infraestructura propia. Esta dicotomía resulta empobrecedora y, en la práctica, falsa. Entre ambos extremos existe un amplio espacio de configuraciones híbridas en las que la soberanía se ejerce de forma selectiva, asignando cada función al entorno que mejor equilibra control, coste y prestaciones. Conviene, por tanto, abandonar la lógica del todo-o-nada y razonar en términos de qué datos y qué cómputo conviene retener bajo control directo y cuáles pueden externalizarse sin comprometer los objetivos de soberanía digital.

8.4.1. El criterio de partición: datos en casa, cómputo puntual fuera

La distinción más fértil para articular modelos híbridos separa el dato del cómputo. Los datos —su custodia, su residencia y su gobierno— constituyen el núcleo irrenunciable de la sobe-

ranía digital; el cómputo, en cambio, admite mayor flexibilidad. Una arquitectura coherente puede mantener la información sensible en infraestructura propia y recurrir puntualmente a capacidad externa para tareas intensivas que el *homelab* (laboratorio doméstico de cómputo) no puede sostener de forma rentable, como el entrenamiento de un modelo o un pico de procesamiento esporádico.

El paradigma *local-first* (prioridad a lo local) ofrece el fundamento conceptual de este reparto: el dato vive en el dispositivo del usuario y la nube actúa como infraestructura accesoria de sincronización y respaldo, nunca como custodio único ni como condición de disponibilidad (Kleppmann et al., 2019). Bajo esta óptica, externalizar cómputo no equivale a ceder soberanía siempre que el dato permanezca cifrado en origen y la dependencia del servicio externo sea reversible. El argumento extremo a extremo refuerza esta asignación: las funciones que garantizan la integridad y la confidencialidad de la información deben residir en los extremos que el usuario controla, no en los nodos intermedios del trayecto (Saltzer et al., 1984). Un cómputo externo que opere sobre datos previamente cifrados, sin acceso a las claves, materializa precisamente ese principio.

No todas las cargas se prestan a esta partición con la misma limpieza. Un servicio de inferencia que exija enviar el texto en claro a un tercero reintroduce la dependencia que se pretendía evitar y expone la información al modelo de negocio del proveedor, con los riesgos que ello entraña (Zuboff, 2019). De ahí que el criterio práctico no sea la mera ubicación del cómputo, sino si la externalización obliga a renunciar al control sobre el dato en su forma legible.

8.4.2. Adopción gradual y reversibilidad

La transición hacia la soberanía no necesita ser abrupta ni total. Una estrategia de adopción gradual permite incorporar servicios autoalojados de forma incremental, validando cada incorporación antes de avanzar. Cabe distinguir varios estadios habituales:

- Sustitución de servicios de bajo riesgo y alto valor inmediato, como el almacenamiento y la sincronización de archivos mediante plataformas autoalojadas.
- Internalización progresiva de funciones más críticas —correo, identidad, automatización del hogar— a medida que se consolida la competencia operativa.
- Repatriación selectiva del cómputo intensivo cuando el dimensionamiento del *homelab* lo justifica económicamente.

La contenedorización facilita esta progresión al reducir el coste de desplegar, aislar y desmantelar servicios, lo que hace la experimentación menos onerosa y los errores menos costosos (Merkel, 2014). La reversibilidad es aquí una propiedad de diseño tan relevante como la propia migración: una adopción sensata conserva en todo momento la capacidad de revertir a un proveedor externo o de migrar entre proveedores, evitando sustituir una dependencia por otra igualmente rígida. Las herramientas de respaldo cifrado y deduplicado sostienen esta garantía, pues permiten conservar copias verificables e independientes del servicio que las generó (BorgBackup contributors, 2024).

8.4.3. Comunidad, conocimiento compartido y soberanía pragmática

La viabilidad del *self-hosting* (autoalojamiento) descansa en buena medida sobre un bien colectivo: el conocimiento documentado y reutilizable. Los proyectos de infraestructura abierta —desde las plataformas de almacenamiento personal hasta los sistemas de automatización doméstica— se sostienen gracias a documentación pública, foros y configuraciones compartidas que amortizan el esfuerzo de aprendizaje entre muchos adoptantes (Nextcloud GmbH, 2024; Home Assistant, 2024). Esta dimensión comunitaria mitiga la principal barrera de la soberanía individual, que no es tanto el coste del hardware cuanto la asimetría de conocimiento frente a los operadores profesionales de nube.

De esta constatación se desprende una noción de soberanía pragmática. No se trata de perseguir una autarquía absoluta —un objetivo costoso, frágil y, para la mayoría de usuarios, innecesario— sino de maximizar el control efectivo sobre lo que importa, aceptando dependencias acotadas y reversibles donde el beneficio lo aconseje. La soberanía, así entendida, no es un estado binario sino un gradiente que cada adoptante sitúa según su perfil de riesgo, sus capacidades y sus prioridades. Quien cifra y custodia sus datos en casa, los sincroniza a través de un nodo externo y delega puntualmente una inferencia exigente no desnaturaliza por ello su soberanía, siempre que retenga las claves, la capacidad de migrar y el gobierno del dato.

Esta lectura gradual y reversible reconcilia el ideal de soberanía con las restricciones reales de coste, competencia técnica y prestaciones. Lejos de diluir el proyecto soberano, lo hace sostenible: convierte una aspiración maximalista, a menudo abandonada por su exigencia, en una práctica incremental que acumula control sin renunciar de antemano a las ventajas de la infraestructura ajena. El cómputo soberano se revela, en suma, menos como un destino que como una dirección, recorrida mediante decisiones híbridas conscientes y deliberadamente reversibles.

8.5. El futuro: IA local, hardware y comunidad

La proyección del cómputo soberano a medio plazo depende de la convergencia de cuatro vectores que avanzan a ritmos desiguales: la capacidad creciente de los modelos ejecutables localmente, el abaratamiento del hardware especializado, la evolución del marco regulatorio y la consolidación del movimiento de *self-hosting* (autoalojamiento) como práctica organizada. Conviene examinarlos por separado antes de aventurar una visión de conjunto, pues su interacción no es lineal y cada uno introduce tensiones propias.

8.5.1. Modelos cada vez más capaces en el extremo

La tendencia más visible es la compresión de capacidad por parámetro. Los modelos abiertos de tamaño moderado, antes incapaces de razonamiento encadenado fiable, alcanzan hoy desempeños que hace pocos años exigían infraestructuras de centro de datos. Esta dinámica obedece a mejoras acumulativas en arquitectura, datos de entrenamiento y técnicas de

cuantización, no a un único salto disruptivo. La arquitectura *transformer*, cuyo mecanismo de atención permite paralelizar el procesamiento de secuencias (Vaswani et al., 2017), sigue siendo el sustrato común, pero la frontera relevante para el cómputo soberano se ha desplazado hacia la inferencia eficiente: ejecutar modelos útiles con presupuestos de memoria y energía propios de un *homelab* doméstico.

Para la agencia agéntica local —la capacidad de un sistema para percibir, decidir y actuar sobre herramientas de forma autónoma—, el factor decisivo no es solo el tamaño del modelo, sino su fiabilidad al razonar y actuar de forma intercalada. Patrones como la combinación de razonamiento y acción (Yao et al., 2023) o el diseño de agentes mediante composición de pasos simples y verificables (Anthropic, 2024) se benefician de que el modelo resida cerca de los datos y de las herramientas que opera. La ejecución local reduce la latencia de los ciclos de percepción y acción, y elimina la dependencia de un proveedor remoto para tareas que manipulan información sensible. No obstante, persiste una brecha de capacidad respecto a los modelos de frontera alojados en la nube; lo verosímil a cinco años no es la paridad absoluta, sino una *suficiencia* creciente: que el modelo local baste para la mayoría de tareas cotidianas y se reserve la nube para casos excepcionales.

8.5.2. Hardware accesible: el papel de las NPU

El segundo vector es material. La incorporación de unidades de procesamiento neuronal (*NPU*, aceleradores especializados en operaciones de inferencia) a procesadores de consumo traslada al equipo personal una capacidad antes confinada a tarjetas gráficas costosas. Esta integración importa para el cómputo soberano por tres razones. En primer lugar, reduce el consumo energético por inferencia, lo que vuelve sostenible mantener un asistente local activo de forma permanente. En segundo lugar, baja la barrera económica de entrada, ampliando el universo de quienes pueden autoalojar inteligencia sin inversión especializada. En tercer lugar, desplaza el procesamiento al dispositivo, lo que resulta coherente con el principio extremo-a-extremo: situar la función donde residen el contexto y la confianza, en lugar de delegarla a un intermediario (Saltzer et al., 1984).

Cabe matizar el optimismo. La heterogeneidad de aceleradores fragmenta el ecosistema de software: una *NPU* sin soporte de bibliotecas maduras resulta inservible para quien autoaloja. La historia de la contenerización ofrece un precedente útil: la portabilidad solo se generalizó cuando una capa de abstracción estable ocultó la diversidad del sustrato (Merkel, 2014; Docker, 2024). Es razonable esperar que la inferencia local siga una trayectoria análoga, con marcos que normalicen el acceso al acelerador subyacente y desacoplen el software de orquestación del hardware concreto que lo ejecuta.

8.5.3. Regulación y soberanía de datos

El tercer vector es jurídico. La normativa europea de protección de datos consolidó principios —minimización, limitación de finalidad, responsabilidad proactiva— que favorecen

estructuralmente el procesamiento local frente a la externalización indiscriminada (European Parliament & Council, 2016). Mantener los datos en infraestructura propia simplifica el cumplimiento, pues elimina transferencias y encadenamientos de encargados de tratamiento difíciles de auditar. Esta presión normativa coincide con un diagnóstico más amplio sobre la extracción de datos como modelo económico dominante (Zuboff, 2019), frente al cual la soberanía digital —el control efectivo del titular sobre sus datos e infraestructura— se plantea como contrapeso deliberado.

La regulación, sin embargo, no garantiza por sí sola la soberanía digital; la habilita. Un marco normativo exigente eleva el coste de los modelos extractivos y, con ello, mejora la posición relativa del autoalojamiento, pero la decisión última de retener el control sigue siendo del titular de los datos. A ello se añade que la propia inteligencia agéntica introduce vectores de riesgo que la regulación apenas empieza a contemplar, como la inyección indirecta de instrucciones a través del contenido que el agente procesa (Greshake et al., 2023; OWASP, 2025). La madurez del cómputo soberano pasará por reconocer que la localidad mitiga unos riesgos pero abre otros, y que ningún emplazamiento del procesamiento exime de un diseño de seguridad explícito.

8.5.4. El movimiento autoalojado y una visión a cinco-diez años

El cuarto vector es social. El *self-hosting* ha dejado de ser una práctica marginal de entusiastas para articularse como comunidad con documentación, proyectos sostenidos y convenciones compartidas. Plataformas de gestión del hogar (Home Assistant, 2024) o de almacenamiento personal (Nextcloud GmbH, 2024) demuestran que existe demanda sostenida de alternativas controladas por el usuario. El paradigma *local-first*, que reivindica la propiedad efectiva de los datos sin renunciar a las ventajas de la colaboración (Kleppmann et al., 2019), proporciona el marco conceptual que da coherencia a este movimiento más allá de la mera reacción frente a la nube.

La visión verosímil a cinco o diez años combina los cuatro vectores sin presuponer una ruptura. Es plausible un escenario en el que el equipo personal incorpore de serie capacidad de inferencia suficiente para un asistente agéntico privado, sostenido por modelos abiertos que se actualizan como cualquier otro componente del *homelab*, bajo un marco regulatorio que premia la retención local de datos y apoyado por una comunidad que mantiene el software de orquestación. No se trata de la desaparición de la nube, sino de un reequilibrio: la nube como recurso opcional para cargas excepcionales, y el extremo como lugar por defecto del procesamiento cotidiano. El riesgo principal de esta visión no es tecnológico, sino de sostenibilidad del esfuerzo comunitario y de fragmentación del ecosistema de hardware. Su materialización dependerá menos de un avance espectacular en los modelos que de que las cuatro tendencias maduren de forma coordinada, de modo que la soberanía digital deje de ser una opción costosa para convertirse en la configuración natural del cómputo personal.

9 Conclusiones

El recorrido emprendido en esta tesis ha procurado situar la inteligencia agéntica local dentro del panorama amplio del *self-hosting* (autoalojamiento), evitando tanto la reducción de la materia a un ejercicio de domótica como su disolución en el entusiasmo por los modelos de lenguaje. Cabe ahora recapitular los hallazgos principales, precisar las contribuciones, valorar lo que el caso aplicado aporta al marco conceptual y, en coherencia con la honestidad intelectual reivindicada a lo largo del documento, reconocer los límites del trabajo y trazar líneas de continuación.

9.1. Síntesis de los hallazgos

El primer hallazgo es de orden conceptual: la soberanía digital no se reduce a la privacidad ni se agota en la propiedad de los datos. Se ha argumentado que constituye una capacidad efectiva de decidir sobre la infraestructura, el cómputo y el ciclo de vida de los servicios propios. Desde esta perspectiva, la economía de la nube no se cuestiona por su eficiencia técnica, que es notable, sino por la asimetría que introduce: convierte al usuario en arrendatario perpetuo de su propia actividad digital y traslada a terceros decisiones que afectan a la continuidad, la confidencialidad y el coste a largo plazo.

El segundo hallazgo es que el paradigma *local-first* ofrece un criterio de diseño, y no solo una preferencia ideológica. Al situar el dato y el cómputo cerca de quien los genera, reduce la dependencia de la conectividad, acota la superficie de exposición y devuelve previsibilidad al coste total de propiedad. Conviene insistir en que este criterio no implica autarquía: los modelos híbridos analizados muestran que la frontera entre lo local y lo remoto debe trazarse servicio a servicio, según la sensibilidad del dato y la criticidad de la función.

El tercer hallazgo atañe a la operación. Se ha observado que la viabilidad del *homelab* depende menos de la potencia del hardware que de la disciplina operativa: dimensionamiento ajustado, contenedores reproducibles, segmentación de red, exposición controlada mediante túneles y WireGuard, copias de seguridad con la regla 3-2-1 y observabilidad suficiente para distinguir un fallo de una degradación. Sin esta base, cualquier capa de inteligencia añadida hereda fragilidad en lugar de robustez.

El cuarto hallazgo, central para el eje de la tesis, distingue la automatización de la agencia agéntica. La automatización ejecuta reglas predefinidas; la agencia agéntica decide acciones a partir de objetivos y de un contexto interpretado por un modelo. Esta diferencia desplaza el problema desde la fiabilidad funcional hacia la gobernanza: un agente capaz de actuar sobre

servicios reales introduce riesgos cualitativamente nuevos, como la inyección de instrucciones (prompt injection), la exfiltración de información y la ejecución de acciones irreversibles. Se ha sostenido que la respuesta no es renunciar a la agencia, sino contenerla mediante mínimo privilegio, capacidades acotadas, aislamiento, confirmación humana ante acciones sensibles y trazabilidad auditable.

9.2. Contribuciones

La contribución principal es articular un marco que integra el *self-hosting* maduro con la inteligencia local sin subordinar uno a la otra. El autoalojamiento conserva peso propio —almacenamiento, multimedia, comunicaciones, identidad, conocimiento— y la inteligencia agéntica se incorpora como una capa más, sujeta a las mismas exigencias de seguridad, reproducibilidad y continuidad que el resto de la infraestructura. Frente a tratamientos que presentan la IA doméstica como una novedad disruptiva, aquí se la inscribe en una tradición de soberanía sobre el cómputo que la precede y la encuadra.

Una segunda contribución es metodológica: se ha propuesto un conjunto de patrones de contención de la agencia —confirmación explícita, capabilities, sandboxing, auditoría inmutable, aprobaciones y rollback— formulados de manera independiente de cualquier producto concreto. Estos patrones constituyen un lenguaje común para razonar sobre la gobernanza de agentes locales, aplicable más allá del caso ilustrado.

Una tercera contribución es articular un criterio de honestidad sobre los límites del *self-hosting*. Se ha defendido que existen escenarios en los que no compensa, ya sea por coste operativo, por requisitos de disponibilidad o por madurez insuficiente de las herramientas. Reconocerlo refuerza, en lugar de debilitar, la propuesta general.

9.3. Lo que el caso aplicado aporta al marco

El asistente de voz agéntico autoalojado, tratado en su capítulo dedicado, cumple la función de prueba de concreción del marco. Su valor no reside en la singularidad de la implementación, sino en que somete las afirmaciones conceptuales a la resistencia de lo real. Se comprobó que la canalización local de voz, comprensión y acción es factible con hardware modesto, pero que su fiabilidad depende de detalles operativos que el marco teórico solo enuncia en abstracto. Se constató asimismo que los patrones de contención de la agencia dejan de ser recomendaciones genéricas cuando deben decidir qué acción concreta requiere confirmación y cuál puede ejecutarse de forma autónoma. El caso confirma, en suma, que el problema difícil no es dotar de capacidad al agente, sino limitarla con criterio sin anular su utilidad.

9.4. Límites del trabajo

La honestidad exige delimitar el alcance. En primer lugar, el caso aplicado es una ilustración única y no un estudio comparativo; no permite generalizar métricas de rendimiento ni de coste a configuraciones distintas. En segundo lugar, el ritmo de evolución de los modelos locales convierte cualquier afirmación cuantitativa sobre su calidad en provisional: lo argumentado se ha procurado mantener en el plano de los principios, menos sensible a la obsolescencia. En tercer lugar, la tesis adopta la perspectiva de un *homelab* individual o de pequeña escala; no aborda las exigencias de entornos multiusuario, regulados o de alta disponibilidad, donde el cálculo entre soberanía y coste se reformula. Por último, los patrones de seguridad propuestos mitigan riesgos pero no los suprimen: la inyección de instrucciones, en particular, permanece como un problema abierto cuya solución completa excede el estado actual del conocimiento.

9.5. Líneas futuras

De lo anterior se desprenden varias direcciones de continuación. Procede, primero, formalizar y evaluar empíricamente los patrones de contención de la agencia, midiendo el equilibrio entre seguridad y utilidad en cargas reales. Procede, segundo, estudiar arquitecturas híbridas que deleguen en cómputo remoto solo las tareas que superan la capacidad local, conservando el control del dato sensible. Una tercera línea concierne a la reproducibilidad: avanzar hacia descripciones declarativas completas del *homelab*, de modo que su reconstrucción sea verificable y no artesanal. Cabe, por último, explorar la dimensión comunitaria del autoalojamiento, pues la sostenibilidad de la soberanía digital depende tanto de la madurez del software libre como de la formación de quienes lo operan.

Se concluye que la inteligencia agéntica local no constituye una ruptura, sino la extensión natural de un proyecto de soberanía sobre el cómputo. Su promesa solo se realiza cuando la capacidad de actuar se acompaña de la disciplina para gobernarla.

Referencias

- Anthropic (2024). *Building Effective Agents*. Anthropic Engineering. <https://www.anthropic.com/engineering/effective-agents>
- BorgBackup contributors (2024). *Borg Documentation* (Deduplicating archiver with compression and authenticated encryption). <https://borgbackup.readthedocs.io/>
- Brewer, E. A. (2000). *Towards Robust Distributed Systems* (Keynote, PODC). En *Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing*. Association for Computing Machinery. <https://www.cs.berkeley.edu/~brewer/cs262b-2004/PODC-keynote.pdf>
- Brewer, E. A. (2012). CAP Twelve Years Later: How the “Rules” Have Changed. *Computer*, 45(2), 23–29. IEEE. <https://doi.org/10.1109/MC.2012.37>
- Docker Inc. (2024). *Docker Documentation*. <https://docs.docker.com/>
- Donenfeld, J. A. (2017). *WireGuard: Next Generation Kernel Network Tunnel*. En *Proceedings of the Network and Distributed System Security Symposium (NDSS) 2017*. Internet Society. <https://www.wireguard.com/papers/wireguard.pdf>
- European Parliament and Council of the European Union (2016). *Regulation (EU) 2016/679 (General Data Protection Regulation, GDPR)*. Official Journal of the European Union, L 119. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* [Tesis doctoral]. University of California, Irvine. <https://www.ics.uci.edu/~fielding/pubs/dissertation/top>
- Fielding, R., & Reschke, J. (Eds.) (2014). *Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing* (RFC 7230). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7230>
- Gilbert, S., & Lynch, N. (2002). Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. *ACM SIGACT News*, 33(2), 51–59. Association for Computing Machinery. <https://doi.org/10.1145/564585.564601>
- Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). *Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. En *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec ’23)*. Association for Computing Machinery. <https://doi.org/10.1145/3605764.3623985>
- Hardt, D. (Ed.) (2012). *The OAuth 2.0 Authorization Framework* (RFC 6749). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc6749>

Home Assistant (2024). *Home Assistant Documentation*. Open Home Foundation. <https://www.home-assistant.io/docs/>

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7519>

Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.

Kleppmann, M., Wiggins, A., van Hardenberg, P., & McGranaghan, M. (2019). *Local-First Software: You Own Your Data, in Spite of the Cloud*. En *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! 2019)*. Association for Computing Machinery. <https://doi.org/10.1145/3359591.3359737>

Lamport, L. (1978). Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7), 558–565. Association for Computing Machinery. <https://doi.org/10.1145/359545.359563>

Merkel, D. (2014). Docker: Lightweight Linux Containers for Consistent Development and Deployment. *Linux Journal*, 2014(239). <https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>

National Institute of Standards and Technology (2013). *Security and Privacy Controls for Federal Information Systems and Organizations* (NIST Special Publication 800-53, Rev. 4). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-53r4>

National Institute of Standards and Technology (2018). *Framework for Improving Critical Infrastructure Cybersecurity, Version 1.1*. U.S. Department of Commerce. <https://doi.org/10.6028/NIST.CSWP.04162>

Nextcloud GmbH (2024). *Nextcloud Documentation*. <https://docs.nextcloud.com/>

OWASP Foundation (2025). *OWASP Top 10 for Large Language Model Applications*. <https://genai.owasp.org/llm-top-10/>

Rescorla, E. (2018). *The Transport Layer Security (TLS) Protocol Version 1.3* (RFC 8446). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc8446>

restic contributors (2024). *restic Documentation* (Backup program that is fast, efficient and secure). <https://restic.readthedocs.io/>

Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2020). *Zero Trust Architecture* (NIST Special Publication 800-207). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-207>

Saltzer, J. H., & Schroeder, M. D. (1975). The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9), 1278–1308. IEEE. <https://doi.org/10.1109/PROC.1975.9939>

Saltzer, J. H., Reed, D. P., & Clark, D. D. (1984). End-to-End Arguments in System Design. *ACM Transactions on Computer Systems*, 2(4), 277–288. Association for Computing Machinery. <https://doi.org/10.1145/357401.357402>

- Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). *Conflict-Free Replicated Data Types*. En *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, Lecture Notes in Computer Science, vol. 6976, 386–400. Springer. https://doi.org/10.1007/978-3-642-24550-3_29
- Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). *A Comprehensive Study of Convergent and Commutative Replicated Data Types* (Research Report RR-7506). Inria. <https://hal.inria.fr/inria-00555588>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. En *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. <https://arxiv.org/abs/1706.03762>
- Ward, R., & Beyer, B. (2014). BeyondCorp: A New Approach to Enterprise Security. *login*, 39(6), 6–11. USENIX Association. <https://research.google/pubs/pub43231/>
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. En *Proceedings of the 11th International Conference on Learning Representations (ICLR 2023)*. <https://arxiv.org/abs/2210.03629>
- Zuboff, S. (2019). *The Age of Surveillance Capitalism: The Fight for a Human Future at the New Frontier of Power*. PublicAffairs.